kt ds SSAM 강의

AI의 AI에 의한 품질과 테스트



2026. 09 | kt ds 김재환 마이스터

강사 소개 + 참석자 인사

[교육 과정 소개]

교육내용	
[세션 1] **인공지능의 이해**	• 인공지능의 정의와 역사 • 인공지능 진화와 기술적 특이점
[세션 2] **품질과 테스트 이해**	• 품질과 테스트의 관계 • AI 관련 품질/테스트 국제 표준 • AI 기반 시스템의 품질 • 테스트 고려사항 • 테스트 설계 및 설계 사례
[세션 3] **AI 발전과 프로젝트 활용**	• AI 모델 자체 능력의 발전 • AI 활용 능력의 진화 • AI를 활용한 시스템 구축 • AI 시스템 평가 방법 • AICC 사례
[부록 #1] **AI 발전 트랜드**	• AI Agent Framework 현황 • AI Agent 오픈소스 생태계 • 주요 통계 • AI 활용 프로젝트 생산성

Contents

1	인공지능 이해
2	품질과 테스트 이해
3	AI 발전과 프로젝트 활용
부록 #1	AI 발전 트랜드

1. 인공지능(AI) 정의(1/2)

지능

[옥스포드 사전]

- **(사람)** 지식과 기술을 습득하고 적용하는 능력

인공지능

[OECD (2023년 개정, 2024년 발효)]

- 명시적이거나 암묵적인 목표를 위해 입력으로부터 콘텐츠·예측·추천·결정과 같은 출력을 생성하는 방법을 추론하는 **기계 기반 시스템**으로, 이러한 출력이 물리적 또는 가상 환경에 영향을 줄 수 있다.

➢ **EU AI Act (유럽 연합 인공지능법)의 기반**

[ISO/IEC 22989:2022] 인공지능 개념 및 용어 표준

- 주어진 인간 정의 목표(objectives)를 위해 콘텐츠·예측·추천·결정과 같은 출력을 생성하는 **엔지니어링된 시스템**

➢ ISO/IEC 42001(AI 경영시스템), 23894(AI 리스크관리)의 기반

1. 인공지능(AI) 정의(2/2)



인공지능이란?

사람의 지능을 흉내내는 **소프트웨어, 프로그램, 인공적인 장치**

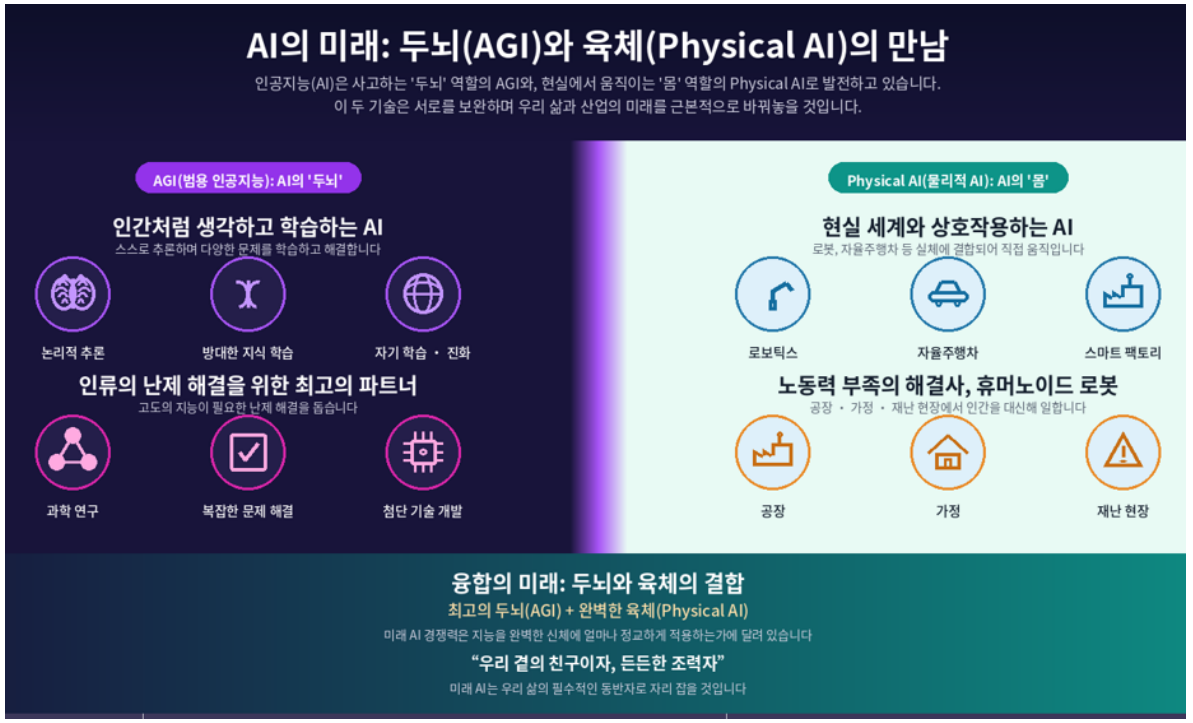
인간의 지각능력



인간의 신체능력



인간의 감각 (5감각)	센서의 종류
시각(빛)	광센서 (카메라)
청각(소리)	음향센서 (마이크)
촉각(온도/습도)	온도/진동/압력 센서
미각(맛)	맛센서
후각(냄새)	냄새센서

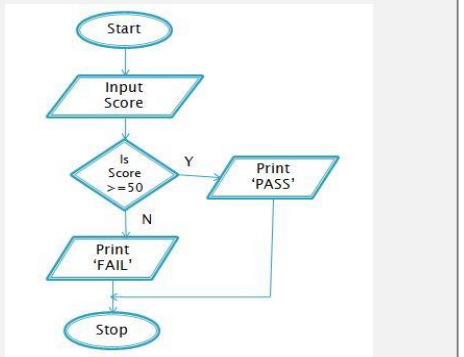


2. 머신러닝과 알고리즘의 차이

일반적인 프로그래밍

Input
(입력값)

알고리즘*주1)
(계산식)



Output
(결과값)

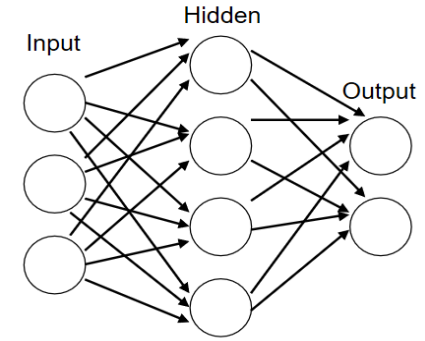
알고리즘과 입력 값을 주어
결과값을 찾음

VS.

머신러닝

Input
(입력값)

Output
(결과값)



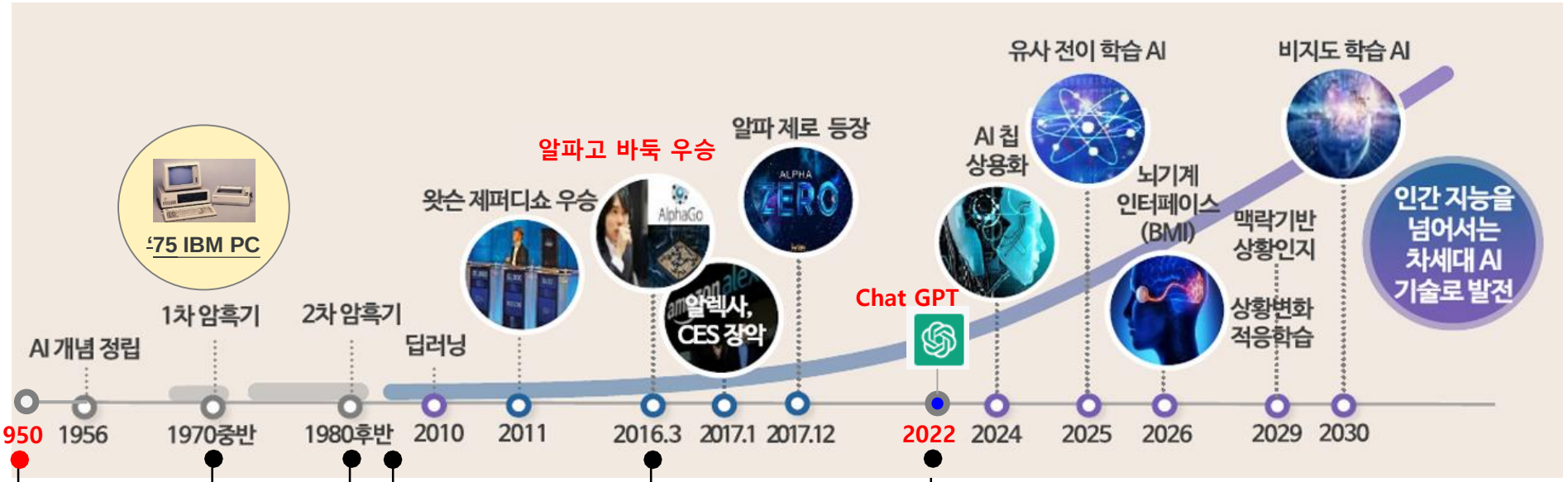
알고리즘
(계산식)

데이터 기반으로 스스로 알고리즘을
찾아내는 방법

※ **알고리즘**이란 어떠한 문제를 해결하기 위한 일련의 절차나 방법을 공식화한 형태**(**조건이나 계산식**)**

3. 인공지능(AI)의 진화 (1/2)

[인공지능 기술의 성장 (출처=과학기술정보통신부)]



튜링 테스트 이미테이션 게임

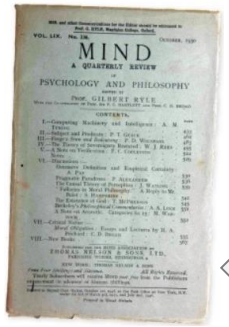
1차 암흑기
'75 PC 등장

AI 성장기
2000년대 들어 컴퓨팅 파워가 성장하고 우수한 알고리즘 등장
스마트폰 보급과 네트워크 발전으로 데이터가 축적되면서 인공지능은 급속히 진보

2차 암흑기
('80년후반 ~ '90년초)
대화 인공지능 등의 개발 실패

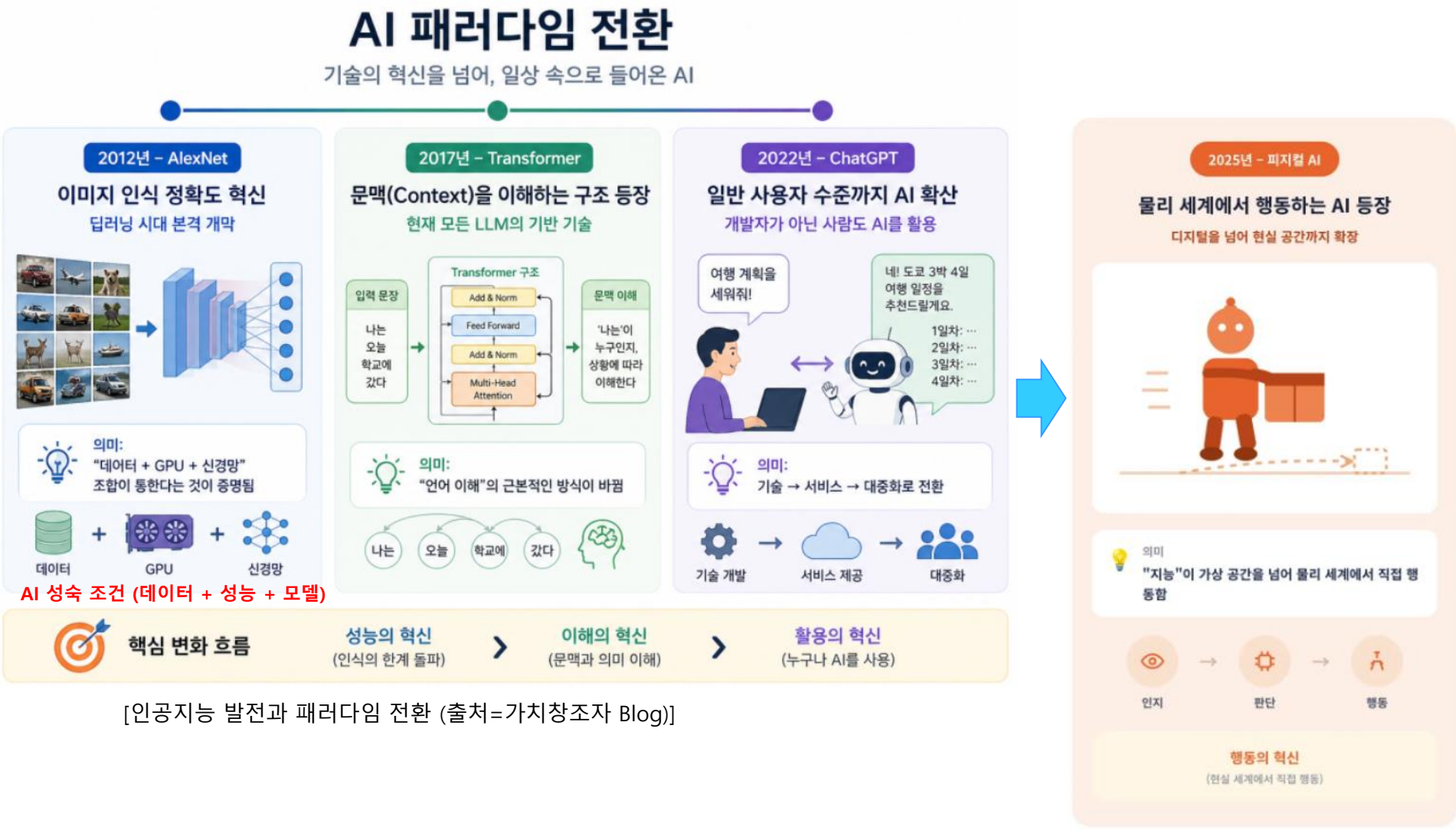
알파고(AlphaGo) 구글의 딥마인드가 개발한 인공지능 바둑 프로그램, 이세돌 프로에 4:1로 승리

OpenAI사의 2022년 Chat GPT 3.5
2023년 Chat GPT 4.0 발표하면서 **생성형 AI에 대한 광풍**


MIND
A QUARTERLY REVIEW
OF
PSYCHOLOGY AND PHILOSOPHY

튜링 테스트 기사가 실린 저널
1943년 에니악보다 3년 빠르게
독일 암호장치 애니그마 해독하기 위해 콜로서스 컴퓨터 개발

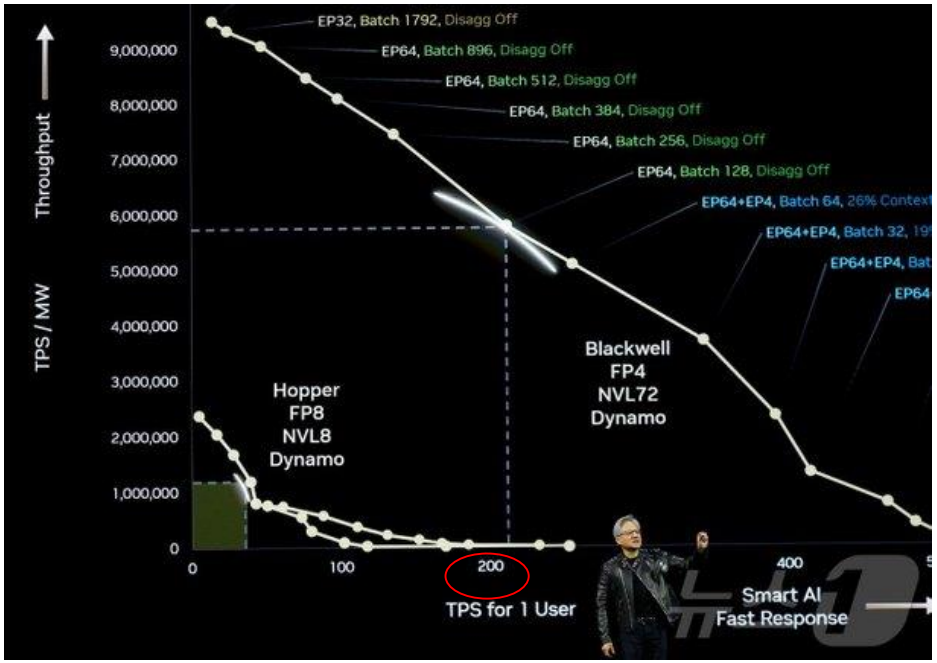
3. 인공지능(AI)의 진화 (2/2)



[인공지능 발전과 패러다임 전환 (출처=가치창조자 Blog)]

4. 기술 특이점(Technical Singularity) (1/2)

[무어의 법칙] – 고든 무어 : 인텔 및 페어차일드 반도체의 창립자
➔ 마이크로칩의 밀도가 24개월마다 2배로 늘어난다는 법칙.



[Nvidia GPU]

- 엔비디아는 2022에 H100를 소개
- 엔비디아는 2023년 11월 호퍼 아키텍처 H200 출시
- 2024년 블랙웰 아키텍처 ' GB200 ' 공개
- 베라 루빈(Vera Rubin) NVL144: 2026년 하반기 대량 양산 및 출시
- 차세대 AI PC 플랫폼: RTX 스파크 (N1X)
  2026년 10월 정식 출시 예정
- 루빈 울트라(Rubin Ultra) NVL576 2027년 출시 예정



[삼성 메모리]

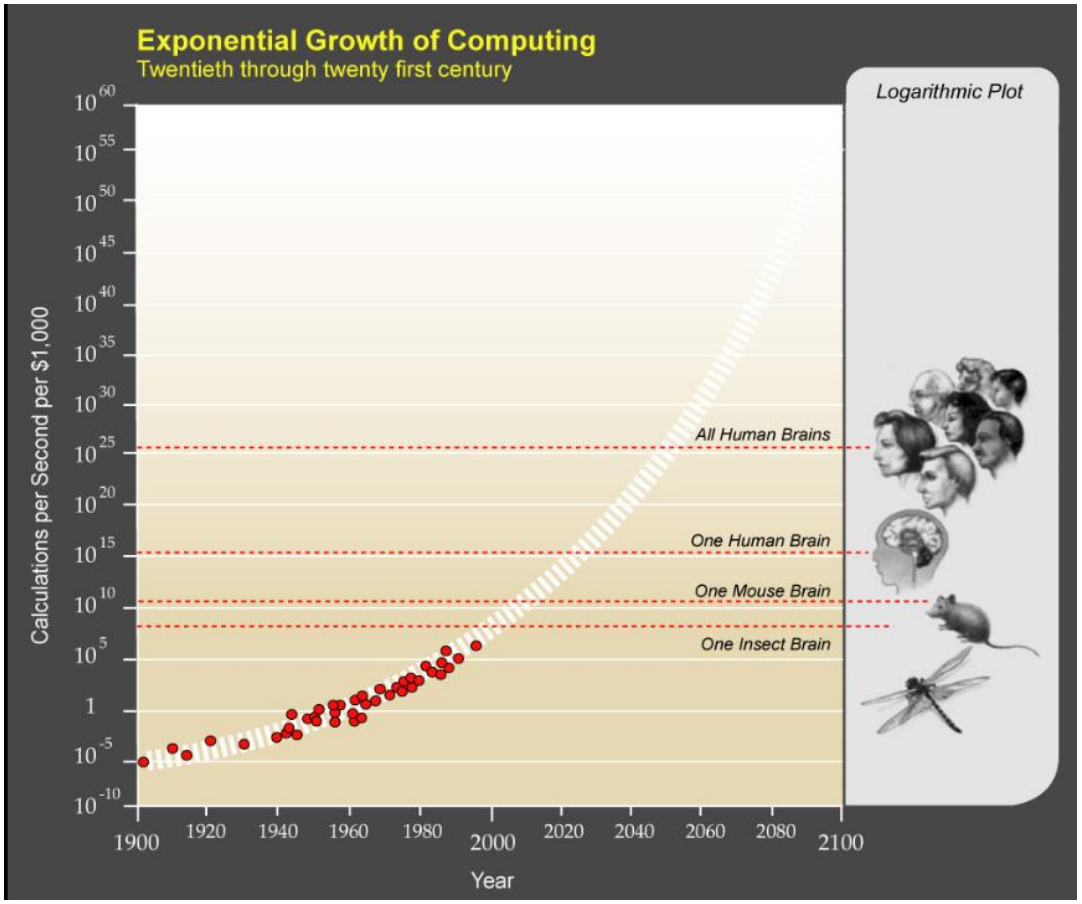
- 2022년 삼성 3nm 양산, TSMC 1.4nm 공정개발
- 2025년 삼성 2nm 양산
- 2027년 삼성 1.4nm 양산 계획

*참고로 1나노미터(nm)는 10억분의 1m로
머리카락 굵기의 1/10만에 해당되는 크기

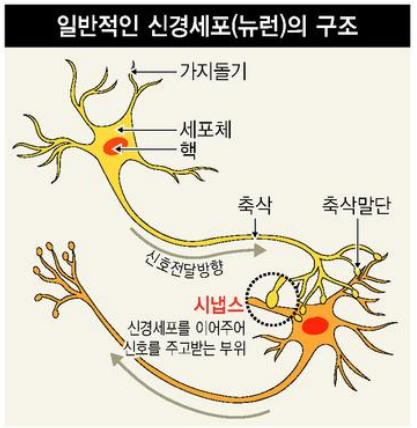
4. 기술 특이점(Technical Singularity) (2/2)

[The Singularity Is Near] – **Raymond Kurzweil,** Massachusetts Institute of Technology BS.)

➔ 2005년 저술할 때는 2045년으로 예측



[출처] https://en.wikipedia.org/wiki/The_Singularity_Is_Near



인간은 1000억개 뉴런, 100조개 시냅스 감짐

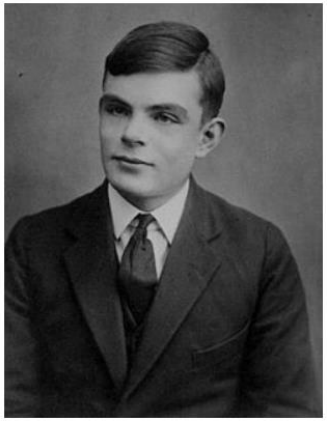
- 인간 한 명의 두뇌는 : 1E*16(10조) 계산 능력으로 추정
- 현재 컴퓨팅 발전 속도로 보면, 1000달러로 1초당 계산할 수 있는 컴퓨팅 능력

➢ 2020년도에는 한 사람 정도의 지능 돌파 (1E*16)

➢ 2045년에는 전 인구의 지능 돌파(1E*25)

CLICK

5. 튜링 테스트



앤런 튜링
(출처 : 위키백과)

✓ **튜링 테스트 : 인공 지능의 판별 기준 (이미테이션 게임)**

- **1950년 영국 앨런 튜링 <Computing Machinery and Intellignece> 논문**

  일반인으로 구성된 심사위원들이 5분 동안 대화를 나눠 기계와 대화를 나눴다는 것을 깨닫는 비율이 70%를 넘지 않을 때,

  컴퓨터가 생각하고, 지능이 있는 것으로 볼 수 있다.

✓ 튜링 테스트는 인간과 동등하거나 구별할 수 없는 지능적인 행동을 보여주는 기계의 능력에 대한 테스트

☞ **튜링 테스트란, 컴퓨터·인간 구분 안 되면 "인공지능"**

Contents

1	인공지능 이해
2	품질과 테스트 이해
3	AI 발전과 프로젝트 활용
부록 #1	AI 발전 트랜드

1. 품질과 테스트의 관계

테스트 목적

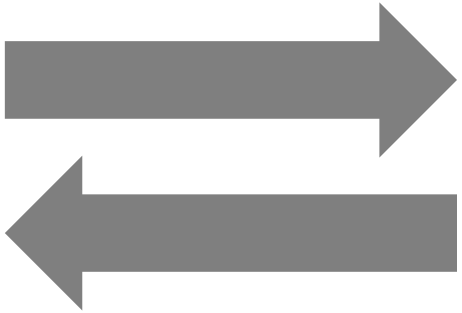
- 사용자의 기대 수준과 요구사항 충족 확인
- **소프트웨어의 여러 측면 품질특성을 평가**
- 결함 발견 및 결함 조치하여 **품질 향상**

S/W 품질 특성

AI 품질 모델 (ISO 25059)

AI 고유 특성

Testing을 통해
품질 측정



결함 발견 및 결함 해결
품질 향상

S/W 테스트

정적 테스트
- 리뷰/동료검토
- Code Inspection

동적 테스트
- 단위/통합/시스템/인수
- 성능 테스트
- 보안 테스트
- Code Coverage

2. AI 관련 품질/테스트 국제 표준

공통 기반 개념표준 ISO/IEC 22989(AI 용어·개념) · ISO/IEC 23053(ML 프레임워크) · ISO/IEC 5338(AI 시스템 생명주기)
ISO/IEC 22989/23894 위험용어 공유

품질 (Quality)

SQuaRE 확장 · 신뢰성

ISO/IEC 25059:2023
AI 품질모델 SQuaRE 8대 특성 + AI 특화 부특성 (적응성·강건성·투명성 등)

ISO/IEC TS 25058:2024
AI 품질평가 절차 25059 기반 실제 평가 프로세스·체크리스트

ISO/IEC TR 24028:2020
AI 신뢰성(Trustworthiness) 신뢰성 관련 개념, 이슈, 접근법 정리

ISO/IEC TR 24027 · TS 12791
편향(Bias) 식별·완화 AI 시스템 편향 원인 분류 및 완화 기법

ISO/IEC TR 24029-1/-2
신경망 강건성 적대적 입력 등에 대한 강건성 평가 방법

ISO/IEC 5259 시리즈
ML 데이터 품질 분석·ML용 데이터 품질 프레임워크(1~5부)

테스트 (Testing)

29119 확장 · AI 전용 테스트

ISO/IEC TS 42119-2:2025
AI 테스트 개요 ISO/IEC/IEEE 29119을 AI 시스템에 적용하는 위험기반 접근

ISO/IEC DTS 42119-3
AI 검증·확인(V&V) 형식적 방법·시뮬레이션 기반 V&V 분석(제정 중)

ISO/IEC DTS 42119-x
편향·강건성 테스트(예정) 공정성·적대적 강건성 등 특화 테스트 파트 순차 발간

ISO/IEC TR 29119-11:2020
AI 기반 시스템 테스트 가이드 AI 테스트 최초 가이드라인(42119 시리즈로 대체 예정)

ISO/IEC TS 4213:2022
ML 분류 성능 측정 분류 모델의 정확도 등 성능 지표 표준화

ISO/IEC TS 6254
설명가능성·해석가능성 ML 모델 설명·해석 방법 분류 체계

거버넌스·리스크

경영시스템 · 위험관리 · 영향평가

ISO/IEC 42001:2023
AI 경영시스템(AIMS) PDCA 기반 AI 거버넌스 요구사항, 인증 가능(ISMS 유사)

ISO/IEC 23894:2023
AI 리스크관리 가이드 AI 특유 위험의 식별·평가·대응 프레임워크

ISO/IEC 38507:2022
조직의 AI 거버넌스 이사회·경영진 차원의 AI 활용 거버넌스 책임

ISO/IEC 42005:2025
AI 시스템 영향평가 개인·집단·사회에 미치는 영향 평가 프레임워크

ISO/IEC 42006:2025
AIMS 인증기관 요구사항
42001 인증·심사기관 자격 요건

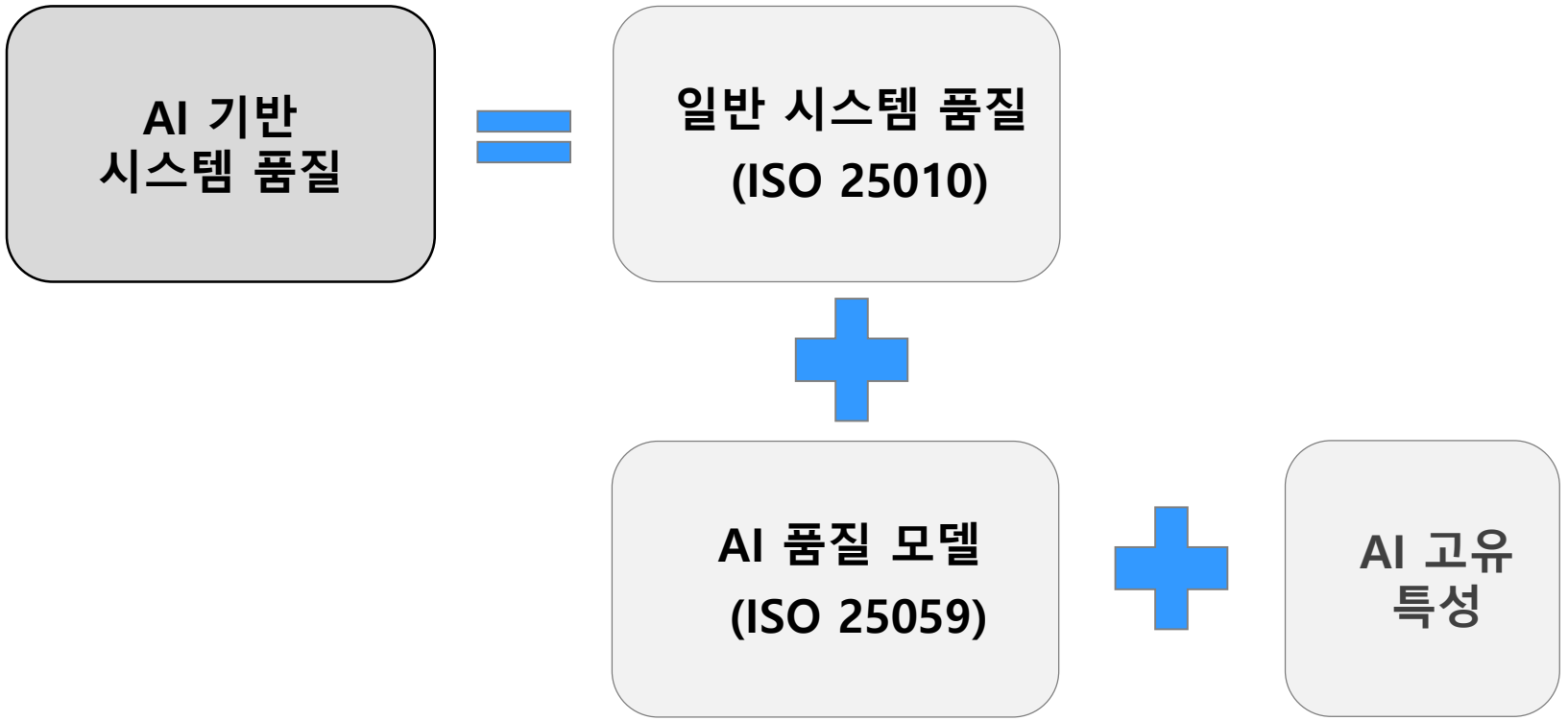
ISO/IEC TR 24368
윤리·사회적 우려 AI의 윤리적·사회적 이슈 개관

※ DTS (Draft Technical Specification, 기술 사양 초안) ➔ TS (Technical Specification 기술 사양) ➔ TR (Technical Report, 기술 보고서)

3. AI 기반 시스템 품질

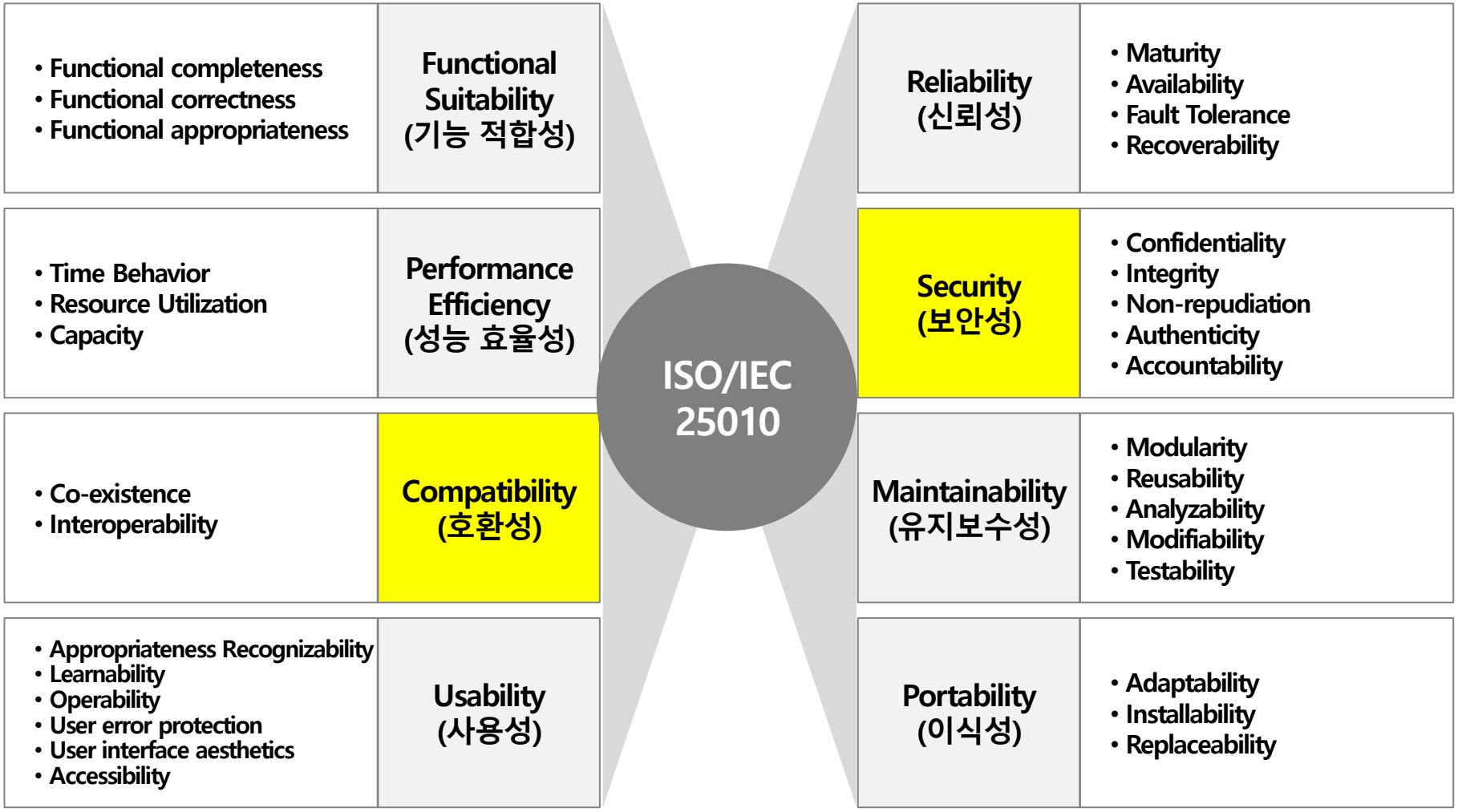
◆ AI 기반 시스템 품질

• AI 기반 시스템은 최소 하나 이상의 AI 구성요소를 포함하는 시스템



3. AI 기반 시스템 품질 – 일반 시스템 품질 (ISO/IEC 25010)

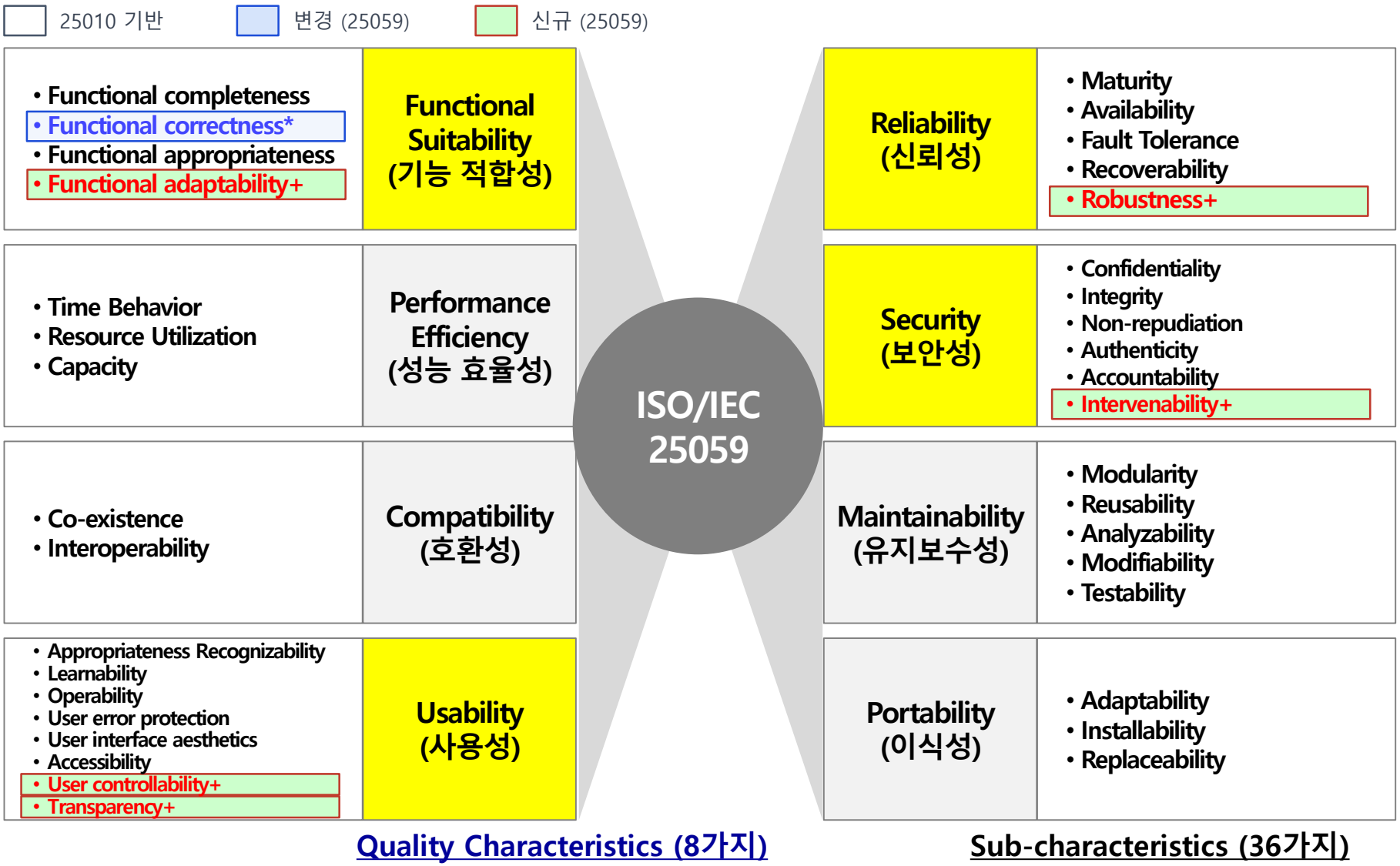
Product Quality Model – 소프트웨어/시스템 개발 및 초기 운영준비 단계에서 주로 적용



Quality Characteristics (8가지)

Sub-characteristics (31가지)

3. AI 기반 시스템 품질 – AI 시스템 품질 (ISO/IEC 25059)





3. AI 기반 시스의 품질 > 3.1 AI 시스템 품질

가. 기능 적합성

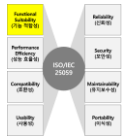
기능 적합성(Functional suitability)

명시된 조건에서 사용될 때 **명시적 요구와 암묵적 요구를 만족하는 기능을 제품/시스템이 제공하는 정도**

25010 기반 변경 (25059) 신규 (25059)

코드	부특성	정의	구분
1.1	기능완전성 Functional Completeness	명시된 작업과 사용자 목표를 만족시키기 위해 필요한 모든 기능을 기능 집합이 포괄하는 정도	-
1.2	기능정확성 Functional Correctness	제품/시스템이 정확한 결과를 정확도로 제공하는 정도. AI(특히 확률적 ML)는 모든 상황에서 기능정확성을 항상 보장하지 않는다는 점이 명시적으로 반영됨	변경
1.3	기능적절성 Functional Appropriateness	기능이 명시된 작업과 목표 달성을 용이하게 하는 정도	-
1.4	기능적응성 Functional Adaptability	지속적 학습 여부와 무관하게, 변화하는 동적 환경에 시스템이 적응할 수 있는 능력	신규

기능 테스트 (단위, 통합, 시스템, 인수)



3. AI 기반 시스의 품질 > 3.1 AI 시스템 품질

가. 기능 적합성

참고 #1. 기능 적합성(Functional suitability) 메트릭 측정 지표 예시

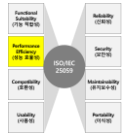
부특성	측정 지표 (Metrics)	설명	측정방법 · 도구	산출 공식 · 예시
1.1 기능완전성 Functional Completeness	기능 구현 커버리지 (Functional Coverage)	요구사항 명세 대비 실제 구현된 기능의 비율	요구사항 추적 매트릭스(RTM) 기반 구현 여부 점검 *Jira, Azure DevOps, RTM 도구*	**FC(%) = 구현 기능 수 / 명세 전체 기능 수 × 100** *요구사항 120건 중 118건 구현 → 98.3%*
	기능 누락률 (Missing Function Rate)	명세된 기능 중 미구현·누락된 기능의 비율	기능명세서와 구현 산출물 교차 검증 *요구사항관리툴, 테스트케이스 매핑*	**MFR(%) = 누락 기능 수 / 전체 기능 수 × 100** *120건 중 2건 누락 → 1.7%*
1.2 기능정확성 Functional Correctness	결과 정확도 (Output Accuracy)	전체 실행 결과 중 정답(기대값)과 일치하는 비율	정답 레이블셋 대비 예측/처리 결과 비교 *scikit-learn, RAGAS, 테스트 자동화*	**Accuracy(%) = 정확 결과 수 / 전체 실행 수 × 100** *테스트케이스 500건 중 476건 정확 → 95.2%*
	결함 밀도 (Defect Density)	기능 규모 대비 발견된 결함의 밀도	결함관리시스템 기반 결함 집계 후 규모 대비 산출 *Jira, SonarQube, 결함관리시스템*	**DD = 결함 수 / 코드 규모(KLOC)** *결함 24건 / 20 KLOC = 1.2건/KLOC*
1.3 기능적절성 Functional Appropriateness	과업 달성 적합도 (Task Fit Score)	사용자가 목표 과업을 기능만으로 달성 가능한 정도에 대한 평가	사용자 시나리오 기반 태스크 완료 평가(전문가·사용자 리뷰) *사용성 평가 세션, 체크리스트*	**TFS(%) = 달성 가능 과업 수 / 전체 평가 과업 수 × 100** *평가 과업 30건 중 27건 달성 가능 → 90%*
	불필요 기능 비율 (Superfluous Function Ratio)	과업 수행에 불필요하거나 과도한 기능이 차지하는 비율	기능 사용 로그 분석 및 전문가 리뷰 *사용 로그 분석 툴, 애널리틱스*	**SFR(%) = 불필요 기능 수 / 전체 기능 수 × 100** *전체 45개 중 3개 저활용/불필요 → 6.7%*
1.4 기능적응성 Functional Adaptability	환경 변화 대응 정확도 (Environment Shift Accuracy)	운영 환경·데이터 분포 변화 후에도 유지되는 기능 정확도	분포 변화 전/후 동일 벤치마크 비교 *Evidently AI, MLflow, 드리프트 테스트셋*	**ESA(%) = Acc_변화후 / Acc_변화전 × 100** *분포 변화 후 정확도 유지율 92%*
	재구성 없는 대응 범위 (Reconfiguration-free Coverage)	재설계·재배포 없이 변화에 대응 가능한 시나리오 비율	변화 시나리오 목록 대비 무재구성 대응 여부 테스트 *시나리오 기반 회귀 테스트 스위트*	**RFC(%) = 무재구성 대응 시나리오 수 / 전체 변화 시나리오 수 × 100** *10개 시나리오 중 8개 무재구성 대응 → 80%*

3. AI 기반 시스의 품질 > 3.1 AI 시스템 품질
가. 기능 적합성 – 기능 테스트



단계	목적	수행방법	수행 주체
단위 테스트 (UT)	프로그램 유형별 개별 기능 검증 - 온라인 - 배치 - 인터페이스	• 개발자 환경 또는 테스트 환경에서 수행하며, 선/후행 모듈이 개발 안되었을 경우 STUB, DRIVER 등을 개발하여 테스트를 수행함 - 온라인 : 입력 값 Validation, 출력 값, 이벤트별 처리 로직, 메시지 등 검증 - 배치 : 입력 파라미터, 출력 값, 에러/로깅 처리, 비즈 로직 처리 등 검증 - 인터페이스 : 입력 파라미터, 출력 값, 에러/로깅 처리, 비즈 로직 처리 등 검증	개발팀, IF팀
통합 테스트 (AIT)	업무 영역(모듈 간) 내부의 업무 프로세스와 데이터 정합성에 대한 검증	업무프로세스 기반으로 테스트시나리오/케이스를 설계하여 업무 간 연계 검증, 업무의 선후관계 업무 처리 검증, I/F를 포함한 연동기능 검증 - 접근권한,로그인과 보안기능에 대해 테스트 시나리오/케이스 도출하여 수행 - 연동 범위 외의 내/외부시스템은 실제 연동하지 않고 시뮬레이터(stub 등) 사용 - Batch Test(Batch성의 Function을 일괄작업 스케줄링 설정하여 검증)	개발팀 제3자 테스트팀
시스템 테스트 (SIT)	사용자 유형 (운영자 포함)별 업무 프로세스와 데이터 정합성 검증 (모든 대내외 연동시스템을 포함)	실운영 환경과 거의 유사하게 모든 대내/대외 시스템과 실제 연동 상태에서 업무 유형별, 사용자 유형별 End-to-End 업무 프로세스 시나리오/케이스를 도출하여 검증 - 조직/사용자 권한 및 보안기능 포함한 테스트 시나리오/케이스 도출하여 검증 - 대외시스템 중 테스트환경이 없어 실제 연동이 불가한 경우 시뮬레이터(stub등) 활용	개발팀 제3자 테스트팀
인수 테스트 (UAT)	사용자 인수 관점의 업무처리 흐름과 데이터 정합성 검증	사용자 유형별 업무 프로세스와 데이터 정합성을 검증 실제 발생한 데이터를 기반으로 현행시스템과 신규시스템과 비교 검증 조직/사용자 권한 및 보안 기능 검증	사용자 그룹

Sample



3. AI 기반 시스의 품질 > 3.1 AI 시스템 품질

나. 성능 효율성

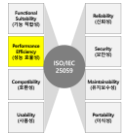
성능 효율성 (Performance efficiency)

명시된 조건에서 사용되는 자원의 양에 대해 요구되는 성능을 제품/시스템이 제공하는 정도

25010 기반 변경 (25059) 신규 (25059)

코드	부특성	정의	구분
2.1	시간효율성 Time Behaviour	기능 수행 시 응답시간, 처리시간, 처리율이 요구사항을 만족하는 정도	-
2.2	자원효율성 Resource Utilization	기능 수행 시 사용되는 자원의 양과 유형이 요구사항을 만족하는 정도	-
2.3	용량 Capacity	제품/시스템 매개변수의 최대 한계치가 요구사항을 만족하는 정도	-

성능 테스트 (부하, 스트레스, 스파이크, 내구성)



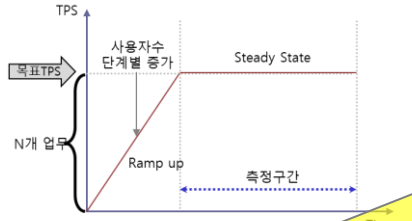
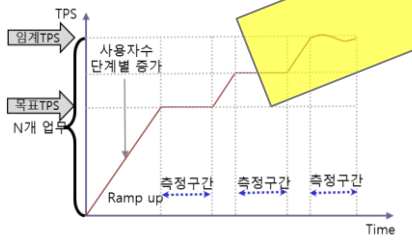
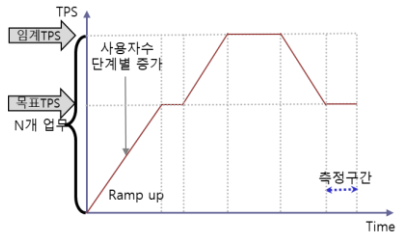
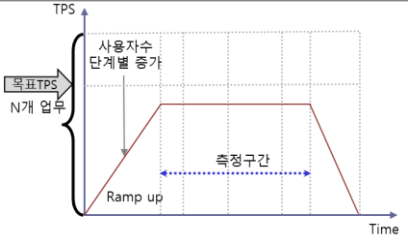
3. AI 기반 시스의 품질 > 3.1 AI 시스템 품질
나. 성능 효율성

참고 #2. 성능 효율성 (Performance efficiency) 측정 지표 예시

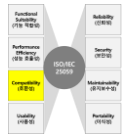
부특성	측정 지표 (Metrics)	설명	측정방법 · 도구	산출 공식 · 예시
2.1 시간효율성 Time Behaviour	**평균 응답시간 (Average Response Time)**	요청부터 응답 완료까지 걸리는 평균 시간	부하테스트 중 응답시간 로그 수집·평균 산출 *load Runner, JMeter, Locust, Grafana*	**ART = Σ응답시간 i / N** *1,000건 요청 평균 응답시간 320ms*
	처리율 (Throughput)	단위 시간당 처리 가능한 요청/트랜잭션 수	부하테스트 중 시간당 처리 완료 건수 집계 *load Runner, JMeter, k6, Prometheus*	**TPS = 처리 완료 요청 수 / 측정시간(초)** *초당 450 트랜잭션 처리(TPS=450)*
2.2 자원효율성 Resource Utilization	**CPU / 메모리 사용률**	부하 상황에서 CPU 및 메모리 자원의 평균·최대 사용률	모니터링 도구를 통한 실시간 자원 사용률 수집 *Grafana, Prometheus, top/htop*	**사용률(%) = 사용 자원량 / 할당 자원량 × 100** *피크 시 CPU 78%, 메모리 65% 사용*
	GPU 자원 효율 (GPU Utilization)	AI 추론·학습 시 GPU 연산 자원의 활용 효율	GPU 모니터링 도구로 사용률·메모리 점유율 측정 *nvidia-smi, DCGM, MLflow*	**GPU Util(%) = GPU 활성 연산시간 / 전체 관측시간 × 100** *추론 서빙 중 GPU 평균 사용률 82%*
2.3 용량 Capacity	**최대 동시 사용자 수 (Max Concurrent Users)**	성능 저하 없이 처리 가능한 최대 동시 사용자 수	점증 부하테스트(Stress Test)로 한계점 도출 *JMeter, Gatling, k6*	**최대 동시 사용자 = SLA 기준 초과 직전 부하 수준** *동시 사용자 5,000명까지 SLA(2초) 이내 응답*
	최대 처리 용량 (Max Throughput Capacity)	시스템이 견딜 수 있는 최대 처리량 한계	한계 부하테스트를 통한 포화점 (Saturation Point) 측정 *Gatling, Locust, APM 도구*	**최대 용량 = 오류율 급증 직전 TPS** *TPS 1,200 초과 시 오류율 5%↑ → 최대용량 1,200 TPS*



3. AI 기반 시스의 품질 > 3.1 AI 시스템 품질
나. 수행 효율성 – 성능 테스트

단계	목적		수행 주체
부하(Load) 테스트	• Performance 테스트는 핵심 업무나 고객이 요구한 업무를 대상으로 요구사항 명세서나 고객과 협의를 통해 응답속도/TPS/자원사용량 등에 대한 목표 기준을 설정하고, 해당 기준 부합 여부를 검증하기 위해 테스트를 실시한다.		성능테스트팀 업무 개발팀
임계 (Capacity) 테스트	• Capacity 테스트는 최대 목표 사용자에 대한 데이터를 구축한 상태에서 성능 테스트 부하를 일정 비율로 증가시켜 과부하를 주었을 때 시스템의 최대 성능을 파악 하고, 부족한 자원이 무엇인지 확인한다.		성능테스트팀 업무 개발팀
과부하(Stress) 테스트	• 성능 테스트 대상 시스템이 발휘할 수 있는 최대 부하를 가한 상태에서 일정 시간을 유지한 이후, 다시 적정 수준의 부하를 가하여 시스템의 회복성을 확인하는 테스트를 수행한다. (예, 예약 시스템 검증)		성능테스트팀 업무 개발팀
내구성 (Endurance) 테스트	• 오랜 시간 동안 시스템에 목표부하의 적정한 높은 부하를 장시간 가하여 시스템의 응답시간, 자원사용량 등 반응을 테스트하여 메모리 누수 등 확인한다.		성능테스트팀 업무 개발팀

Sample



3. AI 기반 시스의 품질 > 3.1 AI 시스템 품질

다. 호환성

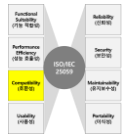
호환성 (Compatibility)

동일한 하드웨어/소프트웨어 환경을 공유하는 상태에서 다른 제품·시스템과 정보를 교환하고, 필요한 기능을 수행할 수 있는 정도

25010 기반 변경 (25059) 신규 (25059)

코드	부특성	정의	구분
3.1	공존성 Co-existence	다른 제품과 공통 환경·자원을 공유하면서 그 제품에 부정적 영향 없이 필요한 기능을 효율적으로 수행할 수 있는 정도	-
3.2	상호운용성 Interoperability	둘 이상의 시스템·구성요소·제품이 정보를 교환하고 교환된 정보를 사용할 수 있는 정도	-

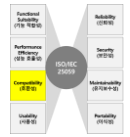
호환성 테스트 (설치, UI/UX, 기능)



3. AI 기반 시스의 품질 > 3.1 AI 시스템 품질
다. 호환성

참고 #3. 호환성 (Compatibility) 측정 지표 예시

부특성	측정 지표 (Metrics)	설명	측정방법 · 도구	산출 공식 · 예시
공존성 Co-existence	자원 충돌률 (Resource Conflict Rate)	공존 환경에서 자원 경합·충돌이 발생하는 빈도	다중 애플리케이션 동시 운영 테스트 중 충돌 로그 집계 *통합테스트 환경, APM 도구*	**RCR(%) = 충돌 발생 건수 / 전체 공존 시나리오 수 × 100** *20개 시나리오 중 1건 충돌 → 5%*
	성능 간섭도 (Performance Interference)	타 제품 동시 운영 시 자사 시스템 성능 저하 정도	단독 운영 대비 공존 운영 시 성능 비교 *JMeter, Grafana*	**PI(%) = (T_단독 − T_공존) / T_단독 × 100** *응답시간이 단독 대비 8% 저하*
상호운용성 Interoperability	인터페이스 연동 성공률 (Integration Success Rate)	외부 시스템과의 연동 테스트 중 성공적으로 데이터 교환된 비율	API/인터페이스 연동 테스트 케이스 실행 및 결과 집계 *Postman, SoapUI, 연동테스트 스위트*	**IISR(%) = 성공 연동 케이스 수 / 전체 연동 테스트 케이스 수 × 100** *연동 테스트 60건 중 58건 성공 → 96.7%*
	데이터 표준 준수율 (Data Standard Compliance)	교환 데이터가 정의된 표준 포맷·스키마를 준수하는 비율	스키마 검증 도구를 통한 자동 검증 *JSON Schema Validator, XSD 검증기*	**DSCR(%) = 표준 준수 데이터 건수 / 전체 교환 데이터 건수 × 100** *교환 데이터 1,000건 중 992건 준수 → 99.2%*



3. AI 기반 시스의 품질 > 3.1 AI 시스템 품질

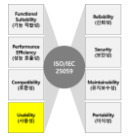
다. 호환성 – 웹 호환성 테스트

전자정부 웹사이트 품질관리 지침 - 웹사이트 웹 호환성

Sample

[전자정부서비스 웹 호환성 진단표]

구분	진단지표	진단기준	진단방법
1.웹 표준 문법 준수	1-1 표준 (X)HTML 문법 준수 여부	W3C Markup Validator에서 출력된 오류를 <심각>, <위험>, <보통>으로 나누어 오류 발생시마다 오류 유형별로 차등을 두어 감점	W3C Markup Validator (http://validator.w3.org)
	1-2 표준 CSS 문법 준수 여부	W3C CSS Validator에서 출력된 오류를 <심각>, <위험>, <보통>으로 나누어 오류 발생시마다 오류 유형별로 차등을 두어 감점	W3C CSS Validator (http://jigsaw.w3.org/css-validator)
2.웹 호환성 확보	2-1 동작 호환성 확보 여부	브라우저 부가 기능을 이용해서 해당 페이지 내에 사용된 Javascript 오류 및 DOM 경고 발생시 감점 Javascript가 의도한 기능이 정상적으로 동작되는지 점검하여 비정상적 동작에 대해 감점	브라우저 부가 기능, 크로스 브라우징 테스트
	2-2 레이아웃 호환성 확보 여부	3종 이상의 브라우저에서 동등한 레이아웃으로 구현되었는지 여부 확인 - 브라우저별 특성에 의한 차이(폰트, 픽셀 등)는 예외로 함	크로스 브라우징 테스트
	2-3 플러그인 호환성 확보 여부	사용자 화면에 영향을 미치거나 로그인 및 동작에 관련된 다음과 같은 플러그인에 한하여 3종 이상의 브라우저에서 정상적으로 동작되는지 조사 - 보안, 구간암호화, 공인인증 - 영상, 멀티미디어(플래시, 실버라이트,그래프, 리포트 등) - 파일 송수신브라우저별 대체 수단을 제공할 경우에는 예외로 함	플러그인 동작 테스트, 크로스 브라우징 테스트



3. AI 기반 시스의 품질 > 3.1 AI 시스템 품질
라. 사용성

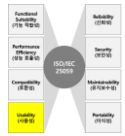
사용성 (Usability)

명시된 사용 맥락에서 명시된 목표를 달성하기 위해 특정 사용자가 효과성, 효율성, 만족도를 가지고 제품/시스템을 사용할 수 있는 정도

25010 기반 | 변경 (25059) | 신규 (25059)

코드	부특성	정의	구분
4.1	적절성 인지성 (Appropriateness Recognizability)	사용자가 제품/시스템이 자신의 요구에 적절한지 인지할 수 있는 정도	-
4.2	학습성 (Learnability)	명시된 목표를 달성하기 위해 특정 사용자가 제품/시스템 사용법을 학습할 수 있는 정도	-
4.3	운용성 (Operability)	제품/시스템이 운용 및 제어하기 쉬운 속성을 갖는 정도	-
4.4	사용자 오류 방지성 (User Error Protection)	시스템이 사용자의 오류를 방지하는 정도	-
4.5	사용자 인터페이스 심미성 (UI Aesthetics)	사용자 인터페이스가 사용자에게 만족스럽고 쾌적한 상호작용을 가능하게 하는 정도	-
4.6	접근성 (Accessibility)	다양한 특성과 능력을 가진 폭넓은 사용자가 제품/시스템을 사용할 수 있는 정도	-
4.7	사용자 통제성 (User Controllability)	사용자가 AI 시스템의 동작에 대해 적시에 적절하게 제어하고 변경할 수 있는 정도	**신규**
4.8	투명성 (Transparency)	AI 시스템에 관한 적절한 정보가 이해관계자에게 소통되는 정도	**신규**

웹접근성 테스트 (인식, 운용, 이해, 견고성) 및 사용자 만족도 조사 (학습, 사용 등)

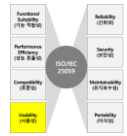


3. AI 기반 시스의 품질 > 3.1 AI 시스템 품질
라. 사용성

참고 #4. 사용성 (Usability) 측정 지표 예시

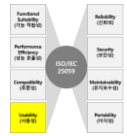
부특성	측정 지표 (Metrics)	설명	측정방법 · 도구	산출 공식 · 예시
4.1 적절성 인지성 (Appropriateness Recognizability)	초기 적합도 평가 (First Impression Fit)	초기 사용자가 제품 목적·적합성을 즉시 인지하는 정도에 대한 설문 점수	사용자 대상 설문·인터뷰 *사용자 설문, UserTesting*	**평균 점수(5점 척도) = ΣUser Score / 응답자 수** *사용자 50명 평균 4.2/5점*
	목적 이해 소요시간 (Time to Understand)	사용자가 제품의 목적을 이해하는 데 걸리는 평균 시간	사용성 테스트 중 태스크 시작~이해 완료 시점 측정 *사용성 테스트 세션 녹화·분석*	**평균 이해 소요시간 = Σ소요시간_i / N** *평균 8.5초 만에 서비스 목적 이해*
4.2 학습성 (Learnability)	학습 소요시간 (Time to Learn)	신규 사용자가 핵심 기능을 능숙하게 사용하기까지 걸리는 시간	신규 사용자 대상 태스크 수행 시간 측정(숙련 전/후 비교) *사용성 테스트, 스크린 레코딩*	**TTL = 숙련 기준 도달 시점 − 최초 사용 시점** *평균 12분 만에 핵심기능 숙련 도달*
	재시도 없는 성공률 (First-try Success Rate)	별도 학습·도움말 없이 첫 시도에 태스크를 완료하는 비율	사용성 테스트 중 최초 시도 성공 여부 기록 *사용성 테스트 세션*	**FTSR(%) = 첫 시도 성공 사용자 수 / 전체 참여자 수 × 100** *참여자 40명 중 31명 성공 → 77.5%*
4.3 운용성 (Operability)	태스크 완료 효율성 (Task Completion Efficiency)	목표 태스크를 완료하는 데 필요한 조작(클릭·입력) 수	사용성 테스트 중 조작 횟수 로깅 *사용성 테스트 도구, 이벤트 트래킹*	**TCE = 최소 필요 조작 수 / 실제 수행 조작 수** *최소 4단계 대비 실제 5단계 → TCE=0.8*
	조작 오류율 (Operation Error Rate)	태스크 수행 중 잘못된 조작이 발생하는 비율	사용성 테스트 중 오조작 이벤트 집계 *이벤트 로깅, 세션 리플레이 툴*	**OER(%) = 오조작 발생 건수 / 전체 조작 시도 수 × 100** *조작 300건 중 9건 오조작 → 3%*
4.4 사용자 오류 방지성 (User Error Protection)	오류 예방 커버리지 (Error Prevention Coverage)	발생 가능한 사용자 오류 시나리오 중 사전 방지 장치가 적용된 비율	오류 시나리오 목록 대비 방지 기능 적용 여부 점검 *체크리스트 기반 UX 감사*	**EPC(%) = 방지장치 적용 시나리오 수 / 전체 오류 시나리오 수 × 100** *오류 시나리오 25건 중 22건 적용 → 88%*
	치명적 오류 발생률 (Critical Error Rate)	데이터 손실 등 치명적 결과로 이어지는 사용자 오류의 발생 빈도	실사용 로그·사용성 테스트 중 치명적 오류 이벤트 집계 *오류 로깅 시스템, Sentry*	**CEOR = 치명적 오류 건수 / 전체 세션 수** *세션 10,000건 중 3건 치명적 오류*

3. AI 기반 시스의 품질 > 3.1 AI 시스템 품질
라. 사용성



참고 #4. 사용성 (Usability) 측정 지표 예시

부특성	측정 지표 (Metrics)	설명	측정방법 · 도구	산출 공식 · 예시
4.5 사용자 인터페이스 심미성 (UI Aesthetics)	심미성 만족도 점수 (Aesthetic Satisfaction)	사용자가 평가한 UI 디자인 만족도 점수	리커트 척도 기반 디자인 만족도 설문 *사용자 설문, UEQ*	**평균 점수 = ΣUser Rating / 응답자 수** *UEQ 매력성 평균 1.8(–3~+3 척도)*
	디자인 일관성 준수율 (Design Consistency Rate)	디자인 시스템·가이드라인을 준수하는 화면 요소의 비율	디자인 가이드라인 대비 화면 요소 감사 *Figma 디자인시스템 감사, UI 린트 툴*	**DCR(%) = 준수 요소 수 / 전체 UI 요소 수 × 100** *UI 요소 200개 중 190개 준수 → 95%*
4.6 접근성 (Accessibility)	웹 접근성 표준 준수율 (WCAG Compliance)	WCAG 등 접근성 표준 성공기준 충족 비율	접근성 자동·수동 진단 도구를 통한 성공기준 점검 *WAVE, axe DevTools, Lighthouse*	**WCR(%) = 충족 성공기준 수 / 전체 평가 기준 수 × 100** *WCAG 2.1 AA 45개 중 42개 충족 → 93.3%*
	보조기술 호환률 (Assistive Tech Compatibility)	스크린리더 등 보조기술로 정상 이용 가능한 화면·기능의 비율	스크린리더(NVDA, VoiceOver) 기반 수동 테스트 *NVDA, VoiceOver, JAWS*	**ATC(%) = 정상 이용 화면 수 / 전체 테스트 화면 수 × 100** *30개 화면 중 27개 정상 이용 → 90%*
4.7 사용자 통제성 (User Controllability)	사용자 통제 성공률 (User Intervention Success)	사용자가 AI 동작을 제어 및 변경 요청했을 때 성공적으로 반영되는 비율	개입 시나리오 테스트(제어. 변경) 실행 및 검증 *기능 테스트 스위트, 시나리오 테스트*	**UISR(%) = 반영된 제어 요청 수 / 전체 제어 요청 수 × 100** *제어 요청 40건 중 38건 반영 → 95%*
	통제 응답 지연시간 (Control Response Latency)	사용자의 통제 요청(정지·수정) 후 실제 반영까지 걸리는 시간	요청 시점과 시스템 반영 시점의 타임스탬프 비교 *이벤트 로그 분석, APM 도구*	**CRL = T(반영 완료) – T(통제 요청)** *평균 통제 반응 지연시간 1.2초*
4.8 투명성 (Transparency)	설명 제공률 (Explanation Provision Rate)	AI 의사결정 중 근거·설명이 함께 제공되는 비율	예측 로그 대비 설명 생성 로그 집계 *모델 서빙 로그, SHAP/LIME 파이프라인*	**EPR(%) = 설명 제공 건수 / 전체 의사결정 건수 × 100** *고위험 판정 100% 설명 제공 확인*
	정보 이해가능성 점수 (Comprehensibility Score)	제공된 설명·정보를 이해관계자가 이해할 수 있는 정도에 대한 평가	이해관계자 대상 설명 이해도 설문·인터뷰 *사용자 설문, 전문가 리뷰 패널*	**평균 이해도 점수 = ΣRating / 응답자 수(5점 척도)** *비전문가 대상 평균 이해도 3.8/5점*



3. AI 기반 시스의 품질 > 3.1 AI 시스템 품질

라. 사용성 – 웹 접근성 테스트

전자정부 웹사이트 품질관리 지침 - 웹 접근성 진단방법

Sample

구분	검사항목	육안 (소스)	도구	비고
인식의 용이성	1.적절한 대체 텍스트	O	O	openwax
	2.자막 제공	O		
	3.색에 무관한 콘텐츠 인식	O	O	CCA
	4.명확한 지시사항 제공	O		
	5.텍스트 콘텐츠 명도 대비		O	CCA
	6.자동 재생 금지	O		
	7.콘텐츠 간 구분	O		
운용의 용이성	8.키보드 사용 보장	O		
	9.초점이동	O		
	10.조작 가능		O	KWCAG a11y Inspector
	11.응답 시간 조절	O	O	
	12.정지 기능 제공	O	O	

구분	검사항목	육안 (소스)	도구	비고
운용의 용이성	13.깜빡임과 번쩍임 사용제한	O		
	14.반복 영역 건너뛰기	O	O	openwax
	15.제목 제공	O	O	openwax
	16.적절한 링크 텍스트	O	O	openwax
이해의 용이성	17.기본 언어 표시		O	openwax
	18.사용자 요구에 따른 실행	O		
	19.콘텐츠 선형 구조	O		
	20.표의 구성	O	O	openwax
	21.레이블 제공	O	O	openwax
	22.오류 정정	O		openwax
견고성	23.마크업 오류 방지	O	O	openwax
	24.웹어플리케이션 접근성준수	-	-	UIA 등

※ 서울시 진단 자료 참고(https://news.seoul.go.kr/gov/archives/47485)

- **웹접근성 자체점검표** : 웹접근성 진단 체크리스트 결과서 (PPT)
- **웹 표준**(HTML, CSS) **및 웹 호환성**(크로스브라우징 지원) **진단 체크리스트 (Excel)**



3. AI 기반 시스의 품질 > 3.1 AI 시스템 품질

마. 신뢰성

신뢰성 (Reliability)

명시된 조건에서 명시된 기간 동안 시스템/제품/구성요소가 명시된 기능을 수행하는 정도

25010 기반 | 변경 (25059) | 신규 (25059)

코드	부특성	정의	구분
5.1	성숙성 Maturity	정상적인 운영 상황에서 시스템/제품/구성요소가 신뢰성 요구를 충족하는 정도	-
5.2	가용성 Availability	시스템/제품/구성요소가 사용을 위해 운용 및 접근 가능한 정도	-
5.3	결함 허용성 Fault Tolerance	하드웨어/소프트웨어 결함이 있음에도 시스템이 의도한 대로 작동하는 정도	-
5.4	회복성 Recoverability	장애 발생 시 직접 영향받은 데이터를 복구하고 원하는 상태로 시스템을 복원할 수 있는 정도	-
5.5	강건성 Robustness	낯선 입력, 편향·적대적·유효하지 않은 입력, 외부 간섭, 다양한 환경 조건에서도 기능정확성을 유지하는 능력	**신규**

가용성 (이중화, Fail-Over), 안정성 (장시간 부하 테스트, 백업/복구) 테스트



3. AI 기반 시스의 품질 > 3.1 AI 시스템 품질

마. 신뢰성

참고 #5. 신뢰성 (Reliability) 측정 지표 예시

부특성	측정 지표 (Metrics)	설명	측정방법 · 도구	산출 공식 · 예시
5.1 성숙성 Maturity	평균 고장 간격 (MTBF)	연속된 두 고장 사이의 평균 정상 운영 시간	운영 로그 기반 고장 발생 간격 집계 *장애관리시스템, Datadog, PagerDuty*	**MTBF = 총 정상 운영시간 / 고장 발생 횟수** *총 운영 3,000시간, 고장 5회 → MTBF=600시간*
	결함 제거율 (Defect Removal Rate)	발견된 결함 중 해결·제거된 결함의 비율	결함관리시스템 기반 상태별 집계 *Jira, Bugzilla*	**DRR(%) = 제거된 결함 수 / 발견된 결함 수 × 100** *결함 150건 중 143건 제거 → 95.3%*
5.2 가용성 Availability	시스템 가용률 (Uptime Ratio)	전체 운영 기간 대비 정상 서비스 가능 시간의 비율	모니터링 시스템 기반 가동·중단 시간 집계 *Prometheus, Datadog, UptimeRobot*	**Availability(%) = (총시간−다운타임) / 총시간 × 100** *월간 가동률 99.95%(다운타임 21.6분)*
	평균 복구 시간 (MTTR)	장애 발생부터 정상 복구까지 걸리는 평균 시간	장애 티켓 기록 기반 발생~복구 시간 집계 *PagerDuty, 장애관리시스템*	**MTTR = Σ복구시간_i / 장애 발생 횟수** *장애 8건 평균 복구시간 18분*
5.3 결함 허용성 Fault Tolerance	장애 격리 성공률 (Failure Isolation Rate)	특정 구성요소 장애가 전체 시스템 장애로 전파되지 않고 격리되는 비율	카오스 엔지니어링 기반 장애 주입 테스트 *Chaos Monkey, Gremlin, Litmus*	**FIR(%) = 격리 성공 시나리오 수 / 전체 장애주입 시나리오 수 × 100** *장애주입 15건 중 14건 격리 → 93.3%*
	우아한 성능 저하율 (Graceful Degradation Rate)	부분 장애 시 전체 중단 없이 저하된 수준으로 서비스 유지되는 비율	부분 장애 시나리오 테스트 후 서비스 지속 여부 확인 *장애주입 테스트 프레임워크*	**GDR(%) = 저하 유지 시나리오 수 / 전체 부분장애 시나리오 수 × 100** *10건 중 8건 저하 상태 유지 → 80%*
5.4 회복성 Recoverability	데이터 복구율 (Data Recovery Rate)	장애 발생 시 손실 없이 복구되는 데이터의 비율	백업·복구 테스트를 통한 데이터 정합성 검증 *백업관리시스템, 복구테스트 스크립트*	**DRR(%) = 정상 복구 데이터 건수 / 손실 발생 데이터 건수 × 100** *손실 1,000건 중 998건 정상 복구 → 99.8%*
	목표 복구 시간 달성률 (RTO Achievement Rate)	목표 복구 시간(RTO) 이내에 시스템이 복구되는 비율	DR(재해복구) 훈련을 통한 복구 소요시간 측정 *DR 훈련 로그, 복구 자동화 스크립트*	**달성률(%) = RTO 이내 복구 건수 / 전체 DR 훈련 건수 × 100** *DR 훈련 6회 모두 RTO(30분) 이내 → 100%*



3. AI 기반 시스의 품질 > 3.1 AI 시스템 품질

마. 신뢰성

참고 #5. 신뢰성 (Reliability) 측정 지표 예시

부특성	측정 지표 (Metrics)	설명	측정방법 · 도구	산출 공식 · 예시
5.5 강건성 Robustness	**적대적 공격 방어율** **(Adversarial Robustness)**	적대적 공격(Adversarial Attack) 입력에도 정확한 예측을 유지하는 비율	적대적 예제(Adversarial Example) 생성 후 모델 예측 평가 *ART(Adversarial Robustness Toolbox), CleverHans, Foolbox*	**ARR(%) = 공격 후 정확 유지 샘플 수 / 전체 공격 샘플 수 × 100** *적대적 샘플 500건 중 421건 유지 → 84.2%*
	이상 입력 처리 정확도 **(OOD* Handling Accuracy)**	학습 분포를 벗어난 (OOD) 입력에 대해 안전하게 처리(거부·경고)하는 비율	OOD 테스트셋 대비 시스템 대응 정확도 평가 *OOD 벤치마크셋, 커스텀 이상탐지 테스트*	**OOD Acc(%) = 안전 처리 샘플 수 / 전체 OOD 샘플 수 × 100** *OOD 샘플 200건 중 178건 안전 처리 → 89%*

※ OOD : 분포 외 데이터(OUT-OF-DISTRIBUTION)

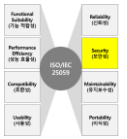
3. AI 기반 시스의 품질 > 3.1 AI 시스템 품질

마. 신뢰성 – 가용성/안정성 테스트



테스트 유형	수행 항목		검증 방법	수행 주체
구성점검	구축 점검 테스트		구축 완료 점검 Check list로 확인	TA/AA/DA
	이중화 점검 테스트		물리적 이중화 구성에 대한 점검 테스트 수행 확인	
	기동점검 테스트		기동 절차서 기준으로 정상 기동/종료 수행 확인	
가용성 테스트	Fail-over 테스트 (기능, 시간점검) (Network, OS, Storage, M/W, DBMS)		Fail-over 시점 이 전과 이 후 동일 TPS 유지 확인	
			Fail-over 시점 이 전과 이 후 Transaction 성공률 동일 여부 확인	
			Fail-over 이후 CPU 사용률 균등 분산 확인	
시스템 안정성 테스트	백업/복구		백업된 파일을 이용하여 절차에 따라 복구를 수행하여 절차 확인	
	24*7 전환		수립된 절차에 의한 24*7 전환 절차 확인	
	운영 점검	통합 관제	가용성 테스트 수행 시 등록하여 모니터링 및 Alert 기능 확인	
		Online Reorg	부하 투입 중 Online Reorg 수행하여 서비스의 영향도를 확인	
		VM live migration	VM 장애 시 다른 Cnode로 VM migration 후 서비스 정상 확인	
	안정성 검증	Long Term 테스트	장시간 부하 유지를 통한 시스템 안정성 확인	
			자원 사용률(CPU/Memory) 안정적 유지 확인	
		기능 검증	WAS Hang-up 시 Block Listen 기능 정상 확인	
		바인딩 CPU 영향도	OS 바인딩 CPU 영향도 검증 확인	
		VM HA	VM 장애 시 다른 C-node로 자동 HA된 후 서비스 정상 확인	

Sample



3. AI 기반 시스의 품질 > 3.1 AI 시스템 품질
바. 보안성

보안성 (Security)

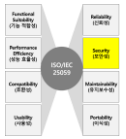
사람이나 다른 제품/시스템이 인가된 접근 수준에 따라 데이터에 접근하도록 정보와 데이터를 보호하는 정도

25010 기반 변경 (25059) 신규 (25059)

코드	부특성	정의	구분
6.1	기밀성 Confidentiality	데이터가 인가된 접근 권한을 가진 사람·제품·시스템만 접근할 수 있도록 보장하는 정도	-
6.2	무결성 Integrity	시스템/제품/구성요소/데이터에 대한 비인가 접근 또는 수정을 방지하는 정도	-
6.3	부인방지성 Non-repudiation	행위나 사건이 발생했음을 나중에 부인할 수 없도록 입증할 수 있는 정도	-
6.4	책임성 Accountability	개체의 행위가 그 개체에게 고유하게 추적될 수 있는 정도	-
6.5	인증성 Authenticity	개체(entity)의 신원이 주장한 대로임을 증명할 수 있는 정도	-
6.6	개입 가능성 (Intervenability)	위험 상황 관리를 위한 비상 조치 및 안정 확보할 수 있는 정도	**신규**

웹 취약점, 시큐어 코딩, 모의 해킹 테스트

3. AI 기반 시스의 품질 > 3.1 AI 시스템 품질
바. 보안성



참고 #6. 보안성 (Security) 측정 지표 예시

부특성	측정 지표 (Metrics)	설명	측정방법 · 도구	산출 공식 · 예시
6.1 기밀성 Confidentiality	**접근 통제 위반률 (Access Control Violation)**	인가되지 않은 접근 시도 중 실제 침입에 성공한 비율	모의 침투테스트(Penetration Test) 및 접근 로그 분석 *Burp Suite, OWASP ZAP*	**ACVR(%) = 성공한 무단접근 건수 / 전체 무단접근 시도 건수 × 100** *무단접근 시도 300건 중 2건 성공 → 0.67%*
	데이터 암호화 적용률 (Encryption Coverage)	민감 데이터 중 암호화가 적용된 비율	데이터 저장소·전송 구간 암호화 적용 여부 점검 *보안 감사 도구, SSL Labs*	**EC(%) = 암호화 적용 항목 수 / 전체 민감데이터 항목 수 × 100** *민감 필드 40개 중 40개 암호화 → 100%*
6.2 무결성 Integrity	**데이터 변조 탐지율 (Tampering Detection Rate)**	비인가 데이터 변조 시도를 탐지하는 비율	변조 시나리오 주입 테스트 후 탐지 여부 확인 *무결성 검증 도구(해시체크), SIEM*	**TDR(%) = 탐지된 변조 시도 수 / 전체 변조 시도 수 × 100** *변조 시도 50건 중 49건 탐지 → 98%*
	체크섬 불일치율 (Checksum Mismatch Rate)	데이터 전송·저장 후 무결성 검증에서 불일치가 발견되는 비율	해시값 비교를 통한 자동 검증 *SHA-256 검증 스크립트, 무결성 모니터링*	**CMR(%) = 불일치 발견 건수 / 전체 검증 건수 × 100** *검증 10,000건 중 0건 불일치 → 0%*
6.3 부인방지성 Non-repudiation	**감사 로그 완전성 (Audit Log Completeness)**	주요 행위·거래에 대한 감사 로그가 누락 없이 기록되는 비율	행위 이벤트 대비 로그 기록 여부 교차 검증 *SIEM, 감사로그시스템*	**ALC(%) = 로그 기록 행위 수 / 전체 대상 행위 수 × 100** *주요 거래 5,000건 중 5,000건 기록 → 100%*
	전자서명 검증 성공률 (Digital Signature Verification)	전자서명·타임스탬프를 통한 행위 증명이 정상 검증되는 비율	서명 검증 테스트 케이스 실행 *PKI 검증 도구, 전자서명 SDK*	**DSVR(%) = 검증 성공 건수 / 전체 서명 검증 시도 건수 × 100** *서명 검증 1,200건 전건 성공 → 100%*
6.4 책임성 Accountability	**행위-주체 추적률 (Action Traceability Rate)**	시스템 내 주요 행위가 수행 주체(사용자/시스템)까지 추적 가능한 비율	로그 기반 행위-주체 매핑 검증 *로깅 시스템, IAM 감사 도구*	**ATR(%) = 주체 추적 가능 행위 수 / 전체 주요 행위 수 × 100** *주요 행위 2,000건 중 1,996건 추적 → 99.8%*
	책임 소재 규명 소요시간 (Attribution Resolution Time)	사고 발생 시 책임 주체를 규명하는 데 걸리는 평균 시간	사고 대응(Incident Response) 기록 분석 *사고관리시스템, 포렌식 도구*	**ART = Σ규명소요시간_i / 사고 건수** *사고 6건 평균 규명 소요시간 2.4시간*

3. AI 기반 시스의 품질 > 3.1 AI 시스템 품질
바. 보안성



참고 #6. 보안성 (Security) 측정 지표 예시

부특성	측정 지표 (Metrics)	설명	측정방법 · 도구	산출 공식 · 예시
6.5 인증성 Authenticity	**인증 성공률** **(Authentication Success Rate)**	정당한 사용자의 인증 시도가 정상 처리되는 비율	인증 로그 기반 성공·실패 집계 *IAM 시스템, 인증 로그 분석 도구*	**ASR(%) = 정상 인증 성공 건수 / 전체 정당 인증 시도 건수 × 100** *정당 로그인 50,000건 중 49,850건 성공 → 99.7%*
	위조 신원 탐지율 **(Spoofing Detection Rate)**	위조·도용된 신원의 인증 시도를 탐지·차단하는 비율	위조 계정 시뮬레이션 테스트 *부정탐지(Fraud Detection) 시스템, WAF*	**SDR(%) = 탐지된 위조 시도 수 / 전체 위조 시도 수 × 100** *위조 시도 120건 중 117건 탐지 → 97.5%*
6.6 개입 가능성 (Intervenability)	**긴급 중단 성공률** **(Emergency Stop Success)**	위험 상황에서 운영자의 긴급 중단(Kill Switch) 요청이 성공적으로 수행되는 비율	긴급중단 시나리오 테스트 반복 실행 *안전성 테스트 하네스, 카오스 엔지니어링 도구*	**ESSR(%) = 성공 중단 건수 / 전체 중단 시도 건수 × 100** *중단 시나리오 20건 전건 성공 → 100%*
	개입 소요시간 **(Time to Intervene)**	위험 징후 감지부터 운영자 개입이 완료되기까지 걸리는 시간	감지 시점~개입 완료 시점 로그 비교 *모니터링·알림 시스템, 사고 대응 로그*	**TTI = T(개입 완료) − T(위험 감지)** *평균 개입 소요시간 45초*

3. AI 기반 시스의 품질 > 3.1 AI 시스템 품질
바. 보안성 – 보안성 테스트



구분	진단 업무	점검 내용	점검 방식	수행팀
어플리케이션 취약점 진단	모의해킹	실제 사용자 관점에서의 취약점 진단	수동점검	보안팀 AA
	웹 진단	웹 스캐너 기반의 URL 전수 진단	App Scan	보안팀 AA
	소스코드 진단	소스코드 스캐너 기반의 보안 코딩 적용 여부 점검	HP Fortify	보안팀 AA
IT 인프라 취약점 진단	서버진단	서버 내 구동 중인 서비스에 대한 취약점 진단	Rapid7 Nexpose	보안팀 TA
	설정진단	OS, WEB서버, WAS서버, DBMS의 시스템 자체 보안 설정의 적정성 여부 점검	스크립트	보안팀 TA
네트워크 진단	네트워크 장비 설정 진단	라우터/스위치장비의 보안 설정의 적정성 여부 점검	수동점검	보안팀 TA
	네트워크 구조 진단	네트워크 구성 진단	수동점검	보안팀 TA
보안장비 진단	보안장비 설정 진단	보안장비 보안 설정의 적정성 여부 점검	수동점검	보안팀 TA
	방화벽 룰셋 진단	방화벽 ACL 적정성 점검	수동점검	보안팀 TA

Sample



3. AI 기반 시스의 품질 > 3.1 AI 시스템 품질

사. 유지보수성

유지보수성 (Maintainability)

의도된 유지보수자에 의해 제품/시스템이 얼마나 효과적·효율적으로 수정될 수 있는지의 정도

25010 기반 변경 (25059) 신규 (25059)

코드	부특성	정의	구분
7.1	모듈성 Modularity	하나의 구성요소 변경이 다른 구성요소에 미치는 영향을 최소화하도록 시스템이 개별 구성요소로 구성된 정도	-
7.2	재사용성 Reusability	하나 이상의 시스템에서 자산을 사용하거나, 새로운 자산을 만들기 위해 기존 자산을 사용할 수 있는 정도	-
7.3	분석성 Analysability	변경 영향을 평가하거나 결함·고장의 원인을 진단하거나 수정 필요 부분을 식별할 수 있는 정도	-
7.4	수정성 Modifiability	품질 저하나 기존 제품에 대한 부작용 없이 제품/시스템을 효과적·효율적으로 수정할 수 있는 정도	-
7.5	시험성 Testability	테스트 기준을 수립하고, 그 기준이 충족되었는지 판별하기 위해 테스트를 수행할 수 있는 정도	-

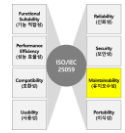
정적 리뷰를 통해 소스의 모듈성/재사용성 검토
모듈 복잡도, 모듈간 결합도 테스트는 도구 사용 (McCabe, SonarQube)

3. AI 기반 시스의 품질 > 3.1 AI 시스템 품질
사. 유지보수성



참고 #7. 유지보수성 (Maintainability) 측정 지표 예시

부특성	측정 지표 (Metrics)	설명	측정방법 · 도구	산출 공식 · 예시
7.1 모듈성 Modularity	**모듈 간 결합도 (Coupling Between Objects)**	모듈(클래스) 간 상호 의존성 수준	정적 코드 분석을 통한 결합도 지표 산출 *SonarQube, CodeMR*	**CBO = 특정 모듈이 의존하는 외부 모듈 수** *평균 CBO 4.2(권장 임계치 5 이하 충족)*
	순환 복잡도 (Cyclomatic Complexity, CC)	코드 안의 제어 흐름(분기점)이 많아질수록 복잡도 점수가 높아짐	정적 코드 분석 도구를 통한 CC산출 *SonarQube, ESLint*	**CC = P + 1 (여기서 P는 if, while, for, case 등 조건 분기문의 개수)** *10 이하 낮음, 11~20 보통, 21 ~50 높음, 51이상 매우 높음*
7.2 재사용성 Reusability	**코드 재사용률 (Code Reuse Ratio)**	전체 코드 중 공통 모듈·라이브러리로 재사용되는 코드의 비율	정적 분석을 통한 중복·재사용 코드 비율 산출 *SonarQube(Duplication), CodeScene*	**CRR(%) = 재사용 코드 라인 수 / 전체 코드 라인 수 × 100** *5만 라인 중 1.2만 라인 재사용 → 24%*
	재사용 컴포넌트 활용 건수 (Reused Component Adoption)	등록된 공통 컴포넌트가 다른 프로젝트·모듈에서 활용된 횟수	컴포넌트 레지스트리 기반 활용 이력 집계 *사내 컴포넌트 레지스트리, 패키지 매니저 로그*	**활용 건수 = Σ(컴포넌트별 참조 프로젝트 수)** *공통 인증모듈이 8개 프로젝트에서 재사용*
7.3 분석성 Analysability	**평균 결함 원인 분석 시간 (MTTD)**	결함 발생 후 원인을 특정하기까지 걸리는 평균 시간	결함관리시스템 기록 기반 신고~원인규명 시간 집계 *Jira, 결함관리시스템*	**MTTD = Σ원인규명소요시간_i / 결함 건수** *결함 30건 평균 원인분석 시간 3.1시간*
	영향 범위 예측 정확도 (Change Impact Prediction)	변경 영향 분석에서 예측한 영향 범위와 실제 영향 범위의 일치도	변경 전 영향분석 결과와 실제 회귀테스트 결과 비교 *정적 분석 기반 영향분석 도구, 형상관리 이력*	**CIPA(%) = 예측·실제 일치 모듈 수 / 실제 영향 모듈 수 × 100** *실제 12개 모듈 중 11개 사전 예측 일치 → 91.7%*



3. AI 기반 시스의 품질 > 3.1 AI 시스템 품질
사. 유지보수성

참고 #7. 유지보수성 (Maintainability) 측정 지표 예시

부특성	측정 지표 (Metrics)	설명	측정방법 · 도구	산출 공식 · 예시
7.4 수정성 Modifiability	**변경 실패율** **(Change Failure Rate)**	배포된 변경 중 장애나 롤백을 유발한 변경의 비율	배포 이력과 장애·롤백 이력 교차 분석 *CI/CD 파이프라인 로그, 배포관리시스템*	**CFR(%) = 장애 유발 배포 건수 / 전체 배포 건수 × 100** *배포 200건 중 6건 장애 유발 → 3%*
	코드 수정 소요시간 **(Lead Time for Changes)**	변경 요청부터 코드 수정 완료까지 걸리는 평균 시간	형상관리시스템 커밋 이력 기반 소요시간 산출 *Git, Jira, DORA 메트릭 대시보드*	**LTC = Σ(수정완료−요청접수) / 변경 건수** *평균 변경 리드타임 1.8 일*
7.5 시험성 Testability	**테스트 커버리지** **(Test Coverage)**	전체 코드·기능 중 테스트 케이스로 검증되는 비율	코드 커버리지 측정 도구를 통한 자동 산출 *JaCoCo, Istanbul, Coverage.py*	**TC(%) = 테스트 실행 코드 라인 수 / 전체 코드 라인 수 × 100** *단위테스트 커버리지 84%*
	테스트 자동화율 **(Test Automation Rate)**	전체 테스트 케이스 중 자동화되어 실행되는 비율	테스트관리시스템 기반 수동·자동 케이스 집계 *TestRail, Zephyr, CI/CD 파이프라인*	**TAR(%) = 자동화 테스트케이스 수 / 전체 테스트 케이스 수 × 100** *테스트케이스 800건 중 620건 자동화 → 77.5%*



3. AI 기반 시스의 품질 > 3.1 AI 시스템 품질

아. 이식성

이식성 (Portability)

하나의 하드웨어·소프트웨어 또는 다른 운영/사용 환경에서 다른 환경으로 시스템/제품/구성요소가 얼마나 효과적·효율적으로 이전될 수 있는지의 정도

25010 기반 변경 (25059) 신규 (25059)

코드	부특성	정의	구분
8.1	적응성 Adaptability	제품/시스템이 다른 하드웨어, 소프트웨어 또는 운영/사용 환경에 효과적·효율적으로 적응할 수 있는 정도	-
8.2	설치성 Installability	제품/시스템이 명시된 환경에 성공적으로 설치·제거될 수 있는 정도	-
8.3	대체성 Replaceability	동일한 목적을 위해 동일한 환경에서 다른 지정된 소프트웨어 제품을 대체할 수 있는 정도	-

호환성 테스트 (설치, UI/UX, 기능)



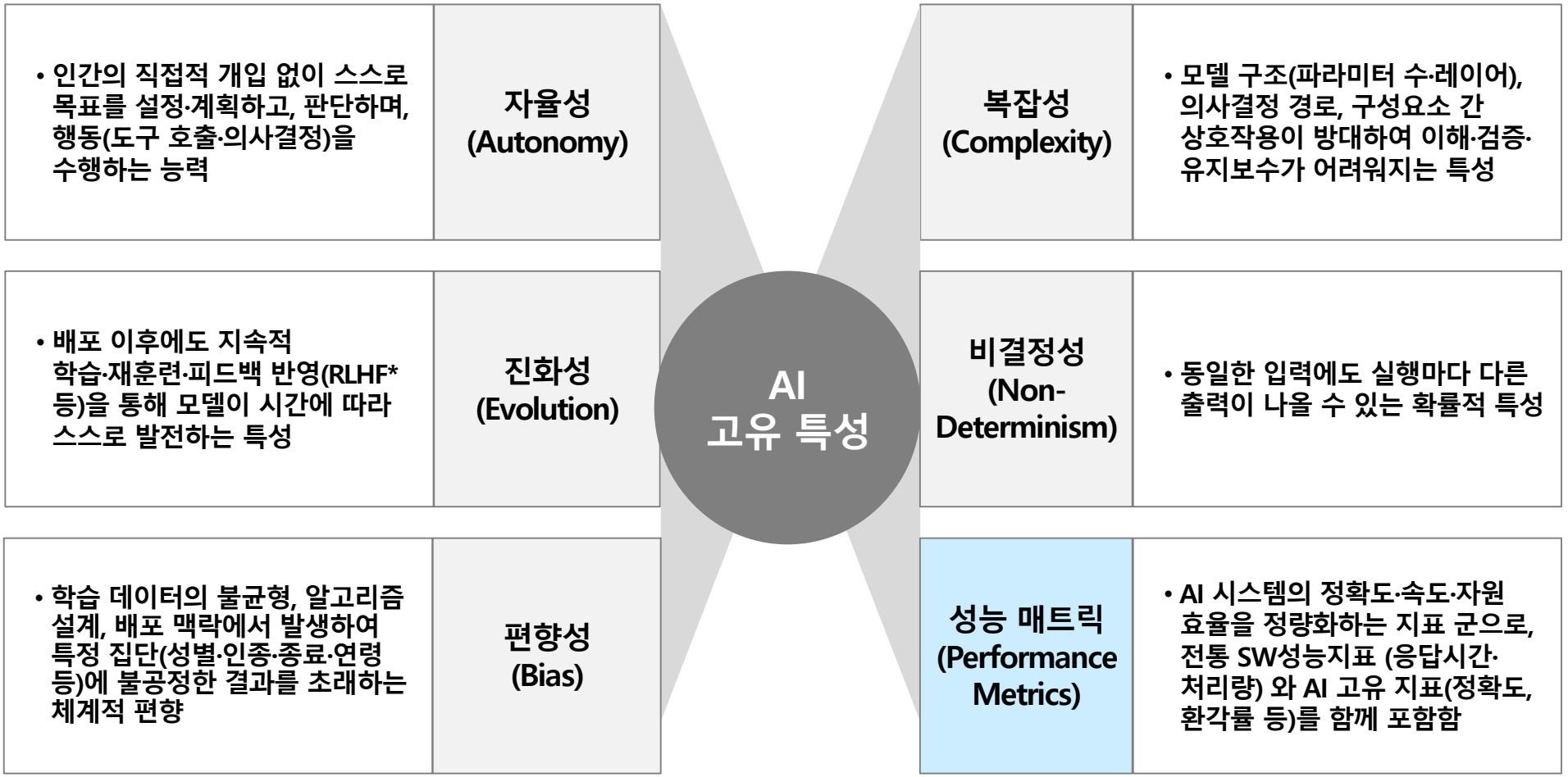
3. AI 기반 시스의 품질 > 3.1 AI 시스템 품질
아. 이식성

참고 #8. 이식성 (Portability) 측정 지표 예시

부특성	측정 지표 (Metrics)	설명	측정방법 · 도구	산출 공식 · 예시
8.1 적응성 Adaptability	환경 이식 성공률 (Environment Portability Success)	대상 환경(OS·클라우드 등)으로 이전 시 정상 동작하는 비율	다중 환경(OS·브라우저·클라우드) 배포 테스트 *Docker, Kubernetes, 크로스플랫폼 테스트 스위트*	**EPSR(%) = 정상 동작 환경 수 / 전체 테스트 환경 수 × 100** *대상 환경 6개 중 6개 정상 동작 → 100%*
	환경 설정 변경 소요시간 (Reconfiguration Time)	신규 환경에 적응시키기 위해 필요한 설정 변경 소요 시간	신규 환경 배포 시 설정 변경 작업 시간 측정 *IaC 도구(Terraform, Ansible), 배포 로그*	**ERT = 설정 완료 시각 − 설정 시작 시각** *신규 클라우드 환경 적응 평균 4시간*
8.2 설치성 Installability	설치 성공률 (Installation Success Rate)	설치 시도 중 오류 없이 성공적으로 완료되는 비율	다양한 환경에서 반복 설치 테스트 *설치 자동화 스크립트, CI 파이프라인*	**ISR(%) = 설치 성공 횟수 / 전체 설치 시도 횟수 × 100** *설치 테스트 50회 중 49회 성공 → 98%*
	평균 설치 소요시간 (Mean Installation Time)	설치 시작부터 정상 사용 가능 상태까지 걸리는 평균 시간	설치 로그 기반 소요시간 측정 *설치 자동화 스크립트, 배포 로그*	**MIT = Σ설치소요시간_i / 설치 횟수** *평균 설치 소요시간 6.5분*
8.3 대체성 Replaceability	마이그레이션 성공률 (Migration Success Rate)	기존 제품에서 신규 제품으로 전환 시 정상 이전되는 기능·데이터의 비율	마이그레이션 리허설 테스트를 통한 검증 *데이터마이그레이션 도구, 검증 스크립트*	**MSR(%) = 정상 이전 항목 수 / 전체 이전 대상 항목 수 × 100** *데이터 테이블 40개 전건 정상 이전 → 100%*
	병행 운영 호환률 (Parallel-run Compatibility)	신규 시스템이 기존 시스템과 병행 운영 시 동일 결과를 산출하는 비율	신구 시스템 병행운영(Shadow Test) 후 결과 비교 *병행운영 테스트 하네스, 결과 비교 스크립트*	**PCR(%) = 동일 결과 산출 건수 / 전체 비교 건수 × 100** *병행 처리 1,000건 중 998건 동일 → 99.8%*

3. AI 기반 시스의 품질
3.2 AI 고유 특성

AI 고유 특성



* RLHF(Reinforcement Learning from Human Feedback)는 인간의 피드백을 강화학습에 활용

3. AI 기반 시스의 품질 > 3.2 AI 고유 특성
가. 자율성

◆ 정의

인간의 직접적 개입 없이 스스로 목표를 설정·계획하고, 판단하며, 행동(도구 호출·의사결정)을 수행하는 능력. Agentic AI의 핵심 특성으로, 자율 수준(Level of Autonomy)에 따라 검증 범위와 리스크가 달라진다.

◆ 핵심 특징

- 계획(Planning)-실행(Action)-반영(Reflection) 루프를 통한 자기주도적 작업 수행
- 도구 사용(Tool Use), 멀티 에이전트 협업 시 자율성 범위 정의 필요
- 자율성이 높을수록 인간 감독(Human-in-the-loop) 설계와 안전장치가 중요

적용 사례

자율 주행

인지-판단-제어를 스스로 수행하여 주행 경로를 결정

AI 에이전트

업무 목표만 주어지면 스스로 도구를 선택해 작업을 완수(Agentic AI)

메트릭	설명	측정방법 · 도구	산출 공식 · 예시
자율 작업 완료율 (Autonomous Task Completion Rate)	인간 개입 없이 끝까지 완료된 작업의 비율	작업 로그 기반 성공/실패 라벨링 후 집계 *LangSmith, Agent 실행 로그, 커스텀 트래커*	**ATCR(%) = 완료된 자율 작업 수 / 전체 시도 작업 수 × 100** *고객문의 100건 중 82건을 상담원 개입 없이 자율 해결 → 82%*
인간 개입 빈도 (Human Intervention Rate)	전체 작업 대비 인간이 개입(수정·승인·중단)한 비율	휴먼-인-더-루프 로그 분석 *감사 로그, LangSmith, Airflow 로그*	**HIR = 개입 발생 건수 / 전체 작업 건수** *1,000건 자율 처리 중 45건 인간 승인 필요 → HIR=4.5%*
자율 의사결정 정확도 (Decision Accuracy)	자율적으로 내린 의사결정이 전문가 판단과 일치하는 비율	전문가 레이블 대비 에이전트 결정 비교(Gold Set) *LLM-as-Judge, 전문가 평가 패널*	**DA(%) = 전문가 판단과 일치한 결정 수 / 전체 결정 수 × 100** *대출 승인 자율판단 300건 중 279건 전문가 판단과 일치 → 93%*

3. AI 기반 시스의 품질 > 3.2 AI 고유 특성
나. 진화성

◆ 정의

배포 이후에도 지속적 학습·재훈련·피드백 반영(RLHF 등)을 통해 모델이 시간에 따라 발전하는 특성. **새로운 지식을 습득하면서 기존 지식을 잃지 않는 '지속 학습(Continual Learning)' 능력이 핵심**이다.

◆ 핵심 특징

- 파인튜닝·RLHF·인간 피드백 루프를 통한 세대 간(Version-to-Version) 성능 개선
- 치명적 망각(Catastrophic Forgetting) 방지가 진화 품질의 핵심 리스크
- 버전 관리 및 회귀 테스트(Regression Test) 체계와 결합되어야 함

적용 사례

LLM RLHF 개선

사용자 피드백을 반영해 다음 버전 모델의 응답 품질을 개선

* RLHF (Reinforcement Learning from Human Feedback, 인간 피드백 기반 강화학습)

지속 학습 모델

신규 도메인 데이터 유입 시 기존 지식 손실 없이 지식 확장

메트릭	설명	측정방법 · 도구	산출 공식 · 예시
지식 보존율 (Knowledge Retention)	신규 학습 이후에도 기존 과업 성능이 유지되는 정도(망각 방지 지표)	신규 학습 전/후 기존 태스크 벤치마크 재평가 *커스텀 회귀 벤치마크셋, MLflow*	**KR(%) = Acc_기존과업(신규학습후) / Acc_기존과업(신규학습전) × 100** *신규 도메인 학습 후 기존 QA 정확도 유지율 96%*
버전 간 개선도 (Version-over-Version Improvement)	연속 버전 간 동일 벤치마크 기준 성능 향상 폭	동일 평가셋으로 버전별 벤치마크 비교 *RAGAS, DeepEval, MLflow Model Registry*	**ΔP = Score_v(n) − Score_v(n−1)** *v2.1 대비 v2.2 벤치마크 F1 +3.4%p 향상*
학습 곡선 수렴 속도 (Convergence Rate)	재학습 시 목표 성능에 도달하기까지 필요한 학습 스텝/데이터량	학습 로그의 loss·metric 곡선 분석 *TensorBoard, MLflow, Weights & Biases*	**필요 Epoch(또는 Step) 수로 비교** *이전 세대 대비 목표 정확도 도달 Epoch 12→8로 단축*

3. AI 기반 시스의 품질 > 3.2 AI 고유 특성
다. 편향성

◆ 정의

재학습 데이터의 불균형, 알고리즘 설계, 배포 맥락에서 발생하여 **특정 집단(성별·인종·연령 등)에 불공정한 결과를 초래**하는 체계적 편향. 공정성(Fairness) 품질과 직결되며 **법적·윤리적 리스크가 크다.**

◆ 핵심 특징

- 데이터 편향(수집·라벨링), 알고리즘 편향, 상호작용 편향 등 발생 지점이 다양
- 전체적으로 정확도가 높아도 특정 집단에서 성능(편향 발생)이 낮으면 품질 결함으로 간주
- 완화(Mitigation)는 전처리·인프로세싱·후처리 단계별로 접근 가능

적용 사례

채용 스크리닝 AI

이력서 성별·출신학교 정보에 따라 합격률이 유의하게 차이

얼굴 인식

피부색·성별 그룹 간 인식 정확도 편차 발생

메트릭	설명	측정방법 · 도구	산출 공식 · 예시
통계적 동등성 차이 (Statistical Parity Difference)	보호 집단 간 긍정 예측 비율의 차이	집단별 예측 결과 분포 비교 *AIF360, Fairlearn*	**SPD = P(Ŷ=1|A=0) − P(Ŷ=1|A=1)** *남성 합격률 42% vs 여성 합격률 31% → SPD = 0.11*
균등 기회 차이 (Equal Opportunity Difference)	실제 정답(Y=1)인 집단 내에서 재현율(TPR) 격차	혼동행렬 기반 집단별 TPR 산출·비교 *Fairlearn, AIF360, What-If Tool*	**EOD = TPR(A=0) − TPR(A=1)** *적격자 재현율 집단A 88% vs 집단B 76% → EOD=0.12*
불균형 영향 비율 (Disparate Impact)	비보호 집단 대비 보호 집단의 긍정 예측 비율 비(80% 규칙 기준)	집단별 긍정 예측 비율의 비율 산출 *AIF360, IBM Fairness 360 Dashboard*	**DI = P(Ŷ=1|A=0) / P(Ŷ=1|A=1) (통상 0.8 미만이면 위험)** *DI=0.65 → 법적 기준(0.8) 미달로 편향 위험 판정*

3. AI 기반 시스의 품질 > 3.2 AI 고유 특성
라. 복잡성

◆ 정의

모델 구조(파라미터 수·레이어), 의사결정 경로, 구성요소 간 상호작용이 방대하여 이해·검증·유지보수가 어려워지는 특성.
복잡성이 높을수록 테스트 커버리지 확보와 결함 원인 추적이 어려워진다.

◆ 핵심 특징

- 수천억 개 파라미터, 다층 어텐션 구조로 인한 블랙박스화
- 구성요소(프롬프트·RAG·에이전트·툴) 조합형 시스템일수록 복잡도 상승
- 복잡도 증가는 곧 테스트·검증 비용 및 예측불가 리스크의 증가로 직결

적용 사례

대규모 LLM
수천억 파라미터·다층 트랜스포머 구조로 내부 동작 해석이 어려움

Agentic 파이프라인
다중 에이전트·툴 체인 조합으로 실패 지점 추적이 복잡해짐

메트릭	설명	측정방법 · 도구	산출 공식 · 예시
모델 파라미터 규모 (Parameter Count)	모델의 학습 가능한 파라미터 총량 (구조적 복잡도의 기본 지표)	모델 아키텍처 정의로부터 파라미터 수 집계 *PyTorch summary, HuggingFace model card*	**Params = Σ(레이어별 가중치·편향 개수)** *7B 파라미터 모델 → 70억 개*
추론 연산량 (FLOPs)	1회 추론에 필요한 부동소수점 연산량(계산 복잡도)	모델 프로파일링 도구로 연산 그래프 분석 *DeepSpeed Profiler, PyTorch Profiler*	**FLOPs ≈ 2 × Params × 처리 토큰 수** *7B 모델, 500토큰 입력 시 약 7×10^{12} FLOPs*
구성요소 상호작용 수 (Component Interaction Count)	파이프라인 내 모듈(프롬프트·RAG·툴·에이전트) 간 호출·연결 수	실행 트레이스(Trace)에서 호출 그래프 추출 *LangSmith, OpenTelemetry Trace*	**CIC = 파이프라인 내 모듈 간 호출(엣지) 총 수** *멀티에이전트 워크플로우 평균 호출 18회/요청*

3. AI 기반 시스템의 품질 > 3.2 AI 고유 특성
마. 비결정성

◆ 정의

동일한 입력에도 실행마다 다른 출력이 나올 수 있는 확률적 특성.
샘플링(Temperature·Top-p), 병렬 연산의 부동소수점 오차, 모델 내부 확률분포 등에 기인하며, 전통 SW 테스트의 '재현 가능성' 전제를 무너뜨린다.

◆ 핵심 특징

- 생성형 AI의 샘플링 파라미터(Temperature, Top-p, Top-k)가 주요 발생 원인
- GPU 병렬 연산 순서 비결정성으로 동일 설정에도 미세한 차이 발생 가능
- 테스트 시 '정답 일치'가 아닌 '허용 범위 내 일관성' 기준으로 전환 필요

사례

LLM 응답 변동
동일 프롬프트에도 실행마다 표현·구조가 달라짐

이미지 생성 모델
동일 시드가 아니면 매번 다른 결과 이미지를 생성

메트릭	설명	측정방법 · 도구	산출 공식 · 예시
출력 일관성 점수 (Output Consistency)	동일 입력을 N회 반복 실행했을 때 동일/유사 출력이 나오는 비율	동일 프롬프트 N회 반복 실행 후 출력 비교 (temperature=0 포함) *커스텀 반복 실행 하네스, DeepEval*	**OC(%) = 일치(또는 임계 유사) 출력 수 / N × 100** *동일 질의 20회 반복 중 17회 의미적으로 일치 → 85%*
응답 의미 유사도 분산 (Semantic Variance)	반복 응답 간 임베딩 코사인 유사도의 분산(낮을수록 안정적)	응답 임베딩 생성 후 쌍별 코사인 유사도의 분산 계산 *Sentence-BERT, OpenAI Embeddings*	**SV = Var(cosine(emb_i, emb_j)), i≠j (N회 반복)** *20회 반복 응답 임베딩 유사도 분산 0.015(높은 안정성)*
재현율 (Reproducibility Rate)	고정 시드·동일 환경에서 완전히 동일한 출력이 재현되는 비율	시드 고정 후 동일 환경 재실행 비교 *실험 재현 파이프라인, MLflow*	**RR(%) = 완전 일치 재현 횟수 / 전체 재실행 횟수 × 100** *시드 고정 100회 재실행 중 100회 동일 출력 → 100%*

3. AI 기반 시스의 품질 > 3.2 AI 고유 특성

바. 성능 메트릭

◆ 정의

AI 시스템의 정확도·속도·자원 효율을 정량화하는 지표 군으로, 전통 SW 성능지표(응답시간·처리량)와 AI 고유 지표(정확도, 환각률 등)를 함께 포괄한다. 지속적 모니터링 없이는 운영 중 성능 저하(Model Decay)를 조기에 발견하기 어렵다.

◆ 핵심 특징

- 품질(정확도)·속도(지연시간)·비용(자원효율) 3축을 균형있게 관리
- 생성형 AI는 환각률(Hallucination Rate) 등 AI 고유 품질지표가 추가됨
- 운영 중 지속 모니터링(Observability)이 성능 열화를 조기에 탐지하는 핵심

적용 사례

LLM 서비스 SLA

TTFT(첫 토큰 응답시간)·초당 처리 토큰 수로 사용자 체감 속도를 관리

품질 모니터링

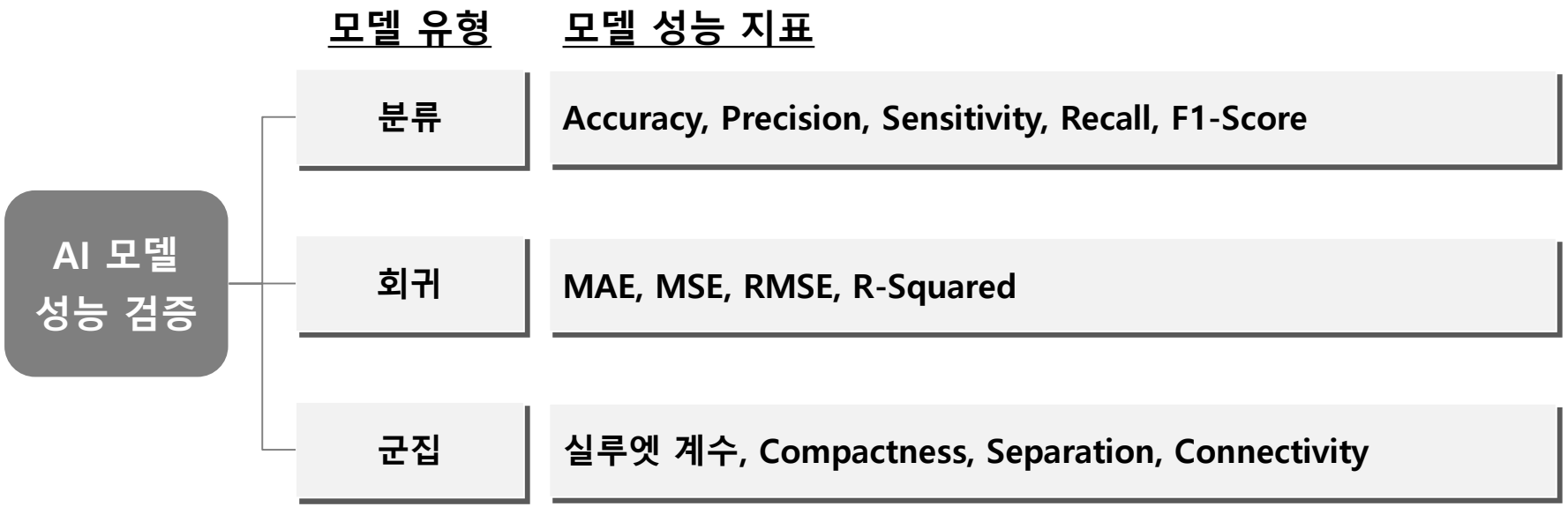
운영 중 환각률·정확도 추이를 대시보드로 지속 추적

메트릭	설명	측정방법 · 도구	산출 공식 · 예시
정확도/F1-Score	예측 결과가 실제 정답과 일치하는 정도(분류/생성 과업 공통 핵심 지표)	검증셋 기준 정밀도·재현율 산출 후 조화 평균 계산 *scikit-learn, RAGAS, DeepEval*	**F1 = 2 × (Precision × Recall) / (Precision + Recall)** *QA 벤치마크 F1 0.89*
응답 지연시간 (Latency: TTFT/TPOT)	요청 후 첫 토큰까지의 시간(TTFT), 토큰당 생성 시간(TPOT)	서빙 레이어에서 요청-응답 타임스탬프 측정 *Grafana, Prometheus, vLLM 메트릭*	**TTFT = T(첫 토큰 반환) − T(요청 접수)** *평균 TTFT 280ms, TPOT 18ms/토큰*
환각률 (Hallucination Rate)	생성 답변 중 근거 문서·사실과 불일치하는 내용이 포함된 비율	근거 문서 대조(Faithfulness) 자동/휴먼 평가 *RAGAS(Faithfulness), G-Eval, 휴먼 평가*	**HR(%) = 사실 불일치 응답 수 / 전체 평가 응답 수 × 100** *RAG 응답 500건 평가 중 21건 사실 불일치 → 4.2%*

3. AI 기반 시스의 품질 > 3.2 AI 고유 특성

차. 성능 메트릭

- **성능 메트릭 (Performance Metrics)**
  - ✓ 기계 학습(ML)모델에 대해 정의된 성능 메트릭으로는 **정확도**, **정밀도**, **재현율**이 가장 널리 사용됨
  - ✓ 이 메트릭은 시스템 **요구사항의 일부로 합의되고 정의**되어야 한다.
  - ✓ 모델 성능 테스트는 종종 기계 학습(ML) 프레임워크(예, 텐서플로우, PyTorch)의 일부로 제공되며, 주어진 테스트 데이터 세트에 대한 이 측정값을 계산한다.



3. AI 기반 시스의 품질 > 3.2 AI 고유 특성 > 차. 성능 매트릭

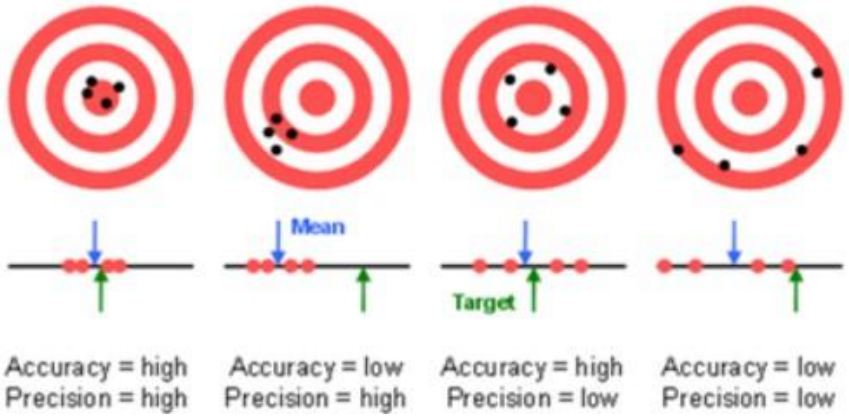
가). 분류 모델 성능 검증 지표 (1/3)

1. Confusion Matrix (오차행렬, 혼동행렬)

구분		예측	
		Positive	Negative
실제	Positive	TP (True Positive)	FN (False Negative)
	Negative	FP (False Positive)	TN (True Negative)

- 앞의 T, F는 실제값과 예측값이 일치하는지 여부
- 뒤의 P, N는 예측값의 Positive, Negative 의미

[Accuracy와 Precision 관계]



정확도 (Accuracy)

- 전체 데이터 수 중 **예측 결과와 실제 값이 동일한 건수**(TN + TP)가 차지하는 비율
- **(TN + TP) / (TN + FP + FN + TP)**

정밀도 (Precision)

- **예측**을 **Positive**로 한 대상(FP + TP) 중 예측과 실제 값이 Positive로 일치한 데이터(TP)의 비율
- **TP / (FP + TP)**

  = 양성 예측도
- **스팸메일 분류 모델**과 같이 실제 Negative(일반메일)인 데이터를 Positive(스팸메일)로 잘못 판단하는 것이 큰 문제인 경우 정밀도가 상대적으로 더 중요

재현율 (Recall)

- **실제**가 **Positive**인 대상(FN + TP) 중 예측과 실제 값이 Positive로 일치한 데이터(TP)의 비율
- **TP / (FN + TP)**

  = **민감도**(Sensitivity) or TPR(True Positive Rate)
- **암 판단 모델**과 같이 실제 Positive(양성)인 데이터를 Negative(음성)로 잘못 판단하는 것이 큰 문제인 경우 재현율이 상대적으로 더 중요

3. AI 기반 시스의 품질 > 3.2 AI 고유 특성 > 차. 성능 매트릭

가). 분류 모델 성능 검증 지표 (2/3)

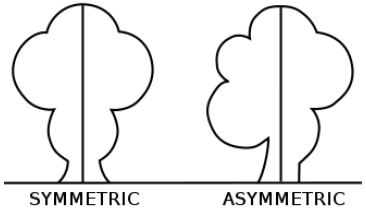
F1 Score

- **모델의 성능을 측정**하는데 있어서 **precision과 recall은 유용함**
- **모델이 얼마나 효과적**인지는 **F1 score를 사용**

- F1 Score 는 precision과 recall의 조화 평균
- **2 * [(Precision * recall) / (precision + recall)]**

- 일반적인 평균이 아닌 조화평균을 계산
  ➔ Precision과 Recall이 0에 가까울수록 F1 score도 낮은 값을 갖도록 하기 위함
  ➔ **조화평균은 작은 값 위주로 평균을 구하는 원리**

- 예를 들어 Recall =1 , Precision = 0.01로 측정된 모델이 있으면
- 2 * ((1* 0.01) / (1+0.01)) = 0.019



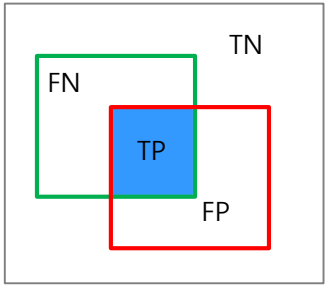
- **밸런스 데이터 면 Accuracy 사용**
  Accuracy = (TP + TN) / (TP + FN + FP +TN)
- **비대칭 데이터면 F1-score 사용**

3. AI 기반 시스의 품질 > 3.2 AI 고유 특성 > 차. 성능 매트릭

가). 분류 모델 성능 검증 지표 (3/3)

IOU (Intersection Over Union)

- **IOU (Intersection Over Union)**
- **개별 객체에 대한 검출이 성공하는지를 결정하는 지표**
- **0~1 사이의 값을 가짐 (1에 가까울 수록 좋음)**



실제 (Ground Truth)

예측 (Prediction)

- True Positive (TP) : 맞다고 추측하고 실제로 맞음. **IOU >= Threshold**
- False Positive (FP) : 맞다고 추측하고 실제로는 틀림. **IOU < Threshold**
- False Negative (FN) : 아니라고 추측하고 실제로는 맞음. Ground Truth를 아예 detect 못함
- True Negative (TN) 아니라고 추측하고 실제로 아님.

※ Threshold 는 일반적으로 50%, 75%, 95%

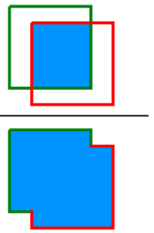
✓ Confidence 레벨은 검출한 것에 대해 알고리즘이 얼마나 확신이 있는지를 알려주는 값

✓ Confidence 레벨이 낮으면 그만큼 검출 결과에 대해서 자신이 없는 것

✓ 보통 Confidence 레벨에 대해 Threshold 값을 부여해서 특정 값 이상이 되어야 검출된 것으로 인정.

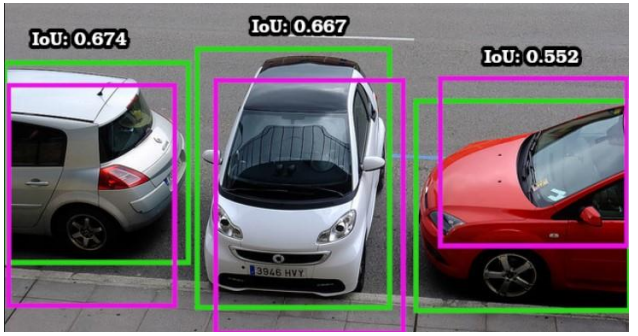
✓ Threshold 값이 0.4라면 0.1, 0.2 는 무시됨

$$IOU = \frac{\text{area of overlap}}{\text{area of union}} = \frac{\quad}{\quad}$$



$$\text{IOU} = \frac{\text{TP}}{\text{FN + TP + FP}}$$

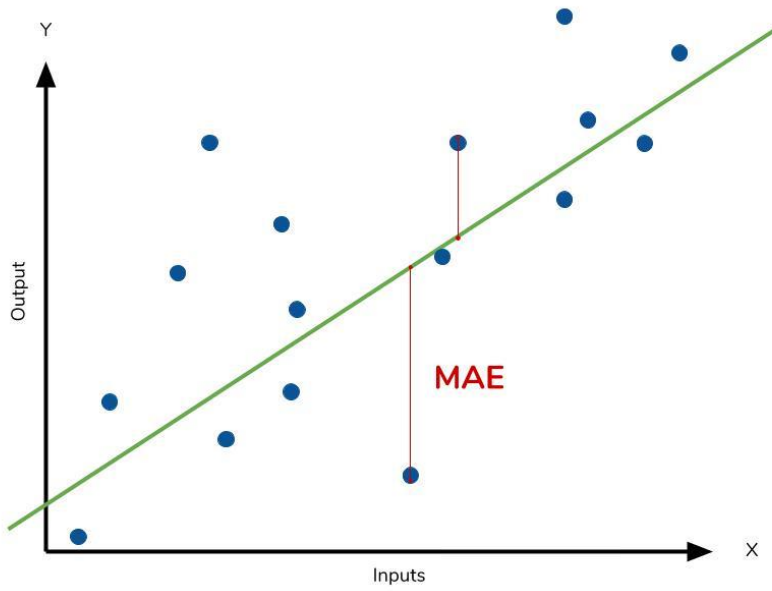




3. AI 기반 시스의 품질 > 3.2 AI 고유 특성 > 차. 성능 매트릭

나). 회귀 모델 성능 검증 지표 (1/5)

1. MAE (Mean Absolute Error)



[MAE]

- 실제 값과 예측 값의 차이(잔차*주1))를 절대값으로 변환해 평균화

$$\frac{1}{n}\sum_{i=1}^{n}(|y_i - \hat{y}_i|)$$

$\hat{y}$ – predicted value of y

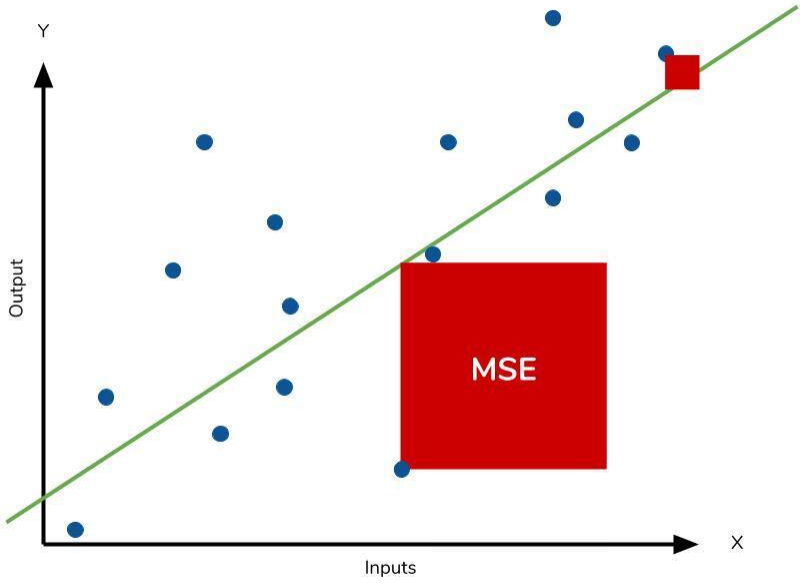
- MAE는 에러에 절대값을 취하기 때문에 에러의 크기 그대로 반영된다. 그러므로 예측 결과물의 에러가 10이 나온 것이 5로 나온 것보다 2배가 나쁜 도메인에서 쓰기 적합한 산식이다.
- 에러에 따른 손실이 선형적으로 올라갈 때 적합하다.
- 절대값을 취하기 때문에 모델이 underperformance 인지 overperformance 인지 알 수 없다.
- **MAE score가 높을수록 성능 낮음**

※주1) 실제 값과 추정한 값의 차이를 잔차(residual)라고 함

3. AI 기반 시스의 품질 > 3.2 AI 고유 특성 > 차. 성능 매트릭

나). 회귀 모델 성능 검증 지표 (2/5)

2. MSE (Mean Squared Error)



[MSE]

- 실제 값과 예측 값의 차이를 제곱해서 평균화

$$\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

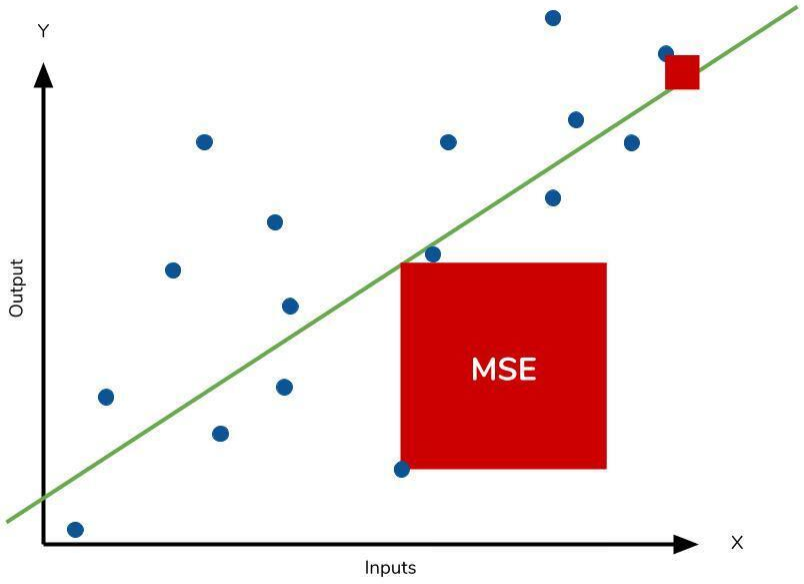
$\hat{y}$ – predicted value of y

- 예측값과 실제값 차이의 면적의 합

- 제곱을 하기 때문에 특이치(아웃라이어)에 민감하다.
- 특이값이 존재하면 수치가 많이 증가한다.
- **MSE score가 높을수록 성능 낮음**

3. AI 기반 시스의 품질 > 3.2 AI 고유 특성 > 차. 성능 매트릭
나). 회귀 모델 성능 검증 지표 (3/5)

3. RMSE (Root Mean Squared Error)



[MSE]

- MSE에 root를 씌운 값

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

$\hat{y}$ – predicted value of y

- MSE 값은 오류의 제곱을 구하므로 실제 오류 평균보다 더 커지는 특성이 있어 MSE에 root를 씌운 RMSE 값을 쓴다.
- 오류 지표를 실제 값과 유사한 단위로 다시 변환하기에 해석이 다소 용이해진다.
- 에러에 따른 손실이 기하 급수적으로 올라가는 상황에서 쓰기 적합하다.
- **RMSE score가 높을수록 성능 낮음**

3. AI 기반 시스의 품질 > 3.2 AI 고유 특성 > 차. 성능 매트릭

나). 회귀 모델 성능 검증 지표 (4/5)

4. MAE vs RMSE 사례 비교

No.	실제값	예측값	error
1	10	8	2
2	20	22	-2
3	30	32	2
4	40	38	2
5	50	52	-2

MAE	2.0
RMSE	2.0

No.	실제값	예측값	error
1	10	10	0
2	20	21	-1
3	30	28	2
4	40	37	3
5	50	54	-4

MAE	2.0
RMSE	2.4

No.	실제값	예측값	error
1	10	10	0
2	20	20	0
3	30	30	0
4	40	40	0
5	50	40	10

MAE	2.0
RMSE	**4.5**

$MAE = |y_{true} - y_{pred}|$의 평균

$RSME = \sqrt{(y_{true} - y_{pred})^2}$의 평균

$$\frac{1}{n}\sum_{i=1}^{n}(|y_i - \hat{y}_i|) \quad \hat{y} - predicted\ value\ of\ y$$

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \quad \hat{y} - predicted\ value\ of\ y$$

- **MAE : 전반적인 차이를 알 수 있지만 에러값이 큰 데이터의 영향을 반영하지 못함**
- **RMSE : 큰 차이의 에러값(이상치)에 민감하게 반응을 함**

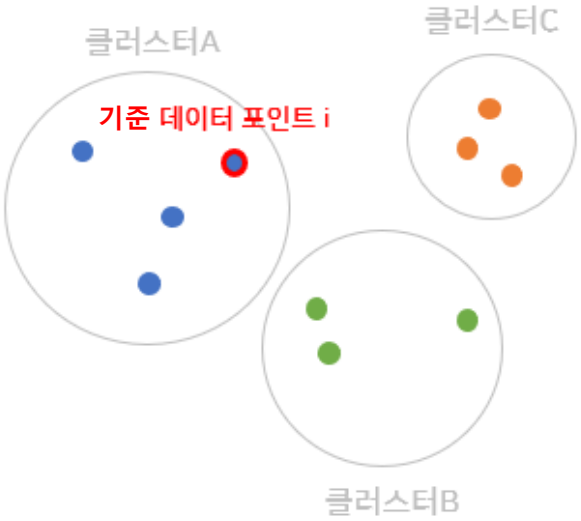
3. AI 기반 시스의 품질 > 3.2 AI 고유 특성 > 차. 성능 매트릭

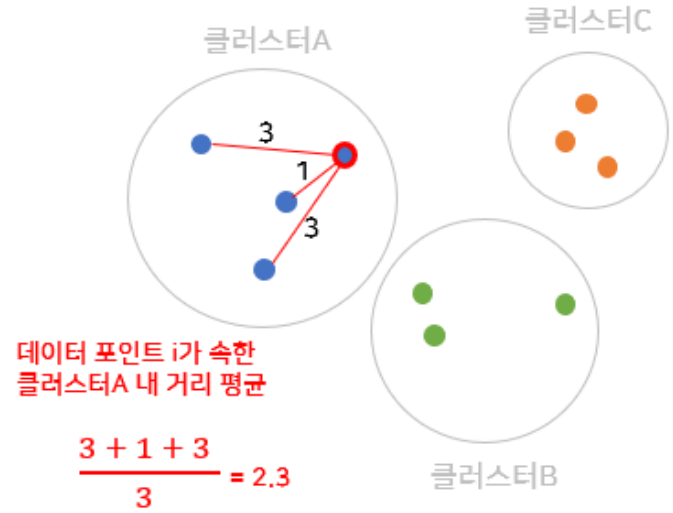
다). 군집 모델 성능 검증 지표 (1/3)

실루엣 계수 (Silhouette Coefficient) – 1/3

- 군집 안에 있는 데이터들은 **잘 모여있는지**, 군집끼리는 서로 **잘 구분되는지** 클러스터링을 평가하는 척도로 활용
- 각 데이터 포인트와 주위 데이터 포인트들과의 거리 계산을 통해 값을 구함

1. 기준 데이터 포인트(i)가 속한 클러스터 내 데이터 포인트들과 거리 평균

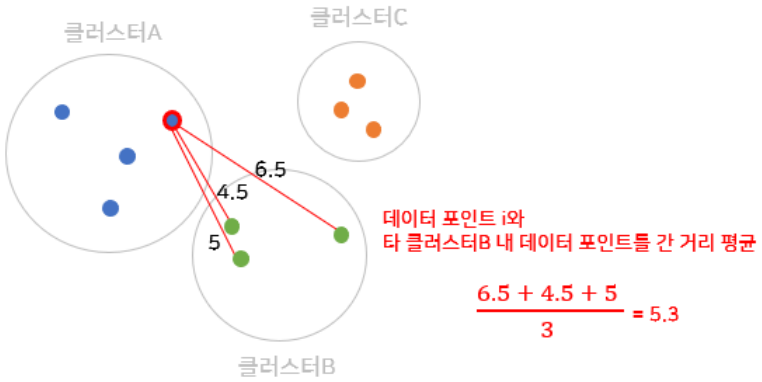


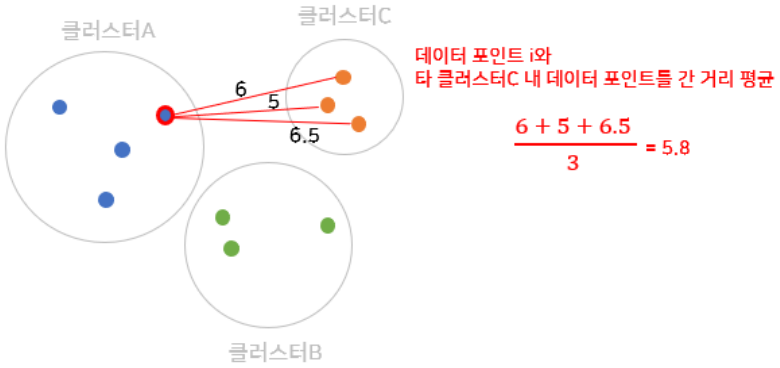


3. AI 기반 시스의 품질 > 3.2 AI 고유 특성 > 차. 성능 매트릭
다). 군집 모델 성능 검증 지표 (2/3)

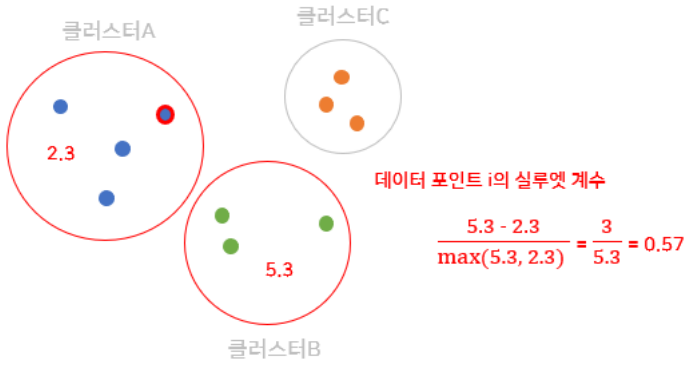
실루엣 계수 (Silhouette Coefficient) – 2/3

2. 기준 데이터 포인트가 속하지 않은 클러스터 B와 C의 데이터 포인트들과 거리 평균을 구함





3. 데이터 포인트 i와 가까운 클러스터(=이웃 클러스터)는 B이므로 b(i) 값은 5.3이 되며,
a(i), b(i) 값으로 데이터 포인트 i의 실루엣 계수 s(i) 값을 구함



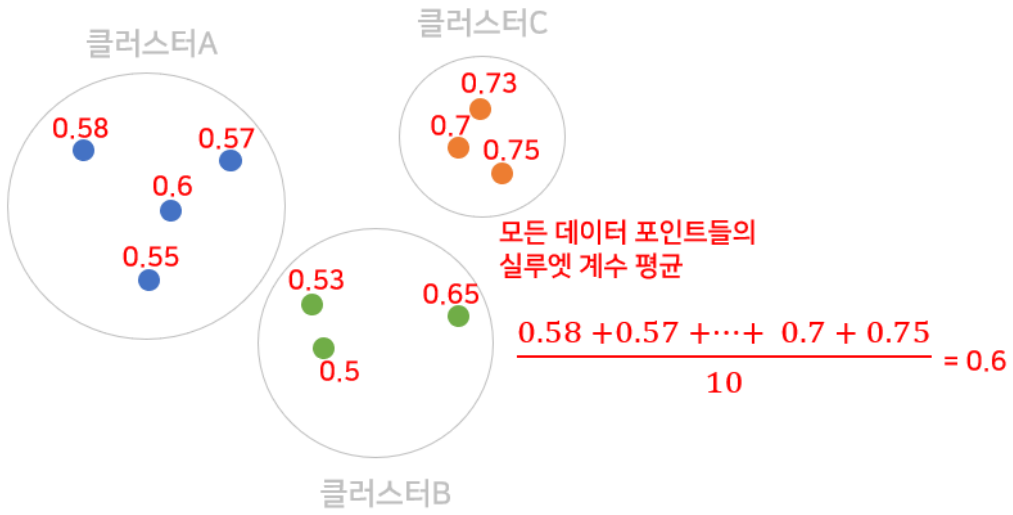
$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \quad -1 \le s(i) \le 1$$

3. AI 기반 시스의 품질 > 3.2 AI 고유 특성 > 차. 성능 매트릭

다). 군집 모델 성능 검증 지표 (3/3)

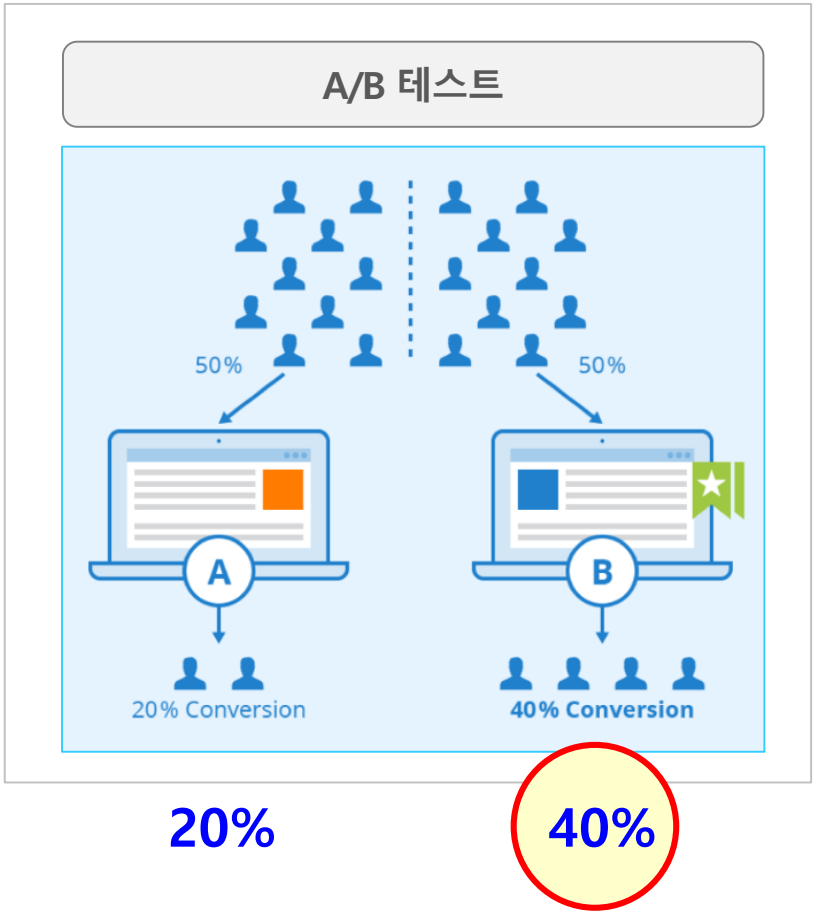
실루엣 계수 (Silhouette Coefficient) – 3/3

4. 모든 데이터 포인트들에 대해 실루엣 계수를 계산했다면, 실루엣 계수들의 평균값(overall average silhouette width)을 계산



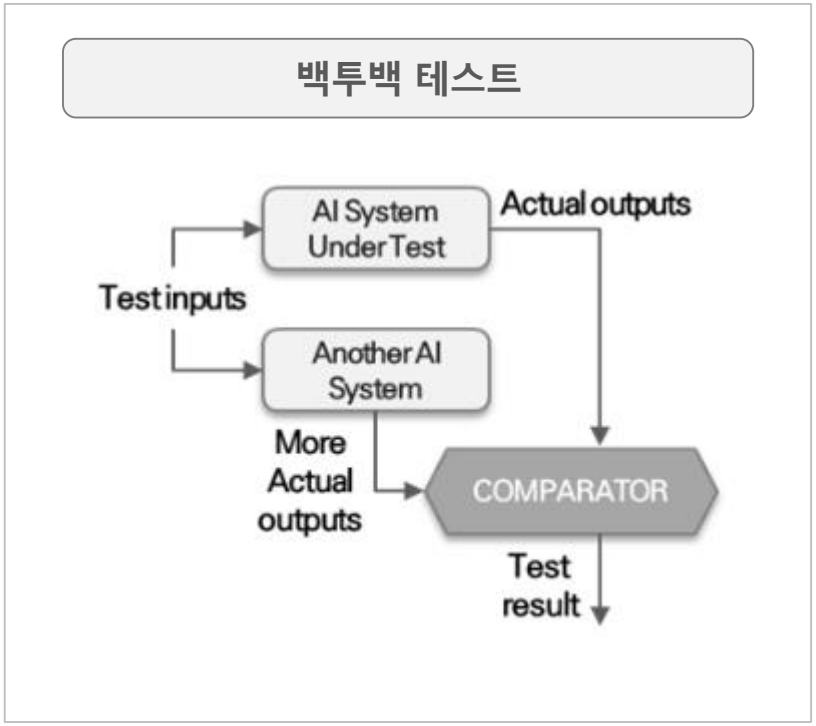
- 실루엣 계수의 전체 평균값이 1에 가까울수록 군집화가 잘 되었다고 평가
- 각 클러스터 내의 데이터 포인트들의 실루엣 계수 평균값을 구하여 1에 가까운 평균값을 가지는 클러스터는 'clear-cut' 클러스터, 0에 가까운 값을 가지는 클러스터는 'weak' 클러스터로 표현
- s(i)가 0에 가까운 경우는, 두 군집 간 거리가 거의 비슷한 경우를 의미하며 잘 구분되지 않은 상태
- s(i)가 -1에 가까운 경우는, 데이터 포인트 i가 오히려 이웃 클러스터에 더 가까운 경우를 의미

별첨 #1. 용어 이해 - A/B 테스트



- **A/B 테스트란?**
  - ✓ 두 시스템 중 더 나은 성능을 결정하도록 해주는 통계적 테스트 기법
  - ✓ **특정한 변화를 주었을 때 어떤 시스템이 목표를 더 잘 달성하는지 알아냄**
  - ✓ 기존 시스템을 부분적인 오라클로 사용하여 테스트 오라클 문제 해결
  - ✓ 기계학습 분류기에서는 정확도, 민감도, 재현율과 같은 성능 매트릭을 사용하여 시스템 측정

별첨 #1. 용어 이해 - 백투백 테스트

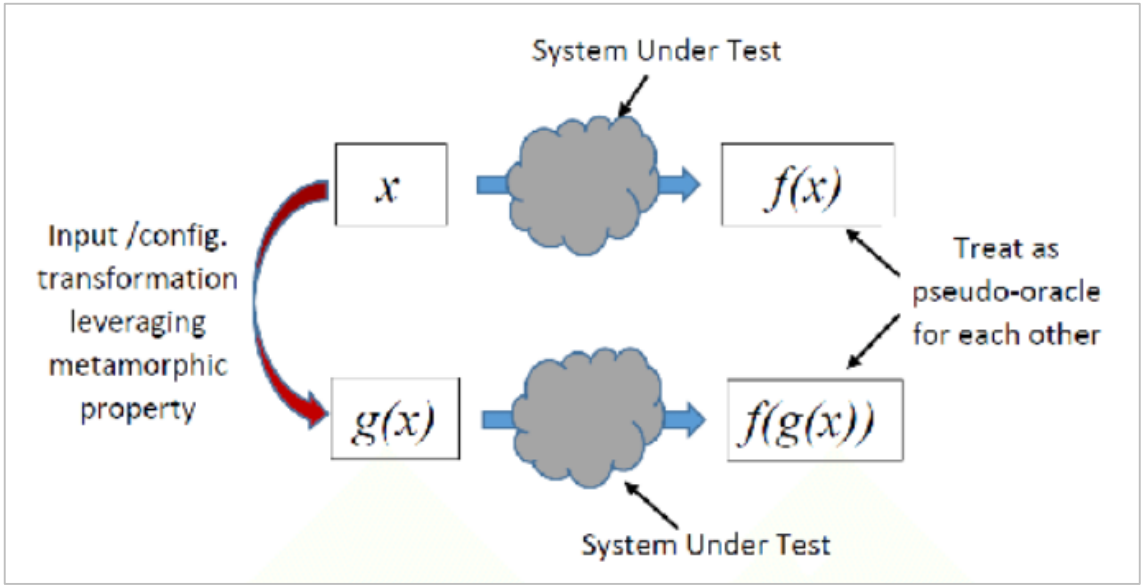


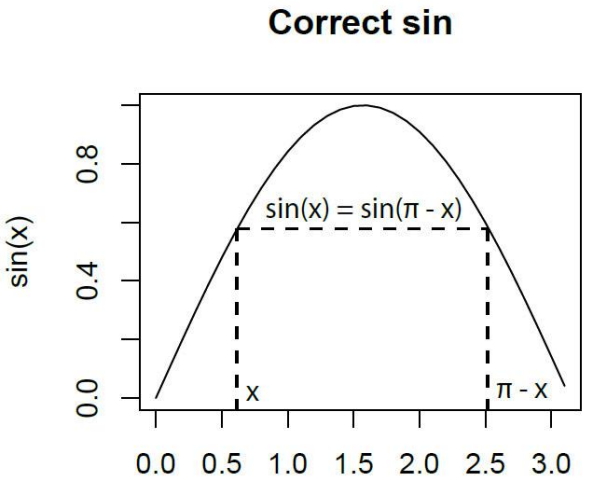
- **백투백 테스트(Back-to-Back Test)란?**
  - ✓ 이미 존재하는 **다른 버전의 시스템을 Pseudo-Oracle(가상 오라클)로 사용**하여 비교하는 테스트 기법
  - ✓ **동일한 테스트 입력값**으로 **두 가지 버전에 대해 결과 비교**
  - ✓ SW 개발 후 검증 수단으로 백투백 병행 프로세스 테스트
  - ✓ 기계학습은 Pseudo-Oracle 생성을 위해 다양한 프레임워크, 알고리즘, 설정 사용

※ 병행 테스트 – As-Is 시스템과 To-Be 시스템 비교

별첨 #1. 용어 이해 - 고차변성 테스트

- **고차변성 테스트(Metamorphic Test)란?**



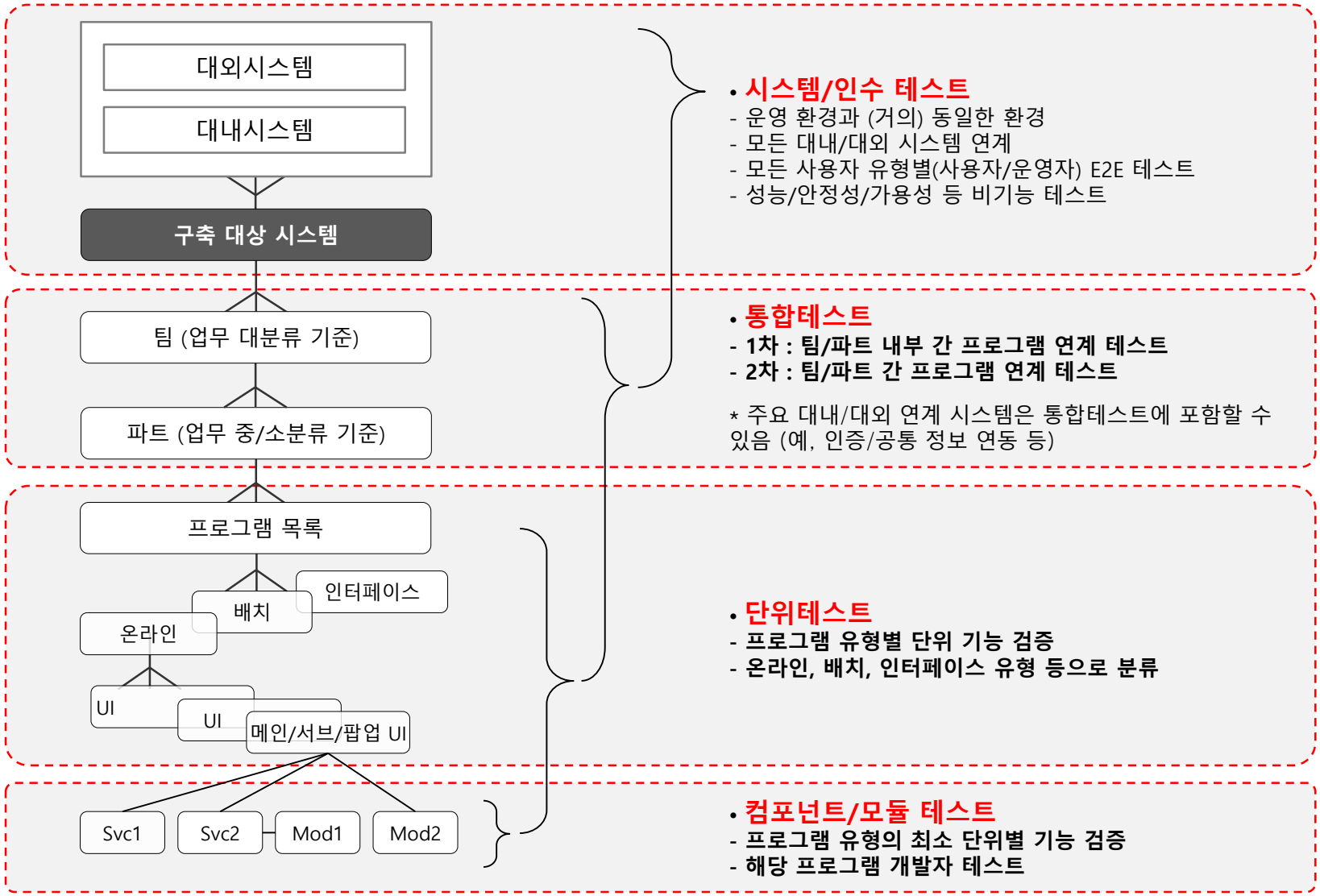


sin(x)와 sin(π - x)의 값이 같은지 확인

- ✓ AI 기반 시스템의 복잡성, 비결정성, 확률적 시스템의 이유로 **고정된 기대 결과 생성 어려움**
- ✓ 기대 결과가 고정 값이 아닌 **다른 기대 결과와의 관계에 의해 정의되는 성질을 이용한 테스트 기법**
- ✓ **정확하다고 판명된 소스 테스트 케이스**에서 **후속 테스트 케이스를 생성**하여 **변성 관계 사용**
- ✓ 후속 테스트 케이스로 바뀐 기대 결과와 원래 케이스의 결과와 비교

4. 테스트 고려사항

가. 테스트 단계 이해



4. 테스트 고려사항

가. 테스트 단계 이해_단위테스트 대상

레벨	유형	테스트 항목	테스트케이스 설계	
			Input	Output
단위 테스트	온라인 (리포트 포함)	• 이벤트 검증 (초기 오픈, 버튼, 더블클릭 등) • 업무 로직 검증 (중요 조건 값 유형별 검증) • UI 표준 및 입력 값 Validation(유효성, 필수 값, 타입 등) 검증 • 출력 결과 항목/값 검증 • 출력 메시지 검증(정상/비정상)	• 화면 정의서 • 메뉴구조도	• 단위테스트 케이스
	공통 모듈	• 입력 파라미터 값 Validation (유효성, 필수 값, 타입 등) 검증 • 업무 처리 로직 검증(중요 파라미터 유형별 검증) • 오류/예외/로깅 처리 검증 • 출력 결과 항목/값 검증	• 프로그램 명세서 (서비스)	• 단위테스트 케이스
	배치	• 입력 파라미터 값 Validation (유효성, 필수 값, 타입 등) • 오류/예외/로깅 처리 검증 • 업무 로직 검증 (중요 입력 파라미터 유형별 검증) • 출력 결과 항목/값 검증	• 프로그램 명세서 (배치)	• 단위테스트 케이스
	인터페이스	• 입력 파라미터 값 Validation (유효성, 필수 값, 타입 등) • 업무 로직 검증 (중요 전문 파라미터 유형별 검증) • 오류/예외/로깅 처리 검증 • 출력 결과 항목/값 검증	• 인터페이스 설계서	• 단위테스트 케이스

4. 테스트 고려사항

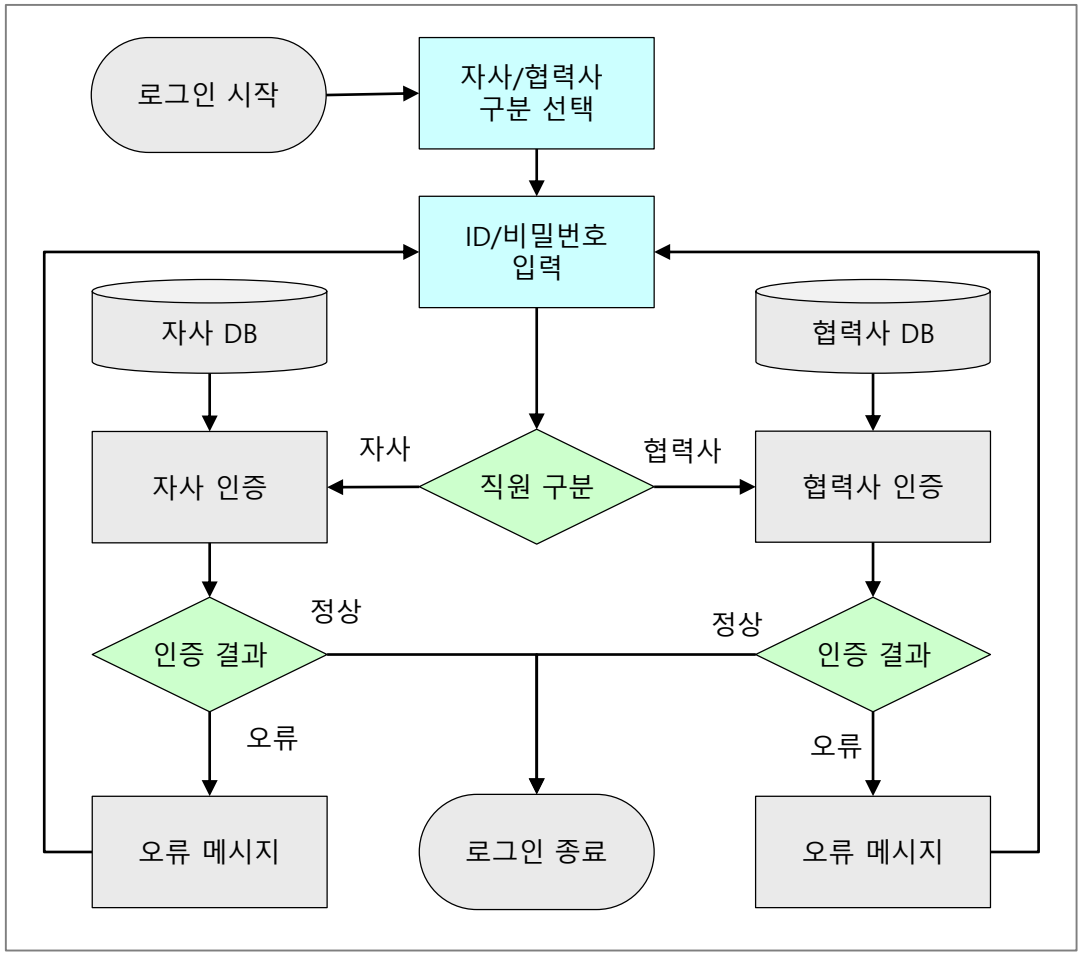
가. 테스트 단계 이해_통합테스트 대상

레벨	유형	테스트 항목	테스트케이스 설계	
			Input	Output
통합 테스트	온라인 (리포트 포함) 서비스	• 요구사항 상세내용 검증 • 전제조건, 제약사항 등 검증 • 조직/사용자 권한별 메뉴 검증 • 업무 처리 흐름 검증 - 정상/비정상/예외 처리 포함 - 연동 시스템 연계 검증	• 요구사항 정의서 • 메뉴 구조도 • 목표업무기능흐름도	통합테스트 시나리오 /케이스
	배치	• Full Data의 데이터 정합성 검증 (일간, 야간배치, 정산 등) • 배치 처리 성능 검증 • 오류/예외/로깅 처리 • 업무 로직 검증 검증 • 선후행 Batch Job Flow 검증 • 배치 스케줄링 검증	• 요구사항 정의서 • 목표업무기능흐름도 • 프로그램 설계서(배치)	통합테스트 케이스
	인터페이스	• End-To-End 연동 기능 검증 • 데이터 정합성 검증 • 오류/예외 처리 검증 • 업무 로직 검증 • 인터페이스 스케줄링 검증	• 인터페이스 설계서	통합테스트 케이스
인수테스트		• 사용자 조직 유형별 업무처리 흐름과 기능 검증 • 업무 처리 데이터 정합성 검증 • 조직/사용자 권한별 메뉴 검증	• 업무매뉴얼 (현행) • 통합 테스트 시나리오 /케이스	인수테스트 시나리오 /케이스

4. 테스트 고려사항

가. 테스트 단계 이해_단위테스트 케이스 수

단위 기능 업무 흐름도 - 로그인



[입력 파라미터 - 3개]

- ✓ 자사 / 협력사 구분
- ✓ ID
- ✓ 비밀번호

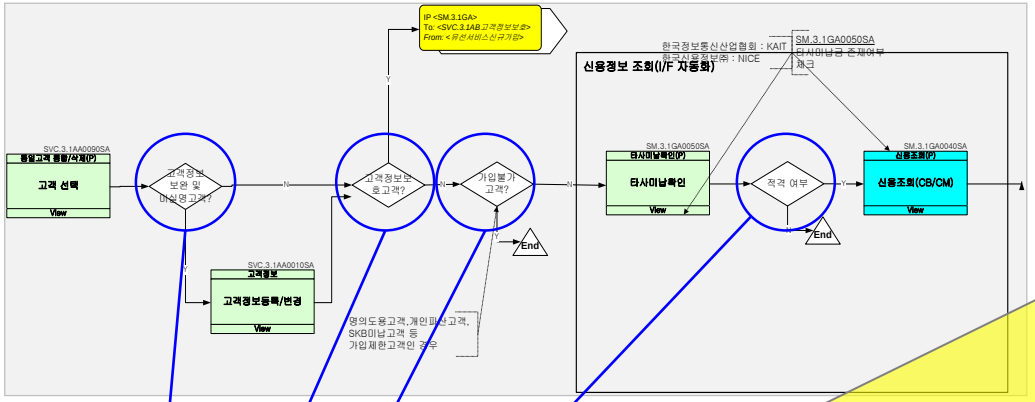
[테스트 케이스 도출]

- ✓ 자사 / 협력사 구분 : 2개
- ✓ ID : 정상, 비정상 2개
- ✓ 비밀번호 : 정상, 비정상 2개
- ✓ **2^3 = 총 8개**

4. 테스트 고려사항

가. 테스트 단계 이해_통합테스트 케이스 수

유선전화 신규가입 업무 프로세스 (E2E)



순번	Choice/Decision Box		비고
1	기존 고객 여부	Decision Box	고객정보보호로 연계되어 완료함
	Y		
	N		
2	내국인 여부	Decision Box	
	Y		
	N		
3	성인 여부	Decision Box	
	Y		
	N		완료
4	타사미납고객-적격여부	Decision Box	
	Y		

Sample

[입력 파라미터 - 31개]

- ✓ 기존 고객 여부
- ✓ 내국인 여부
- ✓ 성인여부
- ✓ 신용등급
- ✓ …

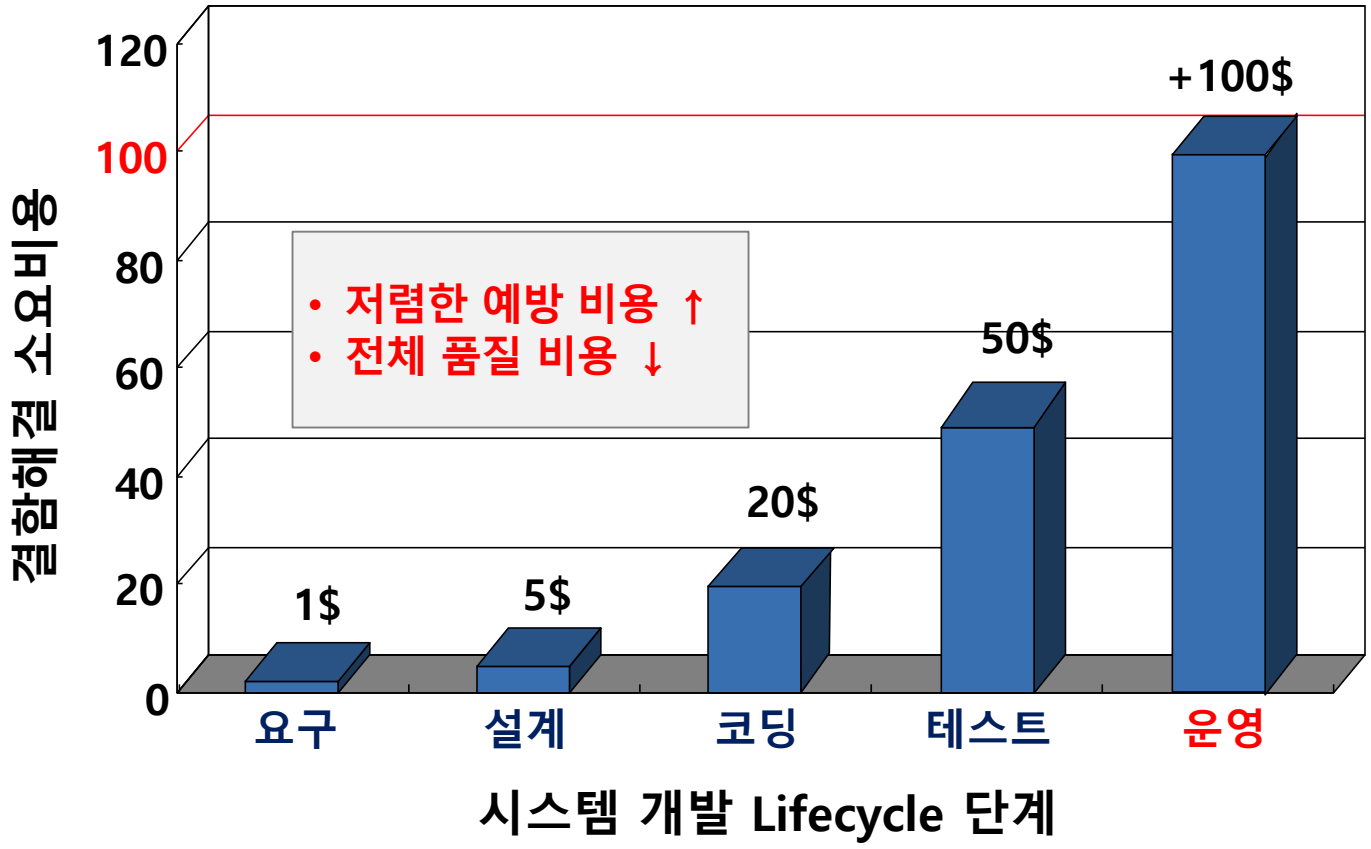
[테스트 케이스 도출]

- ✓ Choice 및 Decision Box : 총 31개
- ✓ 각 Choice 및 Decision Box의 경우의 수를 2개로 가정
- ✓ 2^{31} = 약 21억개

4. 테스트 고려사항

나. 정적 테스트_중요성

•순차적 개발 모델에서 분석단계 대비 테스트/운영 단계의 결함 해결 비용은 큰 폭으로 증가



4. 테스트 고려사항

나. 정적 테스트_리뷰 vs 정적 분석

리뷰의 특징 (수작업)

- ✓ 조기 결함발견 및 수정 가능
- ✓ 개발 생산성 향상 및 개발 기간 단축
- ✓ 테스팅 비용 감소 및 시간 단축
- ✓ 개발 수명주기 전체에 걸친 비용 감소
- ✓ 결함 감소로 품질 향상
- ✓ 초기부터 커뮤니케이션 향상

주요 도출 결함

- ✓ 문서/코딩 표준 위반
- ✓ **요구사항 결함 (불명확, 불완전 등)**
- ✓ **설계 결함 (요구사항의 누락, 부정확한 설계 등)**
- ✓ 기능/비기능 오류 검출

정적 분석의 특징 (도구)

- ✓ 동적 테스팅으로 찾기 힘든 결함을 발견
- ✓ 리뷰와 마찬가지로 정적 분석은 장애 보다는 결함을 발견함
- ✓ 소스코드의 코딩 표준, 잠재적인 오류 등
- ✓ Code Inspection 도구를 활용하여 수행함
- ✓ 정적 분석 도구는 프로그램 코드 (C, JAVA, HTML, XML, Python 등) 분석함

주요 도출 결함

- ✓ 정의 되지 않거나 사용하지 않는 변수
- ✓ **코딩 표준 위반, 코드 복잡성**
- ✓ **보안 취약성**
- ✓ 모듈과 컴포넌트 간에 일관되지 않는 I/F

4. 테스트 고려사항

나. 정적 테스트_좋은 명세서

[요구사항 명세 원칙]

원칙	설명
검증성	요구 명세는 테스트/검증 가능해야 함
명확성	요구 명세는 이해당사자별 명확히 제시
정확성	요구 명세는 정확히 기술
추적성	요구사항은 출처, 제가자 등 근거가 명확해야 함
일관성	요구 명세는 요구사항 간, 이행당사자 간에 충돌이 없어야 함
완전성	기능성, 성능 및 제약사항 등 모든 중요한 내용을 문서화
해석성	요구사항 해석의 보편적 일관성 제공

✓ 요구사항 명세서 **정적 테스트** 시 명세 원칙 참조

[좋은 요구사항 명세서가 가지고 있는 특징]

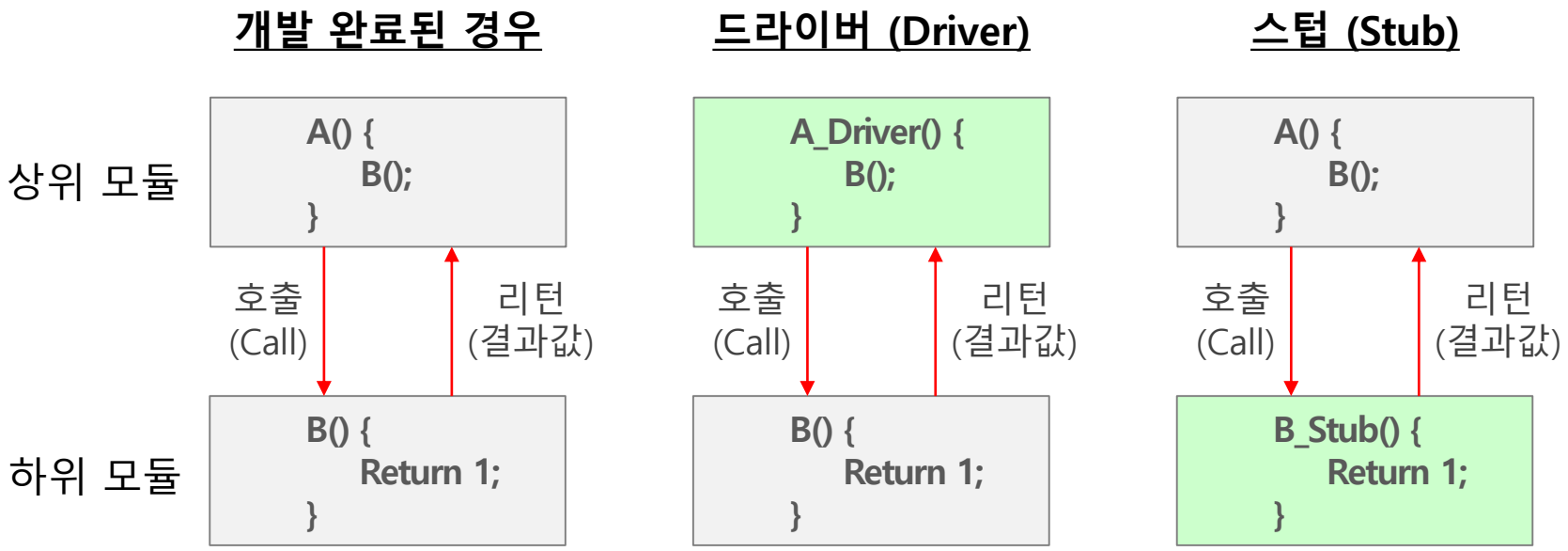
- 요구사항 명세서를 읽는 작업자(개발자, 디자이너 등)가 **이해하기 쉬워야** 한다.
- 무엇을 어떻게 구현되어야 하는지 **명확하게 작성**한다.
- 하나의 요구사항에 여러 가지(복수) 요구사항을 작성하지 않는다.
- 다른 요구사항 **모순 또는 중복되지 않게** 한다.
- 애매한 단어를 사용하지 않고 명확하게 기재한다. (~ 있으면 좋겠다 → ~ 기능 필요) 오해하지 말아야 할 것이 명령하는 것이 아니고 의견을 모호하게 하지 말고 명확하게 표현하라는 의미이다.
- 꼭 필요하고 중요한 요구사항은 구분할 수 있도록 요구사항의 중요도 또는 우선 순위를 관리한다.
- **동일한 용어**를 사용한다. 댓글, 코멘트, 덧글과 같이 모두 같은 의미를 다르게 표현하지 않는다.
- 난이도가 있는 기능이거나 프로젝트 기간이 짧아 모든 요구사항을 구현하기 어려울 상황일 경우, 대체 가능한 다른 방법도 함께 기술하여 전달하는 것이 좋다.

4. 테스트 고려사항

다. 테스트 스텁과 드라이버

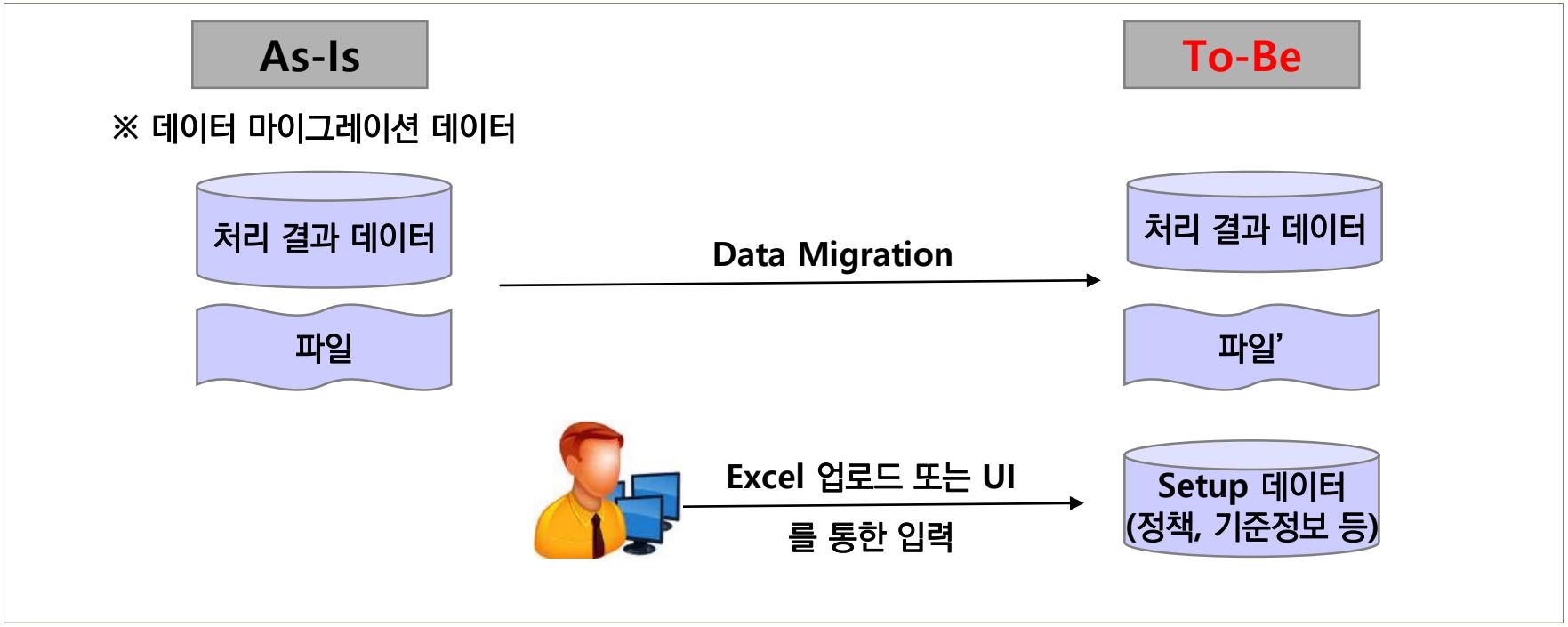
테스트 드라이버와 스텁

- ✓ **테스트 드라이버** : 하위 모듈을 테스트하기 위해 이 모듈을 호출해주는 상위의 임시 모듈
- ✓ **테스트 스텁** : 상위 모듈을 테스트하기 위해 이 모듈에서 호출하는 하위의 임시 모듈



4. 테스트 고려사항
라. 테스트 데이터

➢ 테스트 데이터



✓ 데이터 마이그레이션과 셋업 데이터(정책성 데이터, 기준정보 등)에 대한 주체를 명확히 해서 테스트 전 업로드 완료

4. 테스트 고려사항
라. 테스트 데이터_데이터 준비 샘플 #1

No.	서비스구분	오더유형	오더조건	케이스명	TEST 주체	수행 주관	설명	사전준비 사항	준비 주체	전번	오더 번호	SCN	주소	Wrap-up 내용	비고
1	일반전화	신규 설변 전입	당일 신규 3건 설변 3건 전입 3건 3일 이후 신규/설변/전입 3건씩	개통전변경_예약변경(일반전화)	PnA WM_PC WM_모바일	N팀	1. 당일건을 3일 이내로 예약 변경시 오더 상태 및 단계확인 2. 당일건을 3일 이후로 예약 변경시 오더 상태 및 단계확인 3. 3일 이후건을 당일로 예약 변경 4. 예약변경시 해당 날짜에 가용성이 없을 경우 강제입력 가능여부 5. 강제 입력시 예약가용성 관리의 기준건수가 맞게 차감 되었는지 확인								
2	SOIP	신규	센트릭스 5건(대표번호지정) 당일 5건, 3일 이후5건	개통전변경_예약변경(센트릭스)	WM_PC	N팀	당일건을 3일 이후로 예약변경 3일 이후건을 당일로 예약변경								
3	TV	신규	멀티룸 10건	개통전변경_예약변경(동시성_다량	PnA WM_PC WM_모바일	N팀	1. 동시번호가 같은 다량의 오더를 멀티선택하여 한번에 예약변경 2. 개통완료 후 ICIS에서 룸호실 변경시 상태확인								
4	일반전화+ 인터넷 +TV+VOIP	신규	동시오더 3건	개통전변경_예약변경(동시성_복합	PnA WM_PC WM_모바일	N팀	서비스는 다르나 동시번호가 같은 동시성오더를 멀티선택하여 한번에 예약변경이 가능한지 확인								
5	일반전화+ 인터넷 +TV+VOIP	설변	동시건 신규개통 완료 후 해당번호를 설변	개통전변경_예약변경(동시성_단건	PnA WM_PC WM_모바일	N팀	1. 동시번호가 존재하는 동시성오더중 일부만 예약변경시 동시성 상태 확인 2. 동시번호가 존재하는 동시성오더를 단건으로 하나씩 같은 시간으로 모두 예약 변경시 동시성 유지 확인								

Sample

✓ **예약변경 테스트를 위한 기준 오더번호 준비**

- 상품 유형별(일반전화, SoIP, TV, 인터넷 전화, 복합상품 등)
- 기존 데이터 조회 또는 신규생성 필요

✓ **개통을 위한 고객번호 및 주소 필요**

4. 테스트 고려사항
라. 테스트 데이터_데이터 준비 샘플 #2

구분	데이터 (준비사항)	준비주체	비고
1: 전제조건	테스트 단말기(안드로이드 기반)	수행사/고객사	기회원의 디바이스 변경 테스트
1: 전제조건	테스트 단말기(iOS 기반)	수행사/고객사	기회원의 디바이스 변경 테스트
1: 전제조건	기 회원의 명의로 된 다른 단말기 추가 필요	수행사/고객사	기회원의 디바이스 변경 테스트
2: 회원상태	(중앙회) 비과세 대상 회원	고객사	중앙회 비과세 가능한도 조회 용
2: 회원상태	(중앙회) 사업자정보 : 휴/폐업 사업자상태	고객사	
2: 회원상태	(중앙회) 우수회원	고객사	금리인하 요구 건
2: 회원상태	내부유의고객 회원	(수행사) 테스트 시 데이터 설정	
2: 회원상태	다계좌 개설 이력이 있는 회원	(수행사) 테스트 시 데이터 설정	테스트 시 계좌개설로 테스트 가능
2: 회원상태	신청인 + 담보제공자	고객사	*배우자 여부 체크
2: 회원상태	대주주와 동명이인이며 생년월일이 같은 회원(회원상태)	고객사	
2: 회원상태	(중앙회) 만65세 이상 회원	고객사	
2: 회원상태	외국인, 미성년자, 휴대폰 미보유	고객사	본인인증 시 테스트 용
2: 회원상태	(신규) 비고객	고객사	
2: 회원상태	(중앙회) 기고객 (입출금 보유/미보유, 전자금융 신청/미신청)	고객사	
2-1: 인증 데이터	공동인증서	고객사	테스트로 활용될 대출 신청인의 인증정보
2-1: 인증 데이터	금융인증서	고객사	테스트로 활용될 대출 신청인의 인증정보
2-1: 인증 데이터	KAKAO 사설인증서	고객사	테스트로 활용될 대출 신청인의 인증정보
2-1: 인증 데이터	PASS 사설인증서	고객사	테스트로 활용될 대출 신청인의 인증정보
2-1: 인증 데이터	본인확인용 신분증	고객사	테스트로 활용될 대출 신청인의 인증정보
2-1: 인증 데이터	OTP	고객사	테스트로 활용될 대출 신청인의 인증정보
2-1: 인증 데이터	휴대폰본인인증	고객사	테스트로 활용될 대출 신청인의 인증정보
2-1: 인증 데이터	공동인증서(PC, 모바일)	고객사	테스트로 활용될 대출 신청인의 인증정보

✓ **검증하고자 하는 회원 상태별 테스트 데이터**

- 회원 상태 (비과세, 휴/폐업 사업자, 우수회원, 유의고객, 미성년자....)

✓ **각종 인증을 위한 회원별 인증서, OTP 등 사전 필요**

- 연동시스템 (카카오, KT, 금결원 등)과 테스트 데이터에 대해서 사전에 협의완료되어야 함

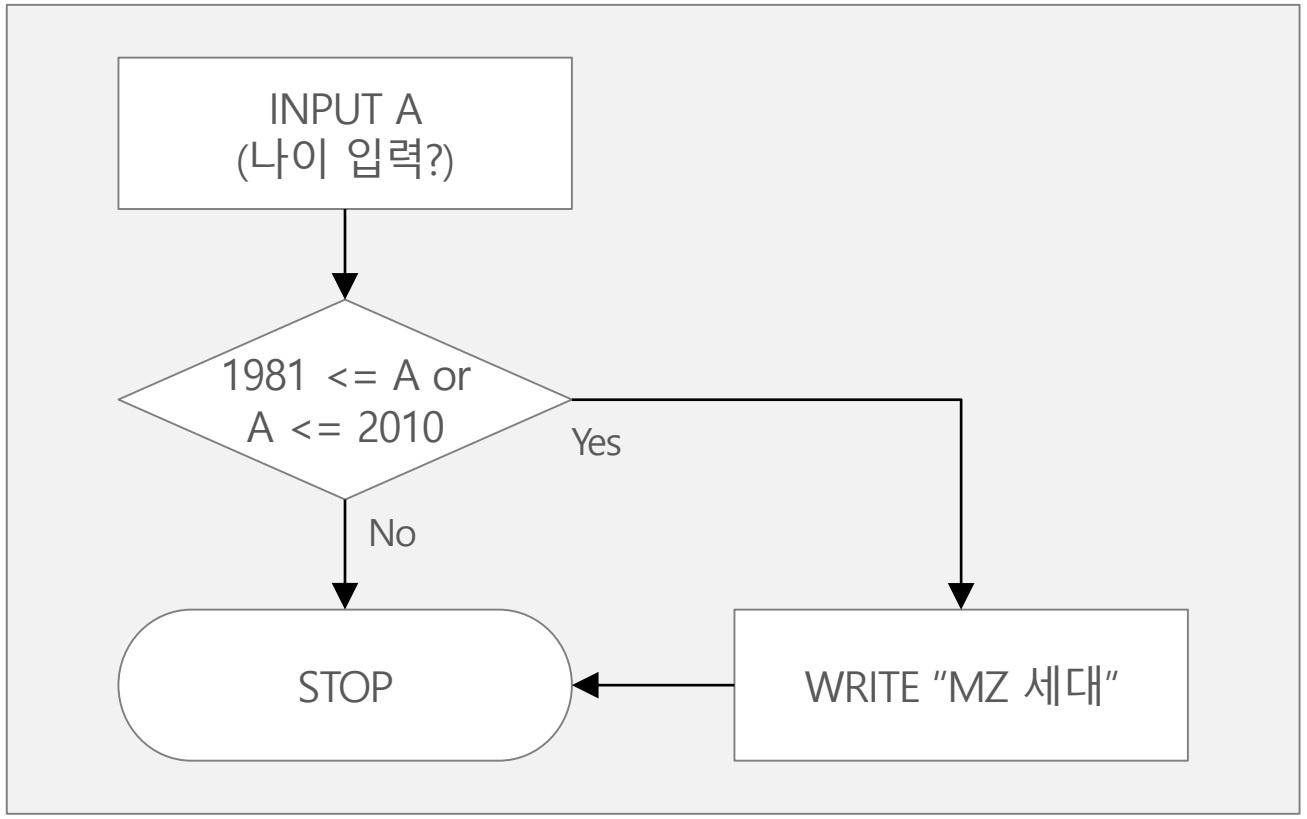
4. 테스트 고려사항
마. 테스트 설계

명세기반 기법	구조기반 기법	경험기반 기법
• 산출물/문서를 기반으로 테스트 케이스 도출 • 공식적/비공식적 모델을 기반으로 테스트 케이스 도출	• SW 설계나 소스 코드 등 구조적인 정보기반 테스트 케이스 도출 • 체계적인 테스트 케이스 도출 및 측정관리 가능	• 테스터, 개발자, 사용자 등의 경험적 지식 활용 • 유사 SW나 기술 경험을 바탕으로 직감적으로 테스트
• 동등 분할 • 경계값 분석 • 조합 테스팅 (Pairwise) • 결정 테이블 기법 • 상태전이 기법 • 유즈케이스 기법	• 구문 커버리지 • 결정 커버리지 • 조건 커버리지 • 변경/조건 커버리지 • 경로 커버리지 • 제어흐름 테스트 • 기본 경로 테스팅	• 에러 추정 • 체크리스트 • 탐색적 테스팅 • 분류 트리 기법

5.1 테스트 설계 개요

실습 #1. 테스트 설계해 보기 – 단위 테스트 설계

아래 MZ 세대를 구별하는 모듈(Flow Chart)에서 필요한 테스트 케이스 수는?



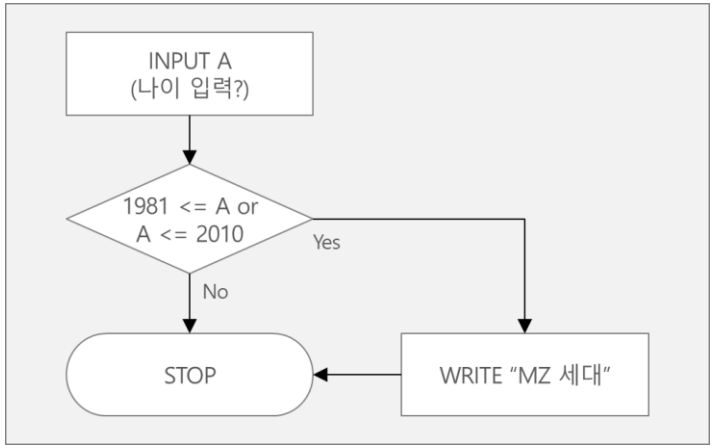
A = OO

1번
2번
3번
…
??

5.1 테스트 설계 개요

실습 #1. 테스트 설계해 보기 – 단위 테스트 설계

아래 MZ 세대를 구별하는 모듈(Flow Chart)에서 필요한 테스트 케이스 수는?



테스트 케이스 수는?

A = OO

입력 값 고려사항

- **입력값 변수 타입**
  - 정수, 실수, 문자
- **입력 값의 길이**
- **테스트 기법**
  - 등가분할
  - 경계값 분석

종류	타입	메모리 사용 크기		저장되는 값의 범위
정수형	byte	1 byte	8 bit	$-2^7 \sim (2^7-1)$
	short	2byte	16 bit	$-2^{15} \sim (2^{15}-1)$ $(-32,768 \sim 32,767)$
	int	4 byte	32 bit	$-2^{31} \sim (2^{31}-1)$ $(2,147,483,648 \sim 2,147,483,647)$
	long	8 byte	64 bit	$-2^{63} \sim (2^{63}-1)$
실수형	float	4 byte	32 bit	$(+/-)1.4E-45$ $(+/-)3.4028235E38$
	double	8 byte	64 bit	$(+/-)4.9E-324 \sim$ $(+/-)1.7976931348623157E308$
문자형	char	2byte	16 bit	$0 \sim (2^{16}-1)$ 유니코드 : \u0000~\uFFFF, 0~65535
논리형	boolean	1 byte	8 bit	TRUE, FALSE

5.1 테스트 설계 개요

가. 테스트 설계란?



- 일반적이고 추상적인 테스트의 목적을 구체적인 상황과 테스트 케이스로 변환하는 단계
- 테스트 베이시스(산출물)와 테스트 기법을 활용하여 테스트 케이스를 만드는 작업

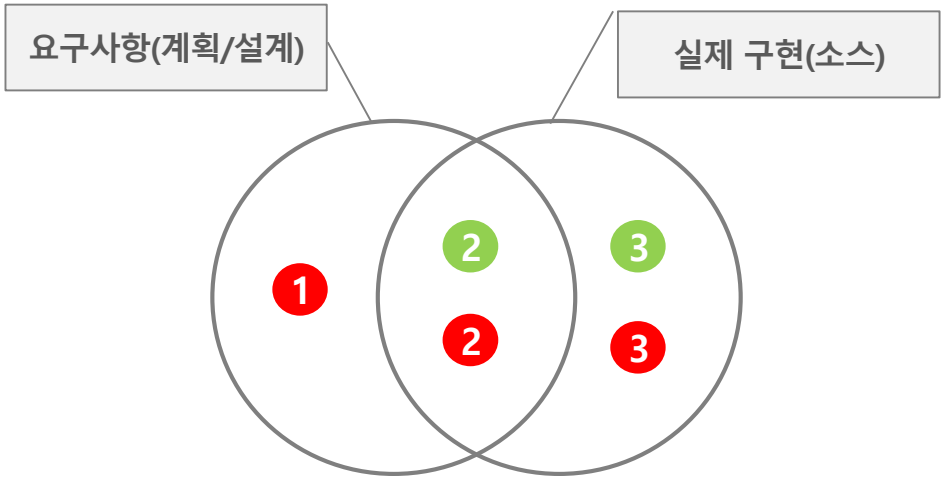
- 다양한 품질 특성(기능성, 사용성, 호환성, 안정성 등)을 확인하기 위해 입력 값, 실행 조건, 실행 절차, 예상 결과의 집합

- **테스트 유형별/단계별 테스트 설계 필요**

- 기능 테스트 : 단위 테스트, 통합 테스트, 인수 테스트
- 성능 테스트 : 단위성능 테스트, 통합성능 테스트, 시스템성능 테스트

5.1 테스트 설계 개요

나. 테스트 설계의 범위



- 정상 작동함
- 정상 작동하지 않음

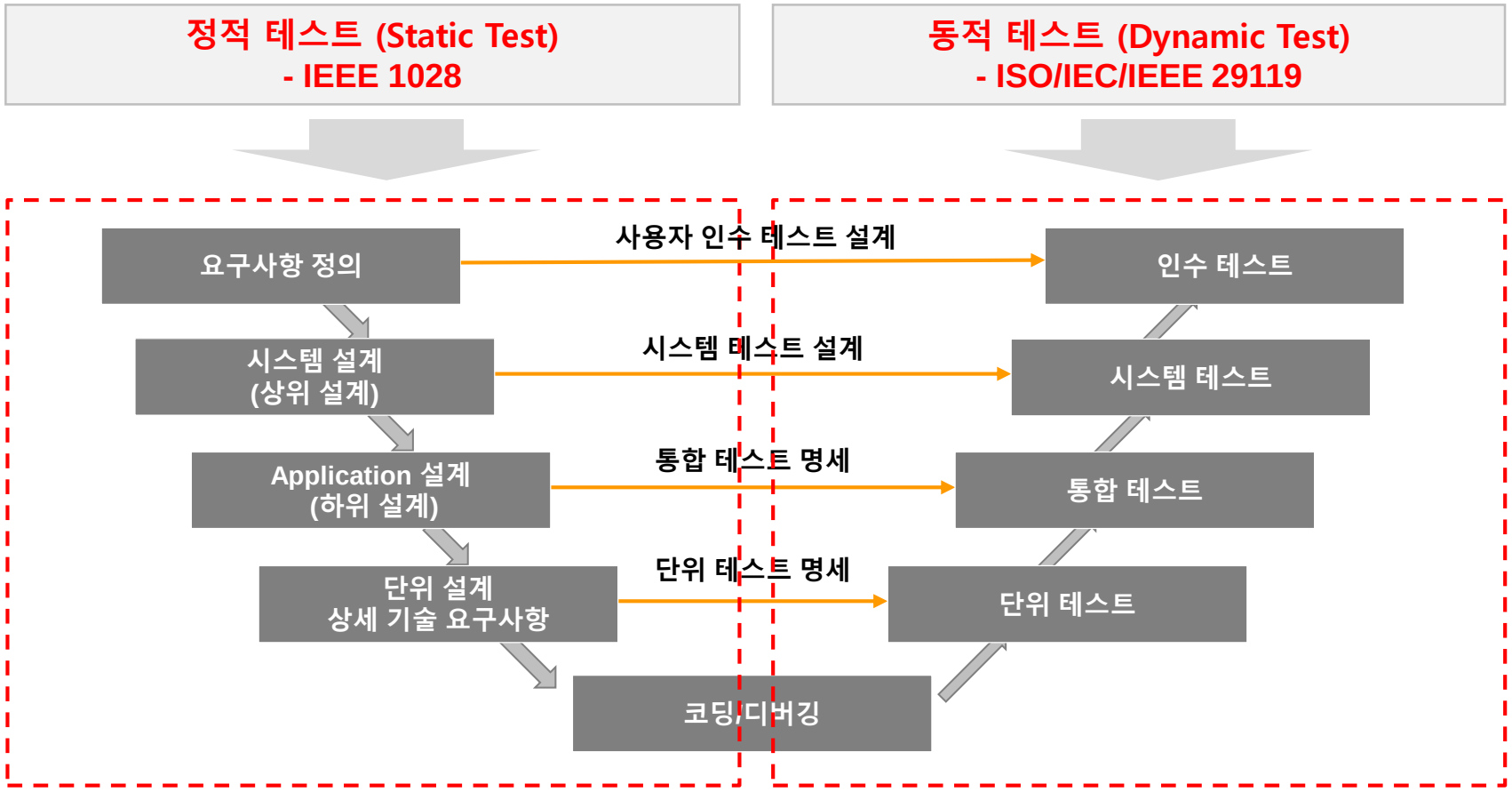
1. 요구사항에 명시되어 있는데 구현되지 않은 경우
   - ➔ 요구사항 기반으로 테스트 케이스를 만들어 실행하여 누락여부 식별
2. 요구사항대로 구현되어 있는 기능
2. 요구사항대로 구현되어 있지 않은 기능
   - ➔요구사항 기반으로 테스트케이스를 만들어 실행하여 정상여부 검증
3. 요구사항에 없는데 구현되어 있고, 기능이 제대로 작동하는 경우
   - ➔ 정상적으로 작동하더라도 결함일수 있음
3. 요구사항에 없는데, 구현되어 있지만 기능이 작동하지 않는 경우
   - ➔ 암묵적인 요구사항인 경우 정상
   - ➔ 경험기반(과거 사례)으로 케이스 또는 체크리스트 작성하여 검증

※ 1, 2번은 명세기반, 3번은 경험기반으로 테스트 케이스 설계

5.1 테스트 설계 개요

다. 테스트 설계 베이시스_V-모델

V-Model : 소프트웨어 개발 프로세스로 폭포수 모델(순차적 개발 모델)의 확장된 형태



- V 모델은 소프트웨어 개발의 각 단계마다 상세한 문서화를 통해 작업을 진행하는 방법 사용
- 테스트 활동을 코딩 이후가 아닌 프로젝트 시작 시에 함께 테스트 설계 시작

5.1 테스트 설계 개요

라. 테스트 설계 베이시스 (1/2)

단계	세부 활동	구조	객체	정보	패키지	산출물
I. 분석	1. 패키지 설치 및 교육				O	패키지 설치계획서/교육계획서/결과서
	2. 현행 업무 분석	O	O	O	O	시스템/패키지 분석서, 현행 기능분해도/흐름도
	3. 요구사항 도출 및 정의	O	O	O	O	요구사항 정의서
	4. Use Case 모델 정의		O			Use Case 명세서
	5. 프로세스 모델링			O		목표 업무 기능 분해도/흐름도
	6. 목표 업무 분석	O	O		O	목표 업무 기능 분해도/흐름도
	7. 패키지 Gap 분석				O	패키지 Gap 분석서
	8. 데이터베이스 분석		O			논리 ERD, 단어/용어/도메인/주제영역 정의서
	9. 데이터 모델링			O		엔터티 정의서, 관계 정의서, 논리 ERD
	10. 프로세스/데이터 연관 분석			O		프로세스/엔티티 매트릭스
	11. 개발표준 수립	O	O	O	O	개발표준 정의서
	12. 마이그레이션 분석	O		O	O	마이그레이션 계획서
	13. 마스터 테스트 계획 수립	O	O	O	O	마스터 테스트 계획서
	14. 변경영향도 분석 및 심의	O	O	O	O	요구사항 변경 요청서
	15. 요구사항 추적 매트릭스	O	O	O	O	요구사항 추적 매트릭스

통합/시스템/인수 TC 설계 Basis

단위테스트 TC 설계 Basis

5.1 테스트 설계 개요

라. 테스트 설계 베이시스 (2/2)

단계	세부 활동	구조	객체	정보	패키지	산출물
II. 설계	1. 아키텍처 설계	O	O	O	O	아키텍처 정의서
	2. 업무 설계		O			설계 클래스 정의서, 설계 시퀀스 다이어그램
	3. 프로시저 설계			O		프로시저 정의서/명세서, 프로세스/프로시저 매트릭스
	4. 화면 설계	O	O	O	O	메뉴 구조도, UI 설계서
	5. 데이터베이스 설계	O	O	O	O	물리ERD, 코드/테이블 정의서, 데이터베이스명세서
	6. 인터페이스 설계	O	O	O	O	인터페이스 설계서
	7. 패키지 구성 설계				O	패키지 Configuration 정의서
	8. 프로그램 설계	O	O	O	O	프로그램 목록, 프로그램 설계서
	9. 마이그레이션 설계	O	O	O	O	테이블/컬럼/코드 매핑 정의서
III. 구현	1. 프로그램 개발	O	O	O	O	소스 코드, 구현된 데이터베이스
	2. 단위 테스트	O	O	O	O	UT 계획서/명세서/결과서, UT Readiness Checklist
	3. 업무전환 전략	O	O	O	O	업무전환 전략서
IV. 테스트	1. 통합 테스트	O	O	O	O	AIT 계획서/명세서/결과서, AIT Readiness Checklist
	2. 시스템 테스트	O	O	O	O	SIT 계획서/명세서/결과서, SIT Readiness Checklist
	3. 인수 테스트	O	O	O	O	UAT 계획서/명세서/결과서, UAT Readiness Checklist
	4. 업무전환 계획	O	O	O	O	업무전환 계획서
V. 이행	1. 업무전환 준비	O	O	O	O	업무전환 시나리오/체크리스트
	2. 업무전환 실행	O	O	O	O	설치 확인서, 사용자/운영자 매뉴얼

통합/시스템/인수 TC 설계 Basis

단위테스트 TC 설계 Basis

5.1 테스트 설계 개요

마. 설계 기법 유형

명세기반 기법	구조기반 기법	경험기반 기법
• 산출물/문서를 기반으로 테스트 케이스 도출 • 공식적/비공식적 모델을 기반으로 테스트 케이스 도출	• SW 설계나 소스 코드 등 구조적인 정보기반 테스트 케이스 도출 • 체계적인 테스트 케이스 도출 및 측정관리 가능	• 테스터, 개발자, 사용자 등의 경험적 지식 활용 • 유사 SW나 기술 경험을 바탕으로 직감적으로 테스트
• 동등 분할 • 경계값 분석 • 조합 테스팅 (Pairwise) • 결정 테이블 기법 • 상태전이 기법 • 유즈케이스 기법	• 구문 커버리지 • 결정 커버리지 • 조건 커버리지 • 변경/조건 커버리지 • 경로 커버리지 • 제어흐름 테스트 • 기본 경로 테스팅	• 에러 추정 • 체크리스트 • 탐색적 테스팅 • 분류 트리 기법

5.2 명세기반 기법

가. 동등 분할 기법

동등분할 기법

- 동일한 결과값을 나타내는 입력 값들의 범위를 그룹핑하여 각 그룹의 대표 값을 선택하여 테스트 케이스화하는 기법
- 유효한 입력 값 뿐만 아니라 유효하지 않은 입력 값도 그룹핑할 수 있음
- 모든 테스트 단계 및 유형에서 적용할 수 있음

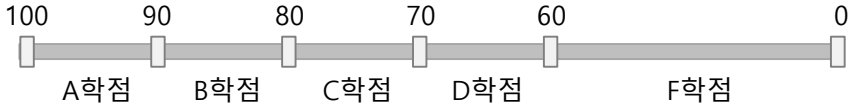
[시험결과 학점 절대 평가]



시험 점수	학점
90 ~ 100	A학점
80 ~ 89	B학점
70 ~ 79	C학점
60 ~ 69	D학점
59이하	F학점

케이스 도출

[테스트 케이스 도출]



테스트 케이스	입력 값	예상 결과
TC_A학점 케이스	91	A학점
TC_B학점 케이스	84	B학점
TC_C학점 케이스	77	C학점
TC_D학점 케이스	63	D학점
TC_F학점 케이스	55	F학점

5.2 명세기반 기법

나. 경계값 분석 기법

경계값 분석 기법

- 동등 분할의 경계 부분 입력값에서 결함이 발견되는 확률이 경험적으로 높기 때문에 경계값을 포함하여 테스트하는 기법임
- 해당 분할영역의 최댓값과 최솟값은 그 영역의 경계값이 됨
- 모든 테스트 단계 및 유형에서 적용할 수 있음

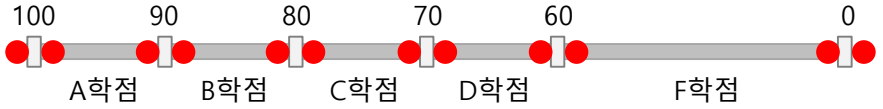
[시험결과 학점 절대 평가]



시험 점수	학점
90 ~ 100	A학점
80 ~ 89	B학점
70 ~ 79	C학점
60 ~ 69	D학점
59이하	F학점

케이스 도출

[테스트 케이스 도출]



테스트 케이스	입력 값	예상 결과
TC_A학점 케이스	100, 99, 91, 90	A학점
TC_B학점 케이스	89, 81, 80	B학점
TC_C학점 케이스	79, 71, 70	C학점
TC_D학점 케이스	69, 61, 60	D학점
TC_F학점 케이스	59, 1, 0	F학점
TC_비정상 케이스	101, -1	오류 메시지

5.2 명세기반 기법

다. 결정 테이블 기법

결정 테이블 기법

- 요구사항의 논리적인 조건이나 상황을 비즈니스 규칙으로 정의하고, 가능한 모든 조건들의 조합을 생성
- 요구사항 등 테스트 베이시스의 문제점을 효과적으로 발견할 수 있음
- 복잡하고 다양한 조건일 경우 많은 시간과 노력이 소요될 수 있음

상품결합 할인



- 3가지 상품결합 30% 할인
- 2가지 상품결합 20% 할인
- 65세 이상 추가 10% 할인

케이스 도출

[상품결합 할인율 검증 테스트 케이스 도출]

테스트 케이스	TC 1	TC 2	TC 3	TC 4	TC 5	TC 6
단일 상품	1	1	0	0	0	0
2가지 상품결합	0	0	1	1	0	0
3가지 상품결합	0	0	0	0	1	1
65세 이상	1	0	1	0	1	0

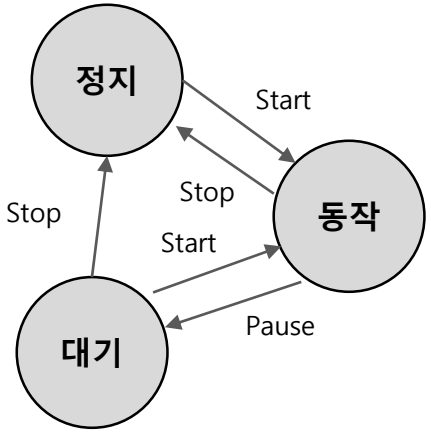
테스트 케이스	TC 1	TC 2	TC 3	TC 4	TC 5	TC 6
상품 결합할인	단품	단품	2결합	2결합	3결합	3결합
노년 할인 (나이)	65세	30세	70세	54세	66세	64세
예상 결과(할인율)	**10%**	**0%**	**30%**	**20%**	**40%**	**30%**

5.2 명세기반 기법

라. 상태 전이 기법

상태 전이 기법

- 시스템이 가질 수 있는 상태가 어떤 것이 있는지 또 각 상태별로 수신된 이벤트(메시지)에 대해서 어떤 행동을 보일지를 기술하는 기법
- 상태 사이의 관계 즉, 상태 간의 변화(전이)를 일으키는 이벤트와 입력값과 그 결과, 동작 등 검증 (요구사항과 맞지 않는 전이를 찾아냄)



[MP Player 상태흐름도]

케이스 도출

[단말기 동작상태 테스트 케이스 도출]

케이스 번호	이전 상태	이벤트	예상결과 / 변이 상태
TC_정지_01	정지	Start	동작
TC_동작_01	동작	Stop	정지
TC_동작_02	동작	Pause	대기
TC_대기_01	대기	Start	동작
TC_대기_02	대기	Stop	정지

5.2 명세기반 기법

마. 조합 테스팅 (Pairwise) 기법 (1/3)

조합 테스팅 정의

- 블랙박스 테스트 설계 기법으로 모든 입력 파라미터에 대하여 **각 쌍의 개별 조합**을 실행하기 위한 테스트 설계기법 (ISTQB 표준용어 v2.0)

조합 테스팅 필요성

- 모든 입력 파라미터들의 모든 테스트 케이스 불가
- 모든 입력 파라미터들의 **각 쌍에 대하여 최소한 한번씩의 조합**

5.2 명세기반 기법

마. 조합테스팅 (Pairwise) 기법 (2/3)

모든 경우의 수 조합

[입력 파라미터]

- 모드 : Play, Stop
- 스피커 : On, Off
- 볼륨 : High, Low

[모든 조합]

순번	모드	스피커	볼륨
1	Play	On	High
2	Play	On	Low
3	Play	Off	High
4	Play	Off	Low
5	Stop	On	High
6	Stop	On	Low
7	Stop	Off	High
8	Stop	Off	Low

조합(Pairwise) 테스팅 기법

[각쌍의 조합]

순번	모드	스피커
1	Play	On
2	Play	Off
3	Stop	On
4	Stop	Off

순번	모드	볼륨
1	Play	High
2	Play	Low
3	Stop	High
4	Stop	Low

순번	스피커	볼륨
1	On	High
2	On	Low
3	Off	High
4	Off	Low

[Pairwise 조합]

순번	모드	스피커	볼륨
1	Play	On	High
2	Play	Off	Low
3	Stop	On	Low
4	Stop	Off	High

각 쌍에 대하여 최소한 한번씩의 조합생성

5.2 명세기반 기법

마. 조합테스팅 (Pairwise) 기법 (3/3)

No.	조건 1	조건 2	조건 3	조건 4	조건 5	모든 조합	Pairwise 기법	감소량(%)
1	2	2	2			8	**4**	**50.0%**
2	3	2	2			12	**6**	**50.0%**
3	3	3	2			18	**9**	**50.0%**
4	3	3	3			27	**9**	**66.7%**
5	3	3	3	2		54	**9**	**83.3%**
5	3	3	3	2	2	108	**12**	**88.9%**
5	4	3	3			36	**12**	**66.7%**
7	4	3	3	2		72	**12**	**83.3%**
8	4	3	3	2	2	144	**12**	**91.7%**
…								

조건의 항목이 많을 수록 Pairwise 기법을 활용하면
테스트 케이스의 수를 급격하게 줄일 수 있다.

5.3 구조기반 기법

가. 코드 커버리지 유형

유형	설명
① 구문 커버리지 Statement Coverage	- 테스트 스위트에 의해 실행된 구문이 몇 퍼센트인지 측정하는 것 - 다른 커버리지에 비해 가장 약함
② 결정 커버리지 Decision Coverage or Branch Coverage	- 테스트 스위트에 의해 실행된 결정 포인트 내의 전체 조건식이 최소한 참(True)이 한번, 그리고 거짓(False)이 한 번의 값을 갖는지 측정하여 퍼센트로 표현하는 것 - 개별 조건 식의 개수와 상관없이 테스트 케이스의 최소 개수는 2 개 - 조건, 조건/결정 커버리지에 비해 약함
③ 조건 커버리지 Condition Coverage	- 전체 조건 식의 결과와 관계없이 각 개별 조건식이 참 한번, 거짓 한번을 모두 갖도록 개별 조건식을 조합하는 것 - 결정 커버리지 보다 강력한 형태의 커버리지
④ 조건/결정 커버리지 Condition/Decision	- 전체 조건 식의 결과가 참 한번, 거짓 한번을 갖도록 각 개별 조건 식을 조합하는데, 이때 각 개별 조건식도 참 한번, 거짓 한번을 모두 갖도록 개별 조건 식을 조합하는 것으로 결정 커버리지와 조건 커버리지를 포함하는 커버리지
⑤ 변경조건/결정 커버리지 Modify Condition / Decision Coverage	- MC/DC는 각 개별 조건 식이 다른 개별 조건 식에 무관하게 전체 조건 식의 결과에 독립적으로 영향을 주도록 함으로써 조건/결정 커버리지를 향상시킨 것 - 결정 커버리지, 조건/결정 커버리지 보다 강력
⑥ 다중조건 커버리지 Multiple Condition Coverage	- 결정 포인트 내에 있는 모든 개별 조건식의 모든 가능한 논리적 조합을 고려한 가장 강력한 논리적 수준의 100% 커버리지를 보장

➔ 명세기반 기법과 함께 사용시 최적의 결과 도출

5.3 구조기반 기법

나. 구문 커버리지

Line Number	Code
1	int func (int arg1, int arg2) {
2	int a = arg1;
3	int b = arg2;
4	int c = 0;
5	if (arg1 > 0)
6	c = a + b;
7	else c= a – b;
8	Return c;
9	}

① 구문 커버리지 (SC)

✓ 소스 코드 라인 수 기준으로 수행된 (Covered) 라인 수 비율

= (수행된 라인 수) / (전체 라인 수) * 100 (%)

✓ 옆 9개의 구문에서 9개 구문 모두 수행이 되면 구문 커버리지는 7 / 9 * 100 = 77.8%가 됩니다.

5.3 구조기반 기법

다. 커버리지 의사결정 테이블 (1/2)

결정포인트(Dpoint)

```
If (A and B) {
  '가'를 실행;
}
else {
  '나'를 실행;
}
```

② 결정 커버리지(DC)

Dpoint	A	B
0	1	0
1	1	1

③ 조건 커버리지(CC)

Dpoint	A	B
0	1	0
0	0	1

④ 조건/결정 커버리지 (CC/DC)

Dpoint	A	B
0	0	0
1	1	1

⑤ 다중 조건 커버리지 (MCC)

Dpoint	A	B
1	1	1
0	1	0
0	0	1
0	0	0

5.3 구조기반 기법

다. 커버리지 의사결정 테이블 (2/2)

결정포인트(Dpoint)

```
If (A and B and C)
{
  '가'를 실행;
}
else {
  '나'를 실행;
}
```

⑥ 변경 조건/결정 커버리지 (MC/DC)

- ✓ 조건 커버리지(CC)와 결정 커버리지(DC)를 보완해서 만든 커버리지
- ✓ MC/DC란 각 개별 조건식이 다른 개별 조건 식에 영향을 받지 않고 전체 조건 식의 결과에 독립적으로 영향을 주도록 함
- ✓ **다른 조건(condition)들의 상태에 상관 없고 자신의 상태가 변경 되었을 때 결과 값에 영향을 미치는 경우 해당 상태는 MC/DC를 만족**

[다중 조건 커버리지 테이블]

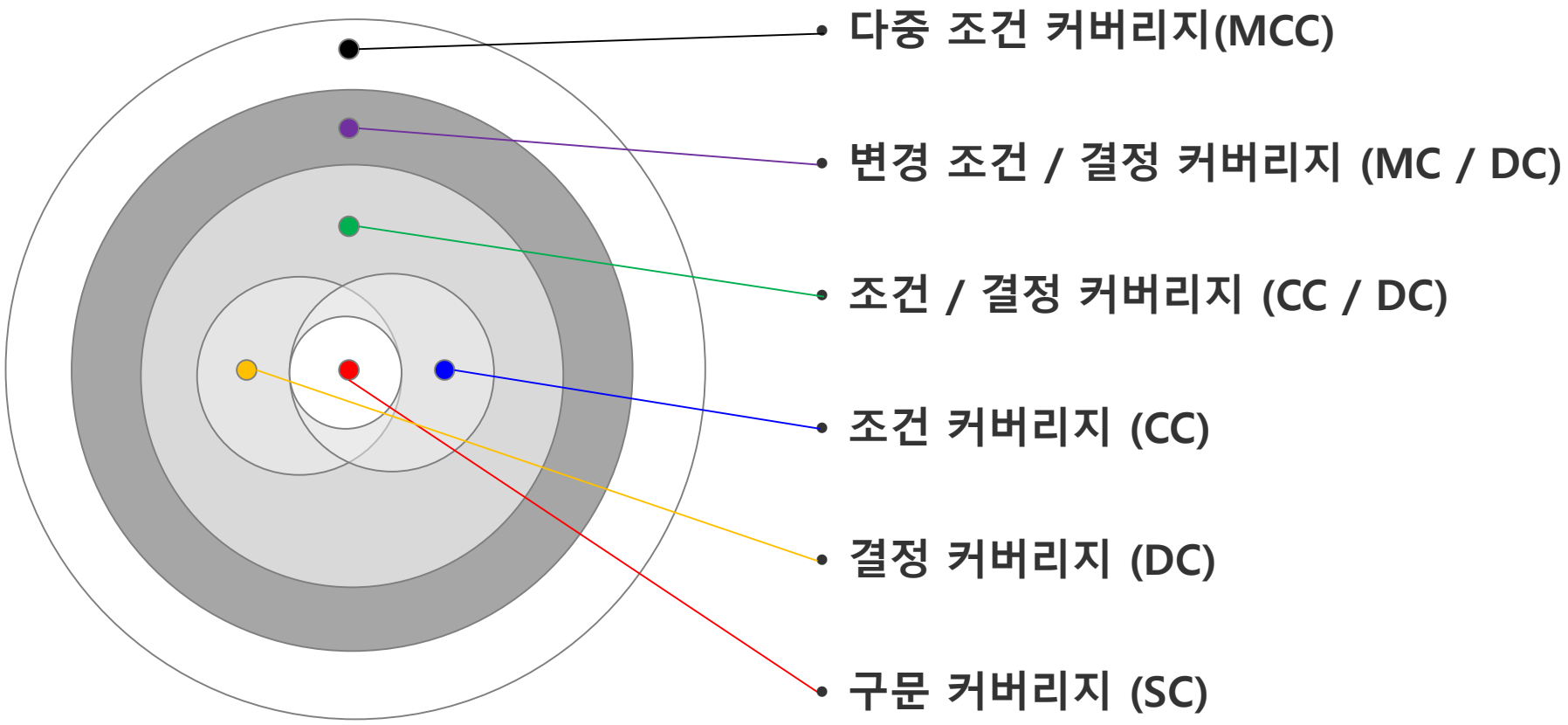
테스트 케이스	A	B	C	Dpoint
1	1	1	1	1
2	1	1	0	0
3	1	0	1	0
4	1	0	0	0
5	0	1	1	0
6	0	1	0	0
7	0	0	1	0
8	0	0	0	0

A 조건을 기준으로 MC/DC를 만족하는 경우는 (1, 5) 테스트 케이스
B 조건을 기준으로 MC/DC를 만족하는 경우는 (1, 3) 테스트 케이스
C 조건을 기준으로 MC/DC를 만족하는 경우는 (1, 2) 테스트 케이스

MC/DC를 만족하는 1,2,3,5 케이스를 모두 만족하면 MC/DC 커버리지가 100%

5.3 구조기반 기법

다. 커버리지 간의 포함 관계



5.4 경험기반 기법

가. 탐색적 테스팅 – 정의 및 구성요소

탐색적 테스팅 정의

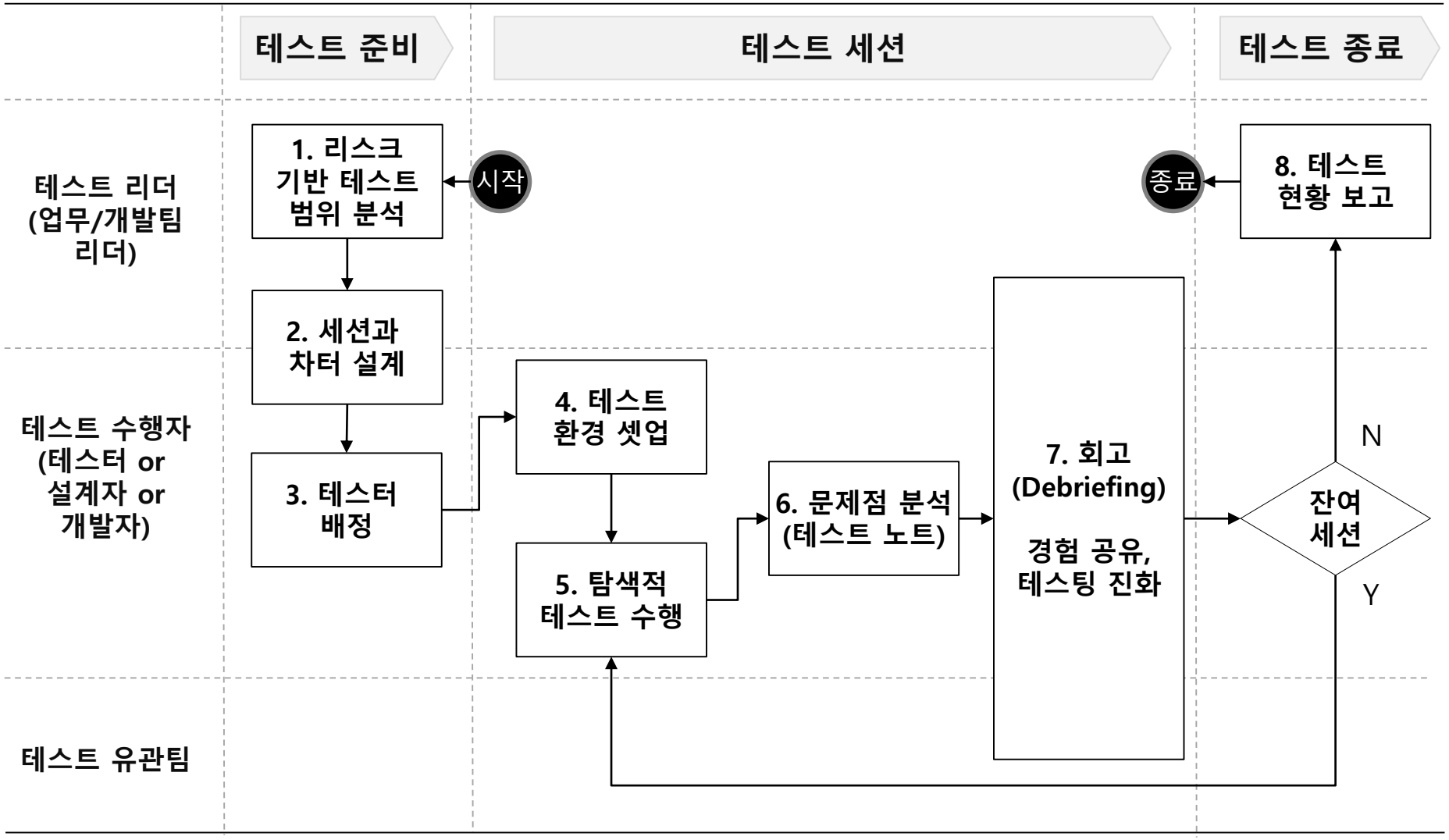
- 테스트 케이스를 먼저 작성하지 않고, 테스트 대상 제품을 실행하면서 익숙해지는 것과 동시에 테스트를 설계하고 테스트를 계획
- 테스트 케이스 작성 시간을 최소화하고 테스트 엔지니어의 유사 경험적인 지적 능력을 최대한 활용하여 테스트를 수행함

[탐색적 테스팅 구성 요소]

테스트 차터 (Test Charter)	테스트 세션 (Test Session)	세션 보고서 (Session Report)	요약 회고 (Debriefing)
• 세션 별 완수해야 할 대상, 목적, 범위 등을 명시 • 기능적 유닛의 설명이 아닌 비전(테스트 중점 사항 등 명시)을 설명	• 테스트를 방해받지 않고, 몰입(집중)할 수 있도록 테스트 수행 시간 지정 예) 60분, 120분	• 테스트 결과지 - 테스트 아이디어, 결과, 특이사항, 결함 등을 기록 • 검토 가능한 결과물 • 테스트 노트라고도 함	• 요약보고 & 리뷰 • 테스트 수행 이력과 경험을 주기적으로 공유 • 잘한 점/못한 점/아이디어 공유하여 다음 Iteration에 적용

5.4 경험기반 기법

가. 탐색적 테스팅 – 테스팅 절차 (Agile 테스팅)



5.4 경험기반 기법

나. 분류 트리 기법 – 정의 및 장점

분류 트리 기법 정의

- 소프트웨어 일부 또는 전체를 트리 구조로 분석 및 표현하고 이를 바탕으로 테스트 케이스를 도출하는 기법분류 트리 기법의 특징
- 분류 트리 기법을 적용할 때 트리를 그려주는 도구(Classification Tree Editor, CTE)의 지원을 받아 테스트할 조합(테스트 케이스) 선택 용이

분류 트리 기법 장점

- 트리구조 시각화 : 의도한 테스트 케이스 도출 가능
- 중복누락 회피 : 시각적으로 보면서 트리구조 끝단을 조합
- 활용 : 복잡한 시스템 or application 일부 또는 전체를 테스팅하는데 적합
- 조기 테스트 설계 활용 : 개발 설계를 체크하는 용도로 사용 가능
- 테스트 비용 추정 가능 : 테스트 케이스 수와 트리의 복잡도 근거

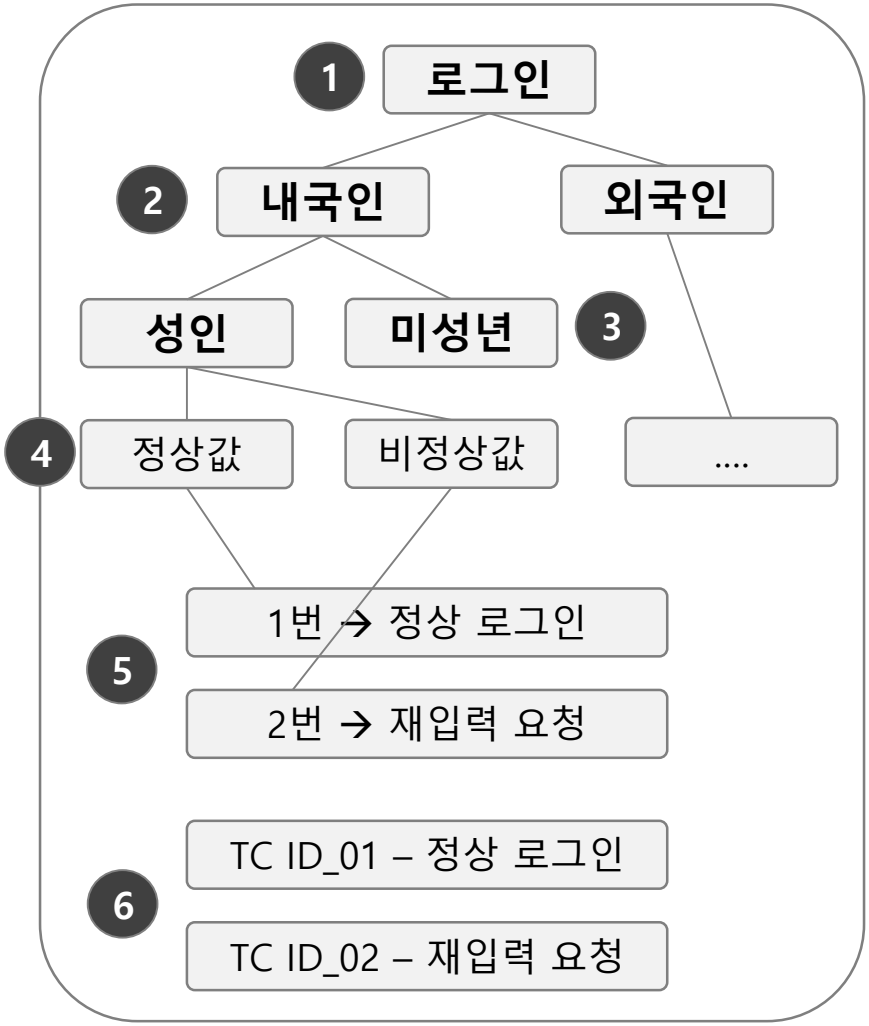
5.4 경험기반 기법

나. 분류 트리 기법 – 절차 및 사례

테스트 절차

1. 테스트 대상 분석
2. 분류 트리의 중요 파라미터 결정
3. 각 파라미터의 세부 트리 구성
4. 각 파라미터 별 한 개 값 선택하고, 해당 지점에 점으로 표시
5. ID 부여 및 기대결과 기술
6. 테스트 케이스 생성

분류 트리 기법 사례



5.4 경험기반 기법

다. 체크리스트 기법 – 정의 및 장점

체크리스트 기법 정의

- 테스트하고 평가해야 할 내용과 경험 및 노하우를 정리하고 목록화
  - 일반 체크리스트, 기능 체크리스트, 시스템 요소 체크 리스트 등 정리하여 활용
- 테스트 경험과 노하우를 정리하고 목록화하여 다음 테스팅에서 재활용 가능

체크리스트 기법 장점

- 기존 경험과 노하우를 목록으로 정리하여 재활용함으로써 테스트 설계 시간 단축가능
- 테스트 초보자나 도메인 지식이 없는 테스터도 일정 수준의 품질 점검 가능

5.4 경험기반 기법

라. 오류 추정 기법 – 정의 및 장점

오류 추정 기법 정의

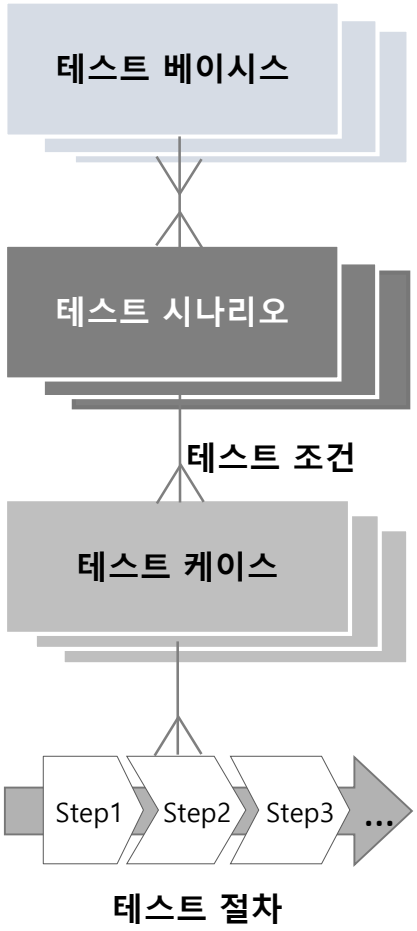
- 가장 일반적으로 사용되는 기법. 테스터의 경험에 기반하여 결함을 예측
  - 애플리케이션의 과거 실패 사례
  - 개발자가 자주 하는 실수 유형
  - 유사 어플리케이션에서 발생한 오류
- 가능한 오류를 모두 나열하고 이를 유발할 수 있는 테스트를 설계 – 결점 공격
- 테스터가 시스템을 잘 알고 있다는 전제로 취약점 식별 작업을 수행하기 때문에 테스트 프로세스의 가장 마지막 단계에서 활용

오류 추정 기법 장점

- 스펙이 거의 없거나 불충분할 때, 시간적인 압박이 극심할 때 유용
- 다른 기법이나 공식적 테스트를 보완할 때 유용
- 가장 심각한 결함을 찾았다고 확신할 수 있도록 지원함으로써 테스트 프로세스를 점검하는 기준 제공

별첨 #2. 테스트 설계용어 정의

테스트 설계 구조



테스트 설계 용어정의

테스트 베이시스 (Test Basis)	✓ 테스트 케이스의 기반이 되는 모든 문서를 의미 ✓ 예) 요구사항 명세서, 업무기능 흐름도, 프로그램 명세서 등
테스트 시나리오 (Test Scenario)	✓ 테스트 대상 시스템을 테스트하기 위한 상위 레벨의 테스트 요건 (기능, 비기능), "What to be Tested" ✓ 예) 기능 목록의 L2 or L3 레벨(회원가입, 탈퇴 등)
테스트 조건 (Test Condition)	✓ 테스트 시나리오를 실행하기 위한 구체적인 테스트 조건으로 테스트 케이스 도출 (테스트 조건이 많을 경우 핵심 조건을 기반으로 도출) ✓ 예) 회원가입 시 유료/무료, 결제 시 카드/현금 결제 조건
테스트 케이스 (Test Case)	✓ 테스트 시나리오의 다양한 경우를 정의한 것으로 사전조건, 테스트 절차, 입력조건, 예상결과 등이 포함됨, "How to be tested" ✓ 예) 회원가입_유료_카드, 회원가입_유료_현금, 회원가입_무료 등
테스트 절차 (Test Step)	✓ 테스트 케이스를 수행하기 위한 테스트 절차를 상세화 ✓ 예) st1.로그인, st2.검색조건입력, st3.조회 클릭 등

Contents

1	인공지능 이해
2	품질과 테스트 이해
3	AI 발전과 프로젝트 활용
부록 #1	AI 발전 트랜드

1. AI 모델 자체 능력의 발전

가. AI 발전 단계

AI 시스템의 발전 과정: 언어 이해에서 자율적 문제 해결까지



각 단계는 이전 단계를 대체하지 않고, 그 위에 새로운 능력층을 쌓아가는 방식으로 발전했습니다.

1. AI 모델 자체 능력의 발전
가. AI 발전 단계_LLM 단계

STEP 1

LLM (Large Language Model)

대규모 텍스트 데이터로 사전 학습된 언어 이해·생성 기반 모델

대표 사용 사례 : 챗봇

GPT, Claude, Gemini, Grok 등
언어모델 기반 챗봇

특징

- 방대한 텍스트 코퍼스*주1)로 사전 학습(pre-training)
- In-context learning(맥락 내 학습)으로 예시만으로 새 작업 수행
- 프롬프트에 따라 다양한 태스크에 범용 적용 가능

작동 원리

- Transformer 디코더 구조 기반
- Self-Attention으로 문맥 내 단어 간 관계 학습
- 다음 토큰을 확률적으로 예측(Next-token prediction)
- RLHF·SFT*주1)로 인간 선호에 맞게 미세조정

* 주1) 코퍼스 - 모델 학습을 위해 모아 둔 대규모 언어(텍스트·음성) 데이터의 집합

* 주2) RLHF(Reinforcement Learning from Human Feedback, 인간 피드백 기반 강화학습)
SFT(Supervised Fine-Tuning, 지도 미세조정)

기술적 구성 요소

- Tokenizer(BPE 등) – 텍스트를 작은 조각(토근)으로 분해
- 멀티 헤드 어텐션(Multi-head Attention)과 피드 포워드 레이어(Feed-Forward Layer)는 입력된 데이터의 관계를 파악하고 정교하게 변환하는 역할
- 대규모 GPU 클러스터 기반 분산 학습

대표 모델

- GPT 시리즈 (OpenAI)
- Claude 시리즈 (Anthropic)
- Gemini 시리즈 (Google)
- Llama 시리즈 (Meta), Qwen 시리즈 (Alibaba)

한계점

환각(Hallucination) 발생 | 학습 시점 이후 정보 반영 불가(지식 컷오프) | 외부 행동·실행 능력 없음(정적 응답)

1. AI 모델 자체 능력의 발전
가. AI 발전 단계_RAG 단계

STEP 2

RAG (Retrieval-Augmented Generation)

외부 지식 검색을 결합해 LLM의 정확성과 최신성을 보강

대표 사용 사례 : 챗봇

사내 문서 Q&A, 기술지원 챗봇,
벡터DB(Pinecone 등) 기반 검색

특징

- LLM 파라미터를 재학습하지 않고 지식 확장
- 최신·도메인 특화 정보를 응답에 반영
- 출처 기반 응답으로 신뢰도·설명력 향상

작동 원리

- ① 질의를 임베딩 벡터로 변환
- ② 벡터 DB에서 유사 문서 검색(Retrieval)
- ③ 검색된 문서를 프롬프트에 삽입(Augmentation)
- ④ LLM이 근거 문서를 바탕으로 답변 생성

기술적 구성 요소

- Embedding Model, Vector Database
- Chunking·Indexing 파이프라인
- Retriever, Reranker(정밀도 향상)

대표 도구

- LangChain, LlamaIndex (프레임워크)
- Pinecone, Weaviate, Chroma (벡터 DB)
- FAISS (Meta)

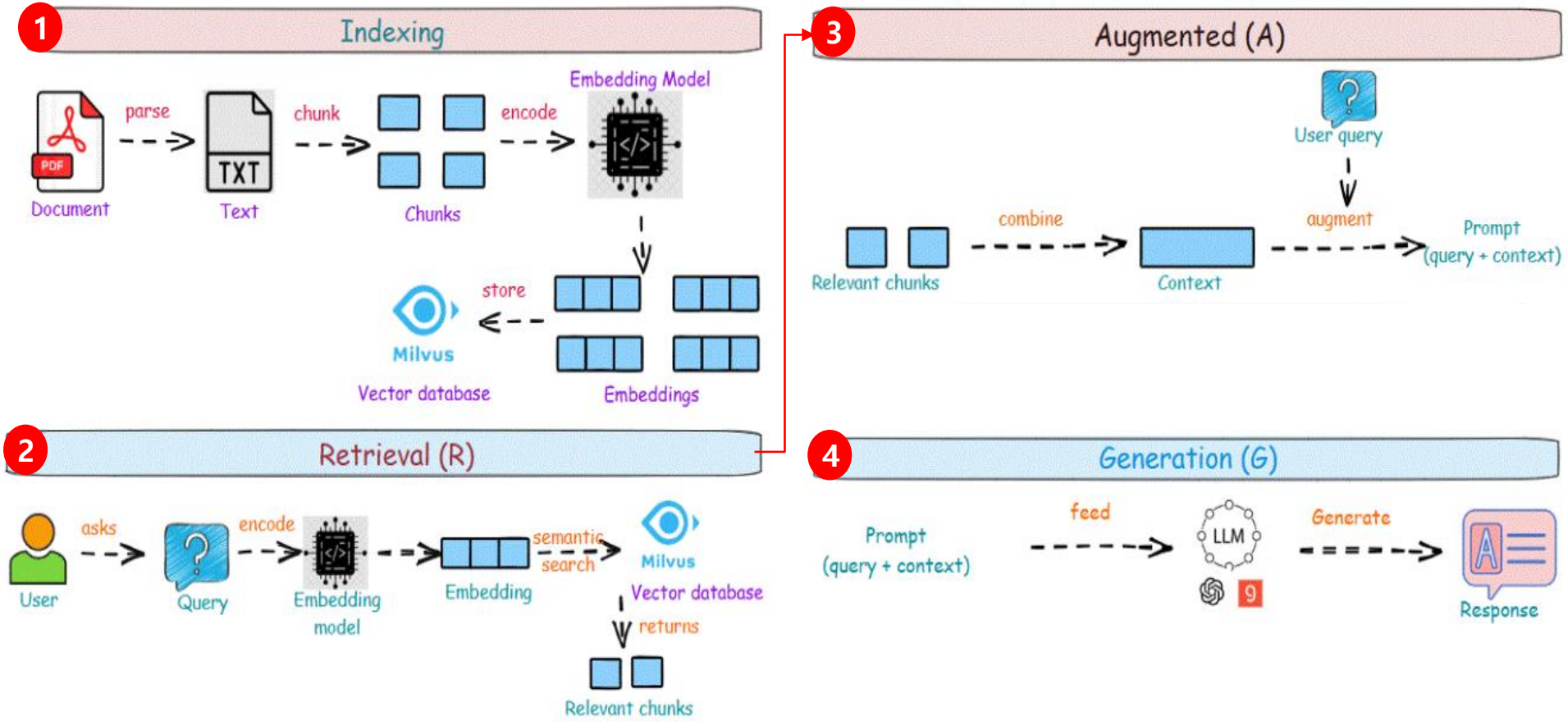
한계점

검색 품질에 결과 의존 | 여전히 단발성 질의응답 중심(연속 행동 불가) | 복잡한 멀티스텝 추론에는 한계

1. AI 모델 자체 능력의 발전

가. AI 발전 단계_RAG 단계_Process

RAG 절차 : Indexing → Retrieval → Augmented → Generation



1. AI 모델 자체 능력의 발전

가. AI 발전 단계_Agent 단계

STEP 3

Agent

LLM이 도구를 호출하고 행동을 수행하며 작업을 실행

대표 사용 사례 : Assistant

코딩 어시스턴트, 자동 리서치,
업무 워크플로 자동화

◎ 챗봇과 에이전트의 차이

구분	일반 챗봇(LLM)	AI 에이전트(Agent)
동작 방식	질문 → 답변 (1회)	작업 계획 및 실행 (목표 → 판단 → 도구 실행 → 결과 확인)
외부 세계와 상호작용	불가 (학습된 지식만 사용)	가능 (파일, API, DB, 웹 검색 등)
예시	"파이썬 리스트 정렬법 알려줘"	"이 폴더의 로그를 분석해서 오류 요약 보고서를 만들어줘"
난이도	쉬움	설계와 안전장치가 필요함

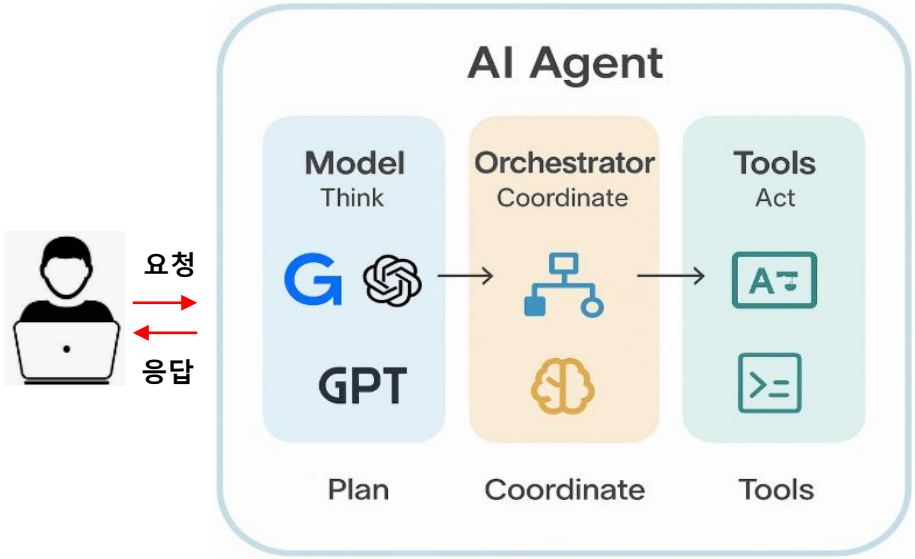
◎ 에이전트의 4가지 구성 요소

구성 요소	역할	비고
모델 (LLM, 두뇌)	상황을 이해하고 다음 행동을 결정	Claude, ChatGpt, Gemini 등
도구 (Tools, 손발)	실제 작업을 수행하는 함수	시간 조회, 계산기, 파일 읽기/목록
지시문 (System Prompt, 업무 지침)	역할, 규칙, 금지사항 정의	규칙파일(Claude.md, Agent.md), Skill
대화 이력 기억 (Memory)	도구 결과를 모델에게 돌려주며 반복, 대화 기록 유지	이전 상호작용과 사용자 선호도를 유지하는 단기/장기 기억

1. AI 모델 자체 능력의 발전

가. AI 발전 단계_Agent 단계

◎ 에이전트의 동작 원리

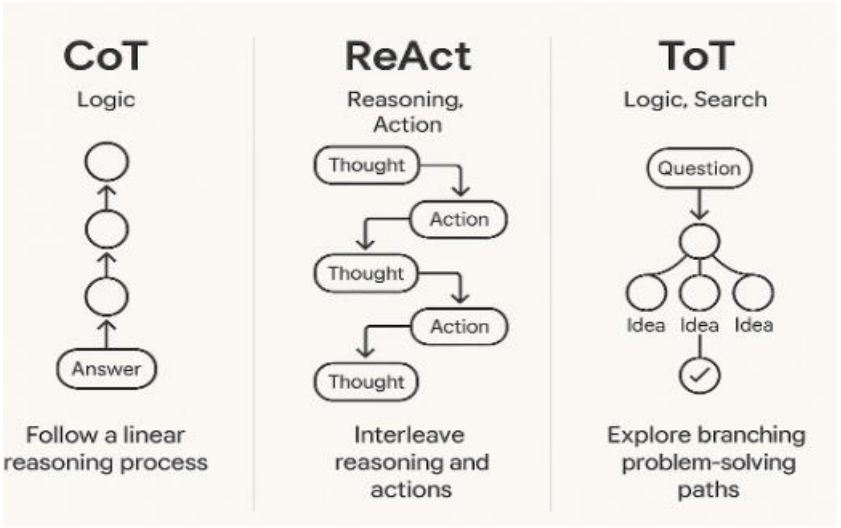


❖ **Model – 생각한다 (Think)**
✓ 대화를 이해하고, 추론하며, 콘텐츠를 생성하는 지능의 핵심
✓ 예: Gemini, GPT(모델), Microsoft Copilot(모델 기반 서비스)

❖ **Orchestration – 조율한다 (Coordinate)**
✓ 요청의 목적을 해석하고, 어떤 도구를 어떤 순서로 사용할지 계획
✓ 상태를 유지하며 흐름을 제어

❖ **Tools – 실행한다 (Act)**
✓ 외부 API, 함수, 데이터 저장소 등을 활용해 실제 행동을 수행
✓ 예: 문서 생성, 캘린더 등록, 시스템 제어 등

◎ 에이전트의 사고 방식



❖ **ReAct – 생각하고 바로 행동!**
✓ **방식**: 판단과 실행을 연속적으로 수행
✓ **적합한 상황**: 빠른 대응과 반복 작업이 필요한 경우
✓ **예시**: "받은 이메일 → 미팅 요청 감지 → 캘린더 자동 등록"

❖ **Chain of Thought (CoT) – 논리적으로 단계 구상**
✓ **방식**: 문제 해결을 위한 중간 사고 과정을 단계적으로 서술
✓ **적합한 상황**: 절차가 복잡하거나 조건이 많은 작업
✓ **예시**: "출장 준비 → 항공권 검색 → 숙소 비교 → 일정 조정"

❖ **Tree of Thought (ToT) – 다양한 가능성을 평가하며 결정**
✓ **방식**: 여러 아이디어를 병렬로 전개한 후 장단점을 비교
✓ **적합한 상황**: 창의적 문제 해결, 옵션 비교
✓ **예시**: "호텔 A vs B vs C → 교통/가격/후기 비교 → 최종 선택"

⚠ 한계점 단일 목표·단일 에이전트 중심 | 장기 계획·다중 협업 부족 | 오류 누적 시 스스로 복구하기 어려움

1. AI 모델 자체 능력의 발전

가. AI 발전 단계_Agent 단계

◎ 클라우드 API 모델 vs 로컬(오픈 웨이트) 모델

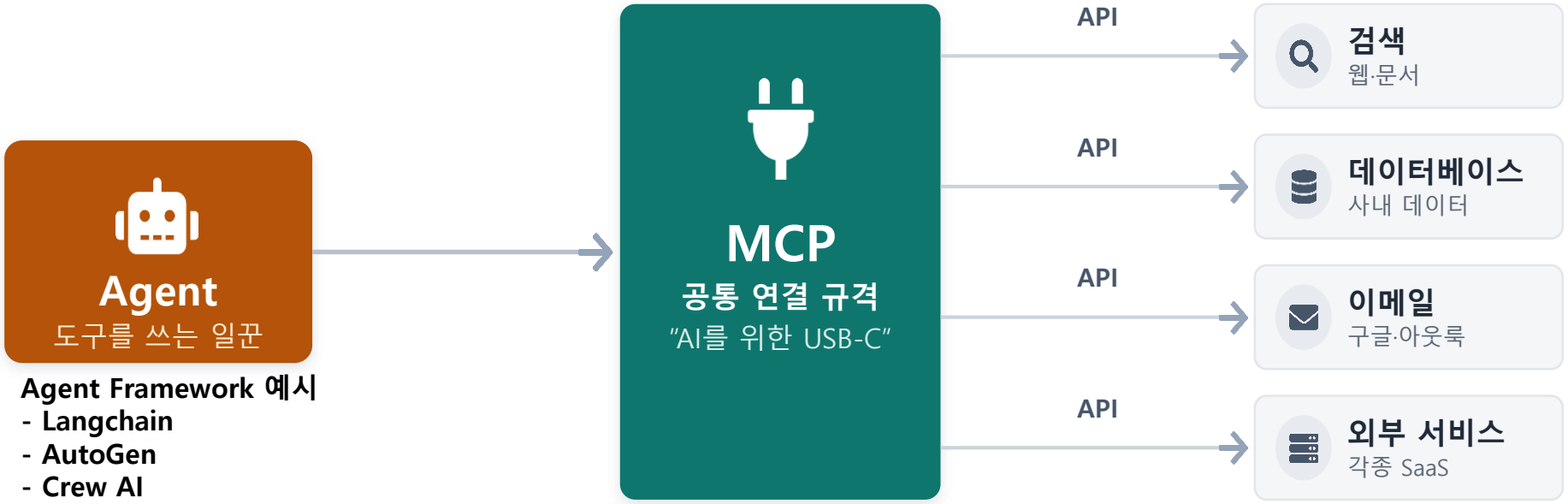
구분	상용 AI (클라우드 기반)	로컬 AI (오픈웨이트/설치형)
운영 방식	원격 데이터센터(OpenAI, Anthropic 등) 서버 이용	내 PC, 맥북, 자체 서버의 GPU/CPU에서 직접 구동
대표 모델	Anthropic Claude, OpenAI GPT, Google Gemini	Llama, Qwen, Gemma, Mistral, EXAONE 등 (Ollama로 실행)
성능 및 추론	최첨단 대형 모델(GPT-4o, Claude 등)로 최고 수준의 복잡한 추론 가능	하드웨어 사양에 제한되며, 최상위 상용 모델보다 성능이 낮음
초기 비용	낮음 (사용량만큼 구독료 또는 토큰 비용 지불)	높음 (고성능 GPU/맥북 등 하드웨어 구축 비용 필요)
장기 비용	사용량이 늘어날수록 비용 증가 (토큰당 과금)	초기 장비 투자 외에는 추가 사용 비용(전기세 등)은 비교적 적음
보안/데이터	데이터가 외부 서버로 전송됨 (기업 기밀 유출 위험 주의)	데이터가 외부로 나가지 않아 완벽한 보안 및 프라이버시 보호
설치 난이도	매우 쉬움 (API 키만 발급)	보통 (모델 다운로드, 사양 확인)
도구 호출 안정성	높음	모델마다 편차 큼 (도구 호출 지원 모델 선택 필수)
초보자 추천	★★★ 처음 시작할 때 추천	★★ 개념을 익힌 뒤 또는 사내 보안상 필요할 때
인터넷 연결	반드시 인터넷 연결 필요	오프라인 환경에서도 사용 가능

1. AI 모델 자체 능력의 발전

가. AI 발전 단계_Agent 단계_MCP 이해

MCP(Model Context Protocol)는 Anthropic에서 발표한 프로토콜

LLM 모델이 외부 애플리케이션과 연동할 수 있도록 해주는 표준화한 스펙



Agent Framework 예시
- **Langchain**
- **AutoGen**
- **Crew AI**
- **n8n**

API 프로그램끼리 데이터를 주고받는 개별 창구(문).

MCP 여러 도구 연결을 표준화한 공통 규격(마스터키).

Agent 도구를 호출해 다단계로 일을 수행하는 LLM 일꾼.

1. AI 모델 자체 능력의 발전

가. AI 발전 단계_Agentic AI 단계

STEP 4

Agentic AI

여러 에이전트가 목표를 스스로 세우고 협업하며 복잡한 과제를 해결

대표 사용 사례

자율 업무 처리, 멀티에이전트 협업 (CrewAI, AutoGen 등)

특징

- 자율적 목표 설정 및 다단계 계획 수립
- 다중 에이전트 간 역할 분담과 협업
- 자기 평가(self-reflection)로 지속적 개선

작동 원리

- Planner - Executor - Critic 구조로 계획·실행·검토
- 멀티 에이전트 오케스트레이션으로 작업 분배
- 장기 메모리로 과거 작업 맥락 유지
- 필요 시 Human-in-the-loop으로 개입·승인

기술적 구성 요소

- Orchestration Layer, Multi-agent Framework
- Persistent/장기 Memory, 도구 생태계 연동
- MCP 등 표준 프로토콜 기반 도구·서비스 연결

대표 도구

- AutoGen (Microsoft), CrewAI
- LangGraph (LangChain)
- Claude Agent SDK / Claude Code (Anthropic)

한계점

연산 비용·응답 지연 급증 | 행동 예측 불가능성·안전성 통제 난이도 상승 | 평가·디버깅 및 책임 소재 파악이 복잡

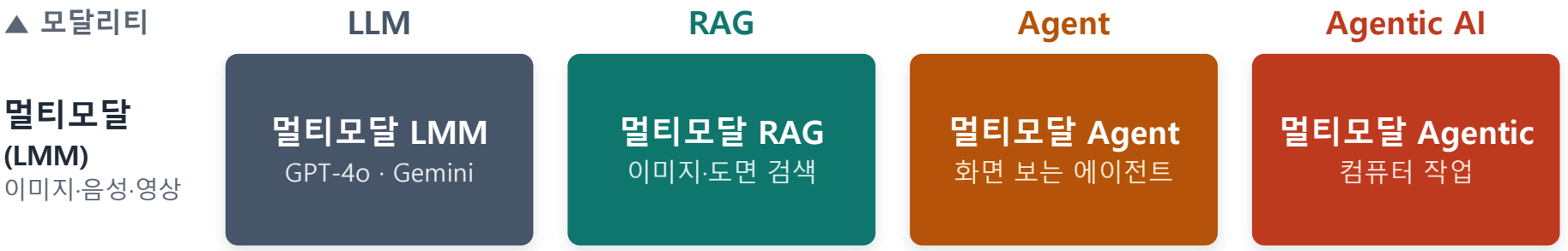
1. AI 모델 자체 능력의 발전

나. 단계별 비교 요약

자율성과 능력은 갈수록 커지고, 그만큼 제어 난이도와 비용도 함께 커집니다.

구분	LLM	RAG	Agent	Agentic AI
자율성	낮음	낮음	중간	높음
지식 출처	내부 학습	+ 외부 검색	+ 도구·API	+ 공유 메모리
도구 사용	없음	검색 한정	다양한 도구	자율 선택·조합
작업 범위	단일 응답	단일 응답	다단계	복합·장기 목표
사람 개입	항상	항상	감독	최소(목표만)

생성형 언어 모델(LLM)에서 생성형 멀티 모달(LMM)로의 진화로 이미지, 음성, 영상으로 넓이가 확대되어 갑니다.



2. AI 활용 능력의 진화

가. AI 활용 능력의 5단계 진화

프롬프트 엔지니어링 → 컨텍스트 엔지니어링 → 하네스 엔지니어링 → 루프 엔지니어링 → 그래프 엔지니어링



용어는 바뀌어도 목적은 하나 — AI가 더 정확하고 안전하게, 더 큰 단위로 일하게 만드는 것입니다.

2. AI 활용 능력의 진화

나. 프롬프트 엔지니어링



STEP 1

프롬프트 엔지니어링 (Prompt Engineering)

2020~2022년경 GPT-3와 ChatGPT 대중화와 함께 확산된 가장 기본적인 AI 활용 기술

AI 모델에게 원하는 결과를 얻기 위해 지시문(질문·명령)을 정교하게 설계하는 기술입니다. 모델이 보는 정보의 대부분이 프롬프트 한 번에 담겨 있던 **초기 LLM 활용 방식의 핵심**이었습니다.

특징

- 사람이 매번 직접 지시문을 작성하고 결과를 확인
- 지시의 명확성·구체성이 결과 품질을 좌우
- 예시(few-shot)를 보여주면 원하는 형식을 더 잘 따름
- 모델·태스크마다 효과적인 표현이 달라 반복 튜닝 필요

대표 도구·기법

- Zero-shot / Few-shot Prompting
- Chain-of-Thought(CoT) — 단계별 사고 유도
- ReAct 프롬프팅(추론+행동 지시)
- Prompt 템플릿(LangChain PromptTemplate 등)

필수 구성 요소

Instruction(지시) 모델이 수행할 작업을 명확히 전달

Few-shot 예시 원하는 출력 형식·톤을 예시로 제시

Output 형식 지정 표·JSON 등 결과 구조를 사전 지정

Role/Persona 모델의 역할과 관점을 설정

예시

"너는 5년 차 백엔드 개발자다. 아래 요구사항을 바탕으로 REST API 설계를 표로 정리해줘."
— 역할·지시·출력 형식을 한 프롬프트 안에 담은 전형적인 예시

2. AI 활용 능력의 진화
다. 컨텍스트 지니어링

STEP 2

컨텍스트 엔지니어링 (Context Engineering)

2025년 6월, Shopify CEO Tobi Lütke의 트윗을 계기로 Karpathy 등이 동참하며 급속히 확산

"질문을 잘 하기"를 넘어, 모델이 참고할 배경 정보·도구·이력 등 컨텍스트 전체를 설계하는 기술입니다. 프롬프트는 모델이 보는 것의 일부일 뿐이며, 나머지는 도구·예제·대화 기록·검색 문서가 채웁니다.

특징

- 프롬프트보다 넓은 범위 — 모델이 보는 모든 입력을 설계
- **프로젝트 규칙·컨벤션을 매번 반복 설명할 필요 없이 지속 제공**
- 외부 데이터·API를 실시간으로 연결해 최신성 확보
- 컨텍스트 윈도우 한계로 인해 관리(압축·요약)가 중요

대표 도구·기법

- CLAUDE.md, AGENTS.md (프로젝트 컨텍스트 파일)
- MCP(Model Context Protocol) — 외부 도구·데이터 연결
- RAG(검색 증강 생성)
- 컨텍스트 압축(Context Compaction)

필수 구성 요소

프로젝트 컨텍스트 파일 코딩 규칙·테스트 방식 등을 지속 제공

외부 지식 연결 RAG·MCP로 최신·정확한 데이터 공급

대화 이력 관리 (메모리) 핵심만 남기고 압축해 컨텍스트 관리

Tool 정의(스키마) 모델이 쓸 수 있는 도구와 사용법 명시

예시

CLAUDE.md에 "이 프로젝트는 JavaScript, 테스트는 Jest, 에러 처리는 Result 타입 사용"이라 기록해두면, 매 요청마다 이 컨텍스트를 참고해 일관된 코드를 생성

2. AI 활용 능력의 진화

다. 컨텍스트 지니어링 – 프롬프트와 비교

프롬프트 엔지니어링 vs 컨텍스트 엔지니어링

구분		프롬프트	컨텍스트
정의		• LLM에 입력할 텍스트(지시, 예시, 형식)를 설계	• 모델이 접하는 모든 정보를 조립하는 시스템 설계
범위		• **질문, 지시, 예시, 출력 형식**	• **프롬프트 + 메모리 + RAG + 도구**
초점		• 더 나은 질문을 하는 것	• 더 나은 시스템을 만드는 것
주요 기법	역할 설정 vs 맥락 유지	• 모델에게 "너는 보안 전문가다"와 같은 역할 지정 → 응답의 어휘·관점이 달라짐	• 단기 대화 이력 + 장기 사용자 기록을 기억해 **맥락 유지** → 고객센터 챗봇이 환불 요청을 자연스럽게 이어감
	Few-shot 예시 vs 검색 증강 생성(RAG)	• 2~3개의 예시를 보여주어 모델이 원하는 포맷을 따라가도록 유도 → 결과에 순서, 출장비 구분, 내용, 설명 포함해줘.	• 방대한 문서 중 필요한 부분만 추출해 자료 제공 → 출장비 템플릿을 활용해서 결과를 알려줘.
	Chain of Thought vs 상태 관리	• "단계별로 생각해보자"라는 지시로 추론 과정을 드러냄	• 다단계 **업무 진행 상태**를 추적해 맥락 단절 방지 → 항공권 예약, 호텔·교통 예약까지 연계 처리
	제약 조건 설정 vs 도구 활용	• 답변을 100단어 이내로 제한 → 간결한 요약 제공	• 외부 API·데이터베이스 연결을 통해 실시간 데이터를 가져옴 → 주식 가격 조회 등 실행 가능
한계		• 추론 능력 부재, 기억 능력 없음, 비학습 정보를 지어내는 환각 현상 발생	• 긴 문서나 대화의 가운데 부분에 중요한 정보가 있으면 AI가 인지 부족 • 대화가 길어지면 추론 능력과 답변 정확도 급락

2. AI 활용 능력의 진화

다. 컨텍스트 지니어링 – 주요 AI 도구의 규칙 파일

1. 주요 AI 도구의 규칙 파일

주요 AI 코딩 도구의 규칙 파일

도구	파일 이름	설명	특징
AI Claude Code	CLAUDE.md	프로젝트의 규칙과 맥락을 정의	세션 시작 시 자동 로딩
GPT Codex (OpenAI)	AGENTS.md	프로젝트 지침과 작업 규칙 정의	여러 도구에서 폭넓게 지원
CUR Cursor	.cursor/rules/ 또는 .cursorrules	프로젝트 규칙과 컨벤션 정의	Cursor 환경에 최적화
GH GitHub Copilot	.github/ copilot-instructions.md	리포지토리 사용 규칙과 코딩 가이드 정의	GitHub 연동 시 자동 활용
GEM Gemini CLI, Cline, Windsurf 등	대부분 AGENTS.md 호환	AGENTS.md 표준을 따르는 프로젝트 지침	표준 포맷으로 호환성 높음

2. AI 활용 능력의 진화

다. 컨텍스트 지니어링 – 작동원리 및 규칙 내용

2. 작동 원리 및 규칙 내용



2. AI 활용 능력의 진화
라. 하네스 지니어링

STEP 3

하네스 엔지니어링 (Harness Engineering)

2025년 말~2026년 초, "Relocating Rigor"(Chad Fowler) 등을 통해 컨텍스트 엔지니어링 다음 단계로 부상

프롬프트와 컨텍스트를 감싸 AI 에이전트가 안전하게, 궤도를 이탈하지 않고 작동하도록 만드는 주변 환경과 인프라를 설계하는 기술입니다. **규칙을 문서로 적어두는 대신 시스템적으로 강제(Guardrails)**합니다.

특징

- 모델 성능보다 "환경 설정"이 결과를 좌우하는 경우가 많음
- 규칙 위반을 기계적으로(mechanically) 차단
- 에러 발생 시 수정 지침을 에이전트 컨텍스트에 자동 주입
- **같은 모델이라도 하네스가 다르면 결과 품질이 크게 달라짐**

대표 도구·기법

- 커스텀 린터(Linter) · 구조 테스트
- Claude Code / OpenAI Codex 실행 환경
- 가드레일(Guardrails)
- 샌드박스 격리 실행 환경

필수 구성 요소

도구 실행 환경
안전한 실행(샌드박스)과 권한 범위를 제한

권한·가드레일
위험한 행동은 차단하거나 사용자 승인 요청

상태·컨텍스트 관리
진행 상황 저장, 컨텍스트 자동 압축·복원

검증·피드백
테스트·린터 등으로 규칙 위반 코드를 기계적으로 차단

관찰·로깅
모든 단계를 기록해 디버깅·평가에 활용

※ 하네스(harness)는 '말을 다루는 마구'라는 뜻 — 강력한 모델을 원하는 방향으로 안전하게 일하게 만드는 실행 시스템

2. AI 활용 능력의 진화
라. 하네스 지니어링

Context와 Harness의 역할 구분

Context Engineering
CLAUD.md – "무엇을 알아야 하는가?

- 기술 스택 (Java 17, Spring Boot 3, Gradle)
- 빌드 · 테스트 · 실행 명령어 안내
- 패키지 구조와 레이어링 규칙
- 코딩 컨벤션 (네이밍, 예외 처리 방식)
- 하지 말아야 할 것 (예, 컨트롤러에 비즈니스 로직 금지)
- 도메인 지식 · 비즈니스 배경 설명

Harness Engineering
settings.json / hooks – "무엇을 해도 되는가?

- 허용된 도구 목록 (allow / deny / ask)
- 위험 명령 차단 (rm –rf, git push –force 등)
- 커밋 전 테스트 강제 실행 (pre-tool-use hook)
- 실행 결과 로깅 (post-tool-use hook)
- 외부 시스템 연결 통제 (.mcp.json)
- 작업 디렉토리 격리 · 타임아웃 ᵒ 재시도

- Context는 판단 재료를, Harness는 실행 통제를 담당
- 예를 들어, CLAUDE.md에 '위험한 명령은 실행하지 마세요'라고 적어도 이는 **권고일 뿐 강제력이 없음**
  실제로 위험한 명령을 차단하려면 settings.json의 deny 목록과 pre-tool-use hook 같은 하네스 장치가 필요

Claude Code 하네스 실행 플로우 – "테스트 실행해줘" 요청 예시



2. AI 활용 능력의 진화

마. 루프 지니어링

사람이 매 턴 프롬프트를 입력하는 대신, **에이전트가 스스로 발견-계획-실행-검증-반복하는 피드백 루프 자체를 설계·운영하는 기술**입니다.

루프 엔지니어링 특징

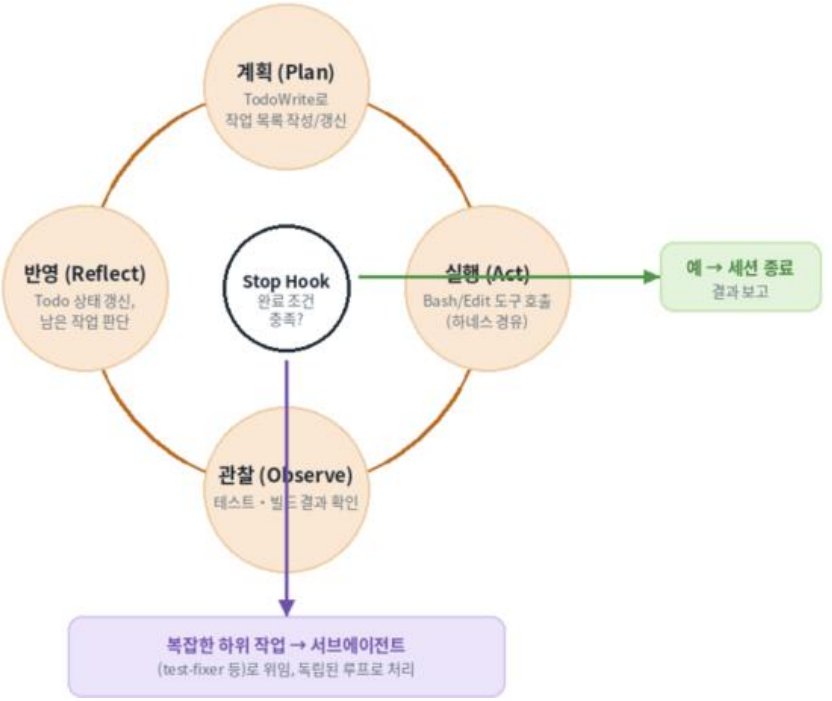
- 사람이 매번 지시하는 대신, 지시를 대신하는 시스템을 설계
- **제작 에이전트와 검증 에이전트를 분리**해 신뢰도 확보
- 목표만 정의하면 완료될 때까지 자동으로 반복
- 종료 조건·검토 절차가 없으면 무한 반복·비용 낭비 위험

Loop Engineering

Stop Hook / Subagent – "언제 실제로 멈추는가?

- Stop Hook으로 **종료 조건 강제 검증** (테스트 실패 시 차단)
- SubagentStop Hook으로 하위 루프 종료 통제
- 서브에이전트로 반복 작업을 격리된 루프로 분리
- 최대 반복 횟수 · 동일 오류 반복 시 강제 중단 로직
- 각 반복의 실행 결과를 다음 반복 상태로 전달

Claude Code 에이전트 루프 – Java 백엔드 작업 예시



통합 테스트 종료 포인트

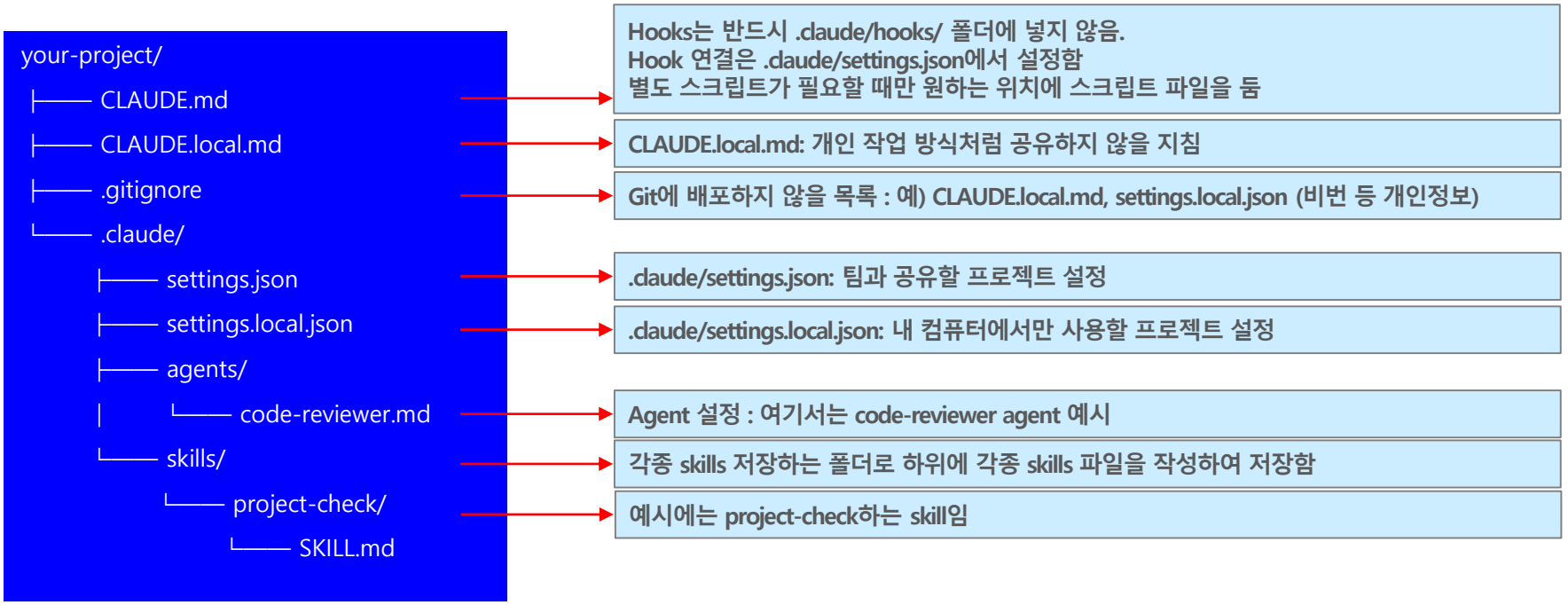
- ✓ **Stop Hook이 테스트 실패 시 종료를 차단**
- ✓ **동일 오류 반복 시 강제 중단**
- ✓ **서브에이전트 루프가 독립적으로 종료 판단**

별첨 #3. CLAUDE Code 예시 (1/6)

1. Claude 주요 환경 설정

구성 요소	주요 내용	사용 시점
CLAUDE.md	프로젝트 업무 지시서	Claude가 항상 알아야 할 정보가 있을 때
Skills	반복 업무 매뉴얼	같은 절차를 여러 번 사용할 때
Agents	역할별 AI 팀원	리뷰·조사 등 독립적인 역할이 필요할 때
Hooks	자동 안전장치	특정 순간 검사나 명령을 실행해야 할 때

2. 최소 폴더 구조 만들기



별첨 #3. CLAUDE Code 예시 (2/6)

3. CLAUDE.md 예시

➔ 3.1 규칙 파일(CLAUDE.md)을 만들 프로젝트로 이동한다.

```
Windows PowerShell
PS C:₩WINDOWS₩system32>cd d:₩my-project
PS D:₩my-project₩claude
```

➔ 3.2 Claude CLI에서 /init 명령어를 통해 CLAUD.md 파일을 만든다.



Claude Code v2.1.283
Haiku 4.5 · API Usage Billing
D:₩my-project

> **/init**

➔ 3.3 D:₩my-projec에 CLAUDE.md 파일이 생성되면 아래와 같이 저장하고 claude를 다시 시작하거나 변경되었음을 프롬프트에 알려준다.

Project Guide

프로젝트 개요
이 프로젝트는 [프로젝트 목적]을 위한 프로젝트입니다.
주요 사용자는 [사용자 유형]이며,
핵심 목표는 [사용자가 얻어야 할 결과]입니다.

기술 스택
- Frontend: [예: Next.js]
- Styling: [예: Tailwind CSS]
- Database: [예: Supabase]
- Deploy: [예: Vercel]

주요 폴더
- `/app`: 페이지와 라우팅
- `/components`: 재사용 컴포넌트
- `/lib`: 유틸리티와 외부 서비스 연결
- `/public`: 이미지와 정적 파일

작업 원칙
- 작업 전 관련 파일과 기존 구조를 먼저 확인한다.
- 현재 디자인과 코드 스타일을 유지한다.
- 불필요한 라이브러리를 추가하지 않는다.
- 요청과 관계없는 리팩터링은 하지 않는다.

금지사항 (안전 장치)
- 환경변수와 비밀키를 출력하지 않는다.
- 승인 없이 배포하지 않는다.
- 데이터베이스 구조를 임의로 변경하지 않는다.
- 기존 사용자 작업을 임의로 삭제하지 않는다.

작업 후 확인
- 관련 테스트를 실행한다.
- 린트와 빌드 오류를 확인한다.
- 수정한 파일과 이유를 요약한다.
- 확인하지 못한 항목은 명확하게 밝힌다.

응답 방식
- 결론을 먼저 말한다.
- 변경한 파일과 변경 이유를 설명한다.
- 위험하거나 되돌리기 어려운 작업은 먼저 확인한다.

[규칙 파일 관리]

✓ 회사 소개나 긴 기획 문서는 docs/에 두고 링크만 연결
✓ "잘 만들어줘" 대신 "수정 후 npm run build를 실행한다"처럼 확인 가능한 규칙을 만들어 점검
✓ 자주 바뀌는 정보는 CLAUDE.md에 넣지 않음
✓ 반복 절차가 길어지면 Skill로 옮김 (최대 500라인 이하로 유지)

별첨 #3. CLAUDE Code 예시 (3/6)

4. Skill 만들기

➔ 4.1 특정 프로젝트만 사용 또는 모든 프로젝트에서 사용

```
# 이 프로젝트에서만 사용
your-project/
└── .claude/
    └── skills/
        └── project-check/
            └── SKILL.md

# 모든 프로젝트에서 사용
~/.claude/
└── skills/
    └── project-check/
        └── SKILL.md
```

특정 프로젝트에서만 사용할 경우 프로젝트 경로 하위의 .claude/ 하위에 skill 생성

사용자(User) 폴더 하위의 .claude/ 하위에 skill 생성
윈도우의 경우 C:₩Users₩사용자ID₩.claude/

➔ 4.2 project-check skill 예시

```
---
Name : project-check
Description : 변경 작업을 완료하기 전에 프로젝트 규칙, 오류, 테스트 여부를 점검함
---

# Project Check
변경사항을 완료하기 전에 다음 순서로 확인함

1. 요청한 기능이 실제로 반영되었는지 확인함
2. 관련 테스트를 실행함
3. 린트와 빌드 오류를 확인함
4. 보안 또는 개인정보 노출 위험을 확인함
5. 확인한 내용과 남은 위험을 요약하여 제시함
```

Skill은 Claude가 설명(Description)을 보고 필요할 때 사용할 수 있고
/project-check 처럼 직접 호출하여 사용할 수 있음

개발자 작업 시 프로젝트 배포 전 사전 점검하는 skill

별첨 #3. CLAUDE Code 예시 (4/6)

5. Agent 만들기

➔ 5.1 검토 기준과 출력 방식 정의 필요

```
# 파일 위치
your-project/
└── .claude/
    ├── agents/
    │   └── code-reviewer.md
```

특정 프로젝트에서만 사용할 경우 프로젝트 경로 하위의 .claude/ 하위에 Agent 생성

➔ 4.2 project-check skill 예시

Name : code-reviewer
Description : 코드 변경사항을 기능, 보안, 복잡성, 테스트 관점에서 검토
Tools : Read, Grep, Glob, Bash
Model : inherit

Inherit 모델은 하위(서브) 에이전트가 상위(부모) 에이전트나 메인 대화 세션에서 사용 중인 대규모 언어 모델(LLM)을 그대로 물려받아 사용하도록 지정하는 설정

Code Reviewer
직접 코드를 수정하지 않고 변경사항을 검토

검토 기준
1. 요청한 기능과 구현이 일치하는가
2. 기존 구조와 스타일을 따르는가
3. 불필요한 복잡성이나 중복이 있는가
4. 비밀키, 개인정보, 권한 관련 위험이 있는가
5. 테스트가 핵심 동작을 검증하는가

별첨 #3. CLAUDE Code 예시 (5/6)

6. Hook 연결하기

➔ 6.1 Hook은 Claude Code의 특정 시점에 자동으로 실행됨 (초기에는 간단한 것부터 시작)

.claude/settings.json 예시:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "npm run format"
          }
        ]
      }
    ]
  }
}
```

특정 프로젝트에서만 사용할 경우 프로젝트 경로 하위의 .claude/ 하위에 Hook 생성

format 명령을 사용하기 위해서는 포맷터(Prettier) 설치를 먼저해야 함
"npm install –save-dev prettier"

➔ **자동 코드 정리: 들여쓰기(스페이스/탭), 세미콜론(;) 유무, 따옴표(' vs ") 등 일관되지 않은 코드 스타일을 정해진 규칙에 따라 자동으로 보기 좋게 수정**

❖ Claude Code는 PreToolUse, PostToolUse, SessionStart, Stop, Notification 등 여러 Hook 이벤트를 지원함

별첨 #3. CLAUDE Code 예시 (6/6)

7. 로컬 설정 및 민감 정보 보호하기

➔ 7.1 gitignore에 다음 항목 추가

```
# .gitignore 예시 :
.env
.env.local
.env.production
CLAUDE.local.md
.claude/settings.local.json
node_modules/
.next/
.DS_Store
```

Git 업로드시 제외 파일
CLAUDE.local.md 파일과 .claude/settings.local.json 은 Git에 업로드하지 않는다.

```
# CLAUDE.local.md 예시

## 내 로컬 환경
- 개발 서버: http://localhost:3000
- 로컬 DB 포트: 54321
- 테스트 브라우저: Chrome

## 개인 작업 방식
- 디자인 수정 전 `/components` 폴더부터 확인한다.
- 실제 배포는 내가 직접 진행한다.
- 데이터베이스 변경이 필요하면 먼저 나에게 확인한다.
```

```
# .claude/settings.json 예시:
{
  "permissions": {
    "deny": [
      "Read(./.env)",
      "Read(./.env.*)",
      "Read(./secrets/**)"
    ]
  }
}
```

CLAUDE.local.md에도 실제 비밀번호나 API 카를 적지 말고 환경 변수에 담을 것

.claude/settings.json에
환경 파일(.env)도 Claude가 읽지 못하게 Permissions 권한 규칙으로 제한함

3. AI를 활용한 시스템 구축

가. AI 주도 시스템 구축 (1/2)

AIDx (AI-Driven execution)

소프트웨어 생명주기 6단계 전반에서 AI가 활용됩니다



→ 단계가 진행될수록 산출물이 코드·인프라·운영 데이터로 구체화됩니다.

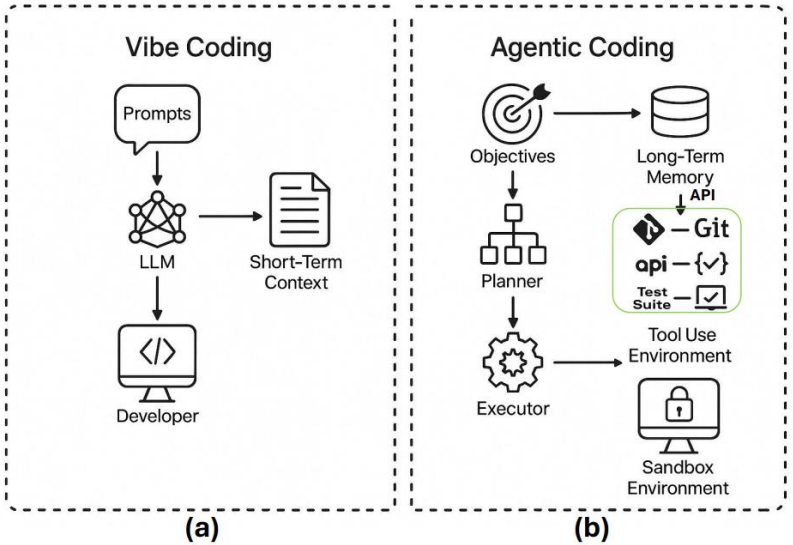
3. AI를 활용한 시스템 구축

가. AI 주도 시스템 구축 (2/2)

AI를 활용한 소프트웨어 생명주기(SDLC) — 단계별 활용 · 도구 · 효과		
단계	**주요 활용 방법**	**생산성·효과**
1. 분석 (요구사항)	• 요구사항 구조화 및 정리 • **유스케이스·사용자 스토리 도출** • 회의록·인터뷰 자동 요약 • **요구사항 충돌·누락 점검** • 도메인 용어 정의	산출물 초안 작성 시간 단축. 모호함·누락을 조기 발견해 후공정 재작업 감소.
2. 설계 (아키텍처·상세)	• 아키텍처 대안 비교·검토 • **다이어그램 생성(시퀀스·ERD·클래스)** • **API 명세 초안 작성** • **데이터 모델링** • 디자인 패턴·트레이드오프 검토	설계 대안의 빠른 탐색, 문서화 자동화, 표준의 일관된 적용.
3. 개발 (구현)	• **레거시 코드 분석** • **코드 자동완성·생성** • **리팩터링 지원** • 코드 설명·주석 생성	반복·패턴 작업 속도 대폭 향상. 단, 환각·보안 취약점 검토 필수.
4. 테스트 (QA)	• **단위·통합 테스트 케이스 생성** • 테스트 데이터·엣지케이스 도출 • **E2E 시나리오 작성 및 커버리지 보강** • **테스트 자동화** • 버그 재현·원인 분석	테스트 작성 부담 경감, 커버리지 향상, 회귀 위험 조기 포착.
5. 배포 (CI/CD·릴리스)	• **CI/CD 파이프라인 스크립트(YAML) 생성** • IaC(Terraform·K8s) 작성·검토 • **릴리스 노트 자동 생성** • **변경 영향 분석** • 배포 스크립트 디버깅	파이프라인 구성 시간 단축, 설정 오류 감소, 문서화 자동화.
6. 운영 (모니터링·유지보수)	• **로그·메트릭 이상 탐지·요약** • **인시던트 원인 분석(RCA)** • 알람 노이즈 감소 • 런북 자동화 • **사용자 문의 대응(챗봇)**	장애 평균복구시간(MTTR) 단축, 운영 부하 경감, 선제적 장애 예방.

3. AI를 활용한 시스템 구축

나. 바이브 코딩 vs Agent 코딩



항목	(a) 바이브 코딩	(b) 에이전트 코딩
핵심 개념	• 인간 중심, 직관적 상호작용	• 자율성 중심, 목표 기반 수행
주요 한계	• 개발자의 지속 개입 필요 • 프롬프트 결과의 비일관성 (품질 편차)	• 의사 결정 과정의 불투명성 • 보안 및 통제 이슈
관리 과제	• 협업 기준 부재 • 추적 불가 및 프롬프트 관리 체계 필요	• 설명 가능한 AI(XAI) 필요 • fail-safe 체계 구축 필요
적용 한계	• 대규모 프로젝트 확장에 어려움	• 초기 프로토타입 등 창의적 작업에 부적합
개선 방향	• 전략적 지침 명확화 • 프롬프트 재현성 확보	• 투명한 로깅 및 추론 구조 강화 • 감독자 개입 지점 설계

3. AI를 활용한 시스템 구축

다. 바이브 코딩 · 디자인 · 테스팅 비교 (1/3)

‘바이브(Vibe)’란?

핵심 — ‘어떻게 만들지(How)’가 아니라 ‘무엇을 원하는지(What·의도)’를 자연어로 전달하면, AI가 실행한다.

① 의도 우선 입력

결과물을 직접 만들지 않고,
원하는 바를 자연어·이미지로 묘사

② AI가 실행

AI가 코드·디자인·테스트 등 실제 산출물을 생성·반복

③ 사람이 판단·정제

사람은 루프 안에서 방향을 정하고
검토·승인(크리에이티브 디렉터)

유래 — 2025년 2월 Andrej Karpathy가 ‘vibe coding’을 명명(“코드가 존재한다는 것조차 잊을 만큼 AI에 맡긴다”). 이후 같은 원리가 디자인·테스팅 등으로 확장되었습니다.
사람의 역할은 ‘직접 제작’에서 ‘의도 전달·검토·판단’으로 이동합니다.

3. AI를 활용한 시스템 구축

다. 바이브 코딩 · 디자인 · 테스팅 비교 (3/3)

타입별 프롬프트 예시

바이브 코딩

예시 프롬프트

"Next.js 14 + Tailwind 앱에 다크모드 토글을 추가해줘.
시니어 React 개발자처럼, TypeScript로 외부 라이브러리 없이.
설정은 localStorage에 저장하고 전환은 부드럽게.
사용 예시와 ESC 키 처리도 포함."

포인트
맥락+역할+작업+제약+검증을 한 번에 담음

바이브 디자인

예시 프롬프트

"핀테크 앱의 모바일 체크아웃 화면을 디자인해줘.
깔끔하고 신뢰감 있는 톤,
넉넉한 여백, 미니멀 타이포.
결제 버튼은 눈에 띄게.
색상 팔레트 3가지 버전으로 보여줘.
"

포인트
톤·구조·변형 요청 → 이후 '버튼 키워' 식 대화로 수정

바이브 테스팅

예시 프롬프트

"체크아웃 과정을 테스트해줘:
① 정상 결제
② 빈 장바구니 결제 차단
③ 잘못된 카드 거부.
각 경우 사용자 메시지가 명확한지 확인."

포인트
시나리오를 평문으로, 엣지케이스를 함께 지정

3. AI를 활용한 시스템 구축

라. AIDD 3대 방법론 비교_핵심 비교표

AI 주도 개발(AI-Driven Development)에서 자주 쓰이는 세 가지 대표 방법론의 핵심 차이

SDD
Spec-Driven Development

명세(Spec)를 단일 진실 공급원으로 삼아 AI가 계획·작업·코드를 생성하는 개발 방식

"무엇을·왜"는 사람이, "어떻게"는 AI가

BMAD
Breakthrough Method for Agile AI-Driven Development

애자일 팀의 역할(분석가·PM·아키텍트·SM·개발자·QA)을 전문 AI 에이전트로 구현한 프레임워크

AI 에이전트 팀 + 사람 의사결정자

AI-DLC
AI-Driven Development Lifecycle

AWS가 제안한 AI 중심 개발 생명주기. AI가 계획·실행을 주도하고 사람은 검증·의사결정

SDLC를 AI 기준으로 재설계

3. AI를 활용한 시스템 구축

라. AIDD 3대 방법론 비교_핵심 비교표

구분	SDD	BMAD	AI-DLC
특징	**명세 기반 개발 기법(Practice)**	에이전트 기반 프레임워크	**조직 수준 개발 생명주기**
핵심 아이디어	명세가 코드의 원천	**애자일 역할을 AI 에이전트 팀으로 구성**	AI 실행 + 사람 검증 · 감독으로 SDLC 재설계
프로세스	Specify → Plan → Tasks → Implement	기획(Planning) → 스토리 개발 사이클	Inception → Construction → Operations
중심 산출물	constitution · spec · plan · tasks	Brief · PRD · Architecture · Story	Requirements · Stories · Units · Design (aidlc-docs)
작업 단위	기능(Feature) 명세	에픽 → 스토리	Intent → Unit of Work → Bolt (시간~일 단위)
AI의 역할	명세 구조화, 계획·작업 분해, 코드 생성	역할별 페르소나로 기획~QA 수행	계획 수립, 명확화 질문, 설계·코드·인프라 생성
사람의 역할	의도 정의, 단계별 검토·승인	에이전트 산출물 승인, 핵심 의사결정	Intent 제시, Mob 세션 검증, 최종 결정
적합 대상	개인~팀, 기능 단위 개발	1인~소규모 팀, 신규 제품 기획·개발	팀·조직 단위의 AI 전환
도입 난이도	낮음~중간	중간~높음	높음 (프로세스·문화 변화)
대표 도구	Spec Kit, Kiro, Tessl, OpenSpec	BMAD-METHOD (오픈소스)	AWS AI-DLC 워크플로

3. AI를 활용한 시스템 구축

라. AIDD 3대 방법론 비교_SDD (1/5)

SDD 프로세스



핵심 원칙: 스펙이 단일 진실 공급원(Single Source of Truth)
요구가 바뀌면 코드가 아니라 스펙을 고치고, 계획·작업·코드를 다시 생성합니다. 각 단계 사이에 사람이 산출물을 검토합니다.

- ❖ Spec-first → Spec-anchored(명세 지속 유지) → Spec-as-source(명세만 편집)
- ❖ **단계별 체크 포인트** – 단계마다 사람이 검토한 뒤 다음 단계로 진행

3. AI를 활용한 시스템 구축

라. AIDD 3대 방법론 비교_SDD (2/5)

SDD 절차 ① 정의 단계 (Spec Kit 기준)

	단계 · 산출물	AI 역할	사람 역할
Constituti 	1 **원칙 수립** 협업 Constitution · constitution.md	프로젝트 원칙·코딩 표준·테스트 정책 초안 작성	아키텍처 원칙과 품질 기준을 결정하고 승인
Specify 	2 **명세 작성** 사람 주도 Specify · spec.md	자연어 의도를 사용자 스토리·기능 요구·수용 기준으로 구조화	무엇을, 왜 만드는지 비즈니스 의도 제시 (기술 결정은 배제)
	3 **명확화** 협업 Clarify · 명세 보완 Q&A	모호하거나 누락된 요구사항을 질문 형태로 도출	질문에 답변하고 범위·우선순위·예외 상황 확정
Plan 	4 **기술 계획** 협업 Plan · plan.md, data-model	기술 스택을 반영한 아키텍처·데이터 모델·인터페이스 설계	기술 스택·제약 조건 제시, 설계안 검토·승인

3. AI를 활용한 시스템 구축

라. AIDD 3대 방법론 비교_SDD (3/5)

SDD 절차 ② 실행 단계 (Spec Kit 기준)

	단계 · 산출물	AI 역할	사람 역할
Tasks	5 **작업 분해** AI 주도 Tasks · tasks.md	의존성과 병렬 가능성을 고려해 코드·테스트 가능한 작업 목록 생성	작업 단위의 크기·순서 적정성 검토
	6 **일관성 분석** AI 주도 Analyze · 교차 점검 리포트	명세·계획·작업 간 불일치, 누락, 원칙 위반을 교차 점검	지적 사항의 중요도 판단, 수정 지시
Implement	7 **구현** AI 주도 Implement · 코드, 테스트	작업 단위로 코드와 테스트 생성 (테스트 우선 적용)	체크포인트별 코드 리뷰와 승인, 방향 수정
	8 **검증·진화** 사람 주도 수용 테스트 결과 · 갱신된 명세	명세 대비 테스트 실행, 변경 시 명세 갱신안 제안	수용 테스트 수행. 기능 변경은 반드시 명세부터 수정

※ Kiro는 requirements.md → design.md → tasks.md 3단계로 동일한 흐름을 구현

3. AI를 활용한 시스템 구축

라. AIDD 3대 방법론 비교_SDD (4/5)

SDD 장점 · 단점 · 보완 방안

장점	단점	보완 방안
• 바이브 코딩의 모호함 · 재작업 감소 • 의도 → 명세 → 코드로 이어지는 추적성 확보 • AI 출력의 일관성 · 예측 가능성 향상 • 리뷰 대상이 코드에서 명세로 이동해 검토 효율 증가 • 문서가 개발 과정의 부산물로 자동 최신화 • 특정 AI 에이전트에 종속되지 않음	작은 변경에도 명세 작성 부담이 과함	작업 규모별 경량 모드 적용 (간단 수정은 명세 생략, 변경 단위 명세)
	시간이 지나며 명세와 코드가 어긋남 (드리프트)	"명세 먼저 수정" 규칙, CI에서 정합성 점검 자동화
	AI가 명세를 무시하거나 과잉 해석	작업을 잘게 쪼개고 수용 기준을 테스트로 고정
	생성되는 마크다운 문서가 많아 리뷰 피로	템플릿 간소화, 수용 기준·결정 사항 위주로 리뷰
	기존(브라운필드) 시스템 적용이 어려움	기존 코드를 역공학해 기준 명세 생성 후 점진 적용

3. AI를 활용한 시스템 구축

라. AIDD 3대 방법론 비교_SDD (5/5)

SDD 주요 프레임워크



GitHub Spec Kit

GitHub · 오픈소스 CLI

- /specify · /plan · /tasks · /implement 등 슬래시 명령
- Copilot, Claude Code, Gemini CLI 등 다수 에이전트 지원
- constitution으로 프로젝트 불변 원칙 관리



Kiro

AWS · Spec 기반 AI IDE

- requirements.md(EARS 표기) → design.md → tasks.md
- Hooks(이벤트 자동화), Steering(프로젝트 규칙) 제공
- IDE 안에서 명세·작업 진행 상태를 시각적으로 관리



Tessl

Tessl · Spec-as-source 지향

- 명세를 주 편집 대상으로, 코드는 생성물로 취급
- 라이브러리 사용법 명세를 모은 Spec Registry
- 가장 급진적인 SDD 성숙도 지향



OpenSpec

오픈소스 · 경량 SDD

- 변경 제안(proposal) → 적용(apply) → 아카이브 흐름
- 기존 시스템(브라운필드)의 변경 관리에 강점
- 가벼운 폴더 구조로 빠르게 도입 가능

3. AI를 활용한 시스템 구축

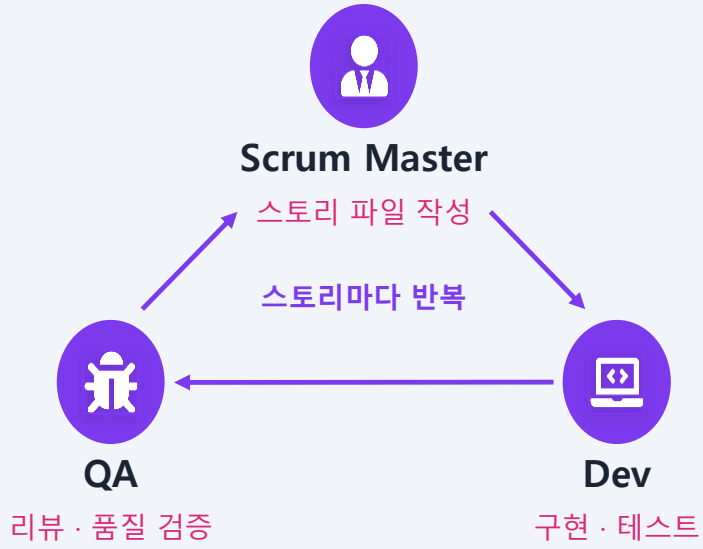
라. AIDD 3대 방법론 비교_BMAD (1/5)

BMAD 프로세스

① 에이전틱 기획 (Agentic Planning)

- **Analyst** — 아이디어 조사 → 프로젝트 브리프
- **PM** — 요구사항 정리 → PRD · 에픽 정의
- **Architect** — 기술 설계 → 아키텍처 문서
- **PO** — 문서 정합성 검증 · 업무 분해

② 컨텍스트 엔지니어링 개발



규모별 트랙 (v6): Quick Flow (소규모 수정) · BMad Method (제품 개발) · Enterprise (대규모·규제 환경)

3. AI를 활용한 시스템 구축

라. AIDD 3대 방법론 비교_BMAD (2/5)

BMAD 절차 ① 기획 단계 (Planning)

단계 · 산출물	AI 역할	사람 역할
1 **아이디어 분석** 협업 Analyst · project-brief.md	브레인스토밍 진행, 시장·경쟁 조사, 프로젝트 브리프 초안	아이디어·목표 제시, 브리프 내용 확정
2 **요구사항 정의** 협업 PM · PRD.md (FR/NFR, 에픽)	브리프 기반 PRD 작성, 에픽·스토리 구조화	기능 범위와 우선순위 결정, PRD 승인
3 **UX 설계** 협업 UX Expert(선택) · UI 명세	화면 흐름·UI 명세, UI 생성 도구용 프롬프트 작성	사용자 경험 방향 결정, 디자인 검토
4 **아키텍처 설계** 협업 Architect · architecture.md	기술 스택, 시스템 구조, 코딩 표준·소스 트리 설계	기술 선택·제약 결정, 아키텍처 승인
5 **정합성 검증** 사람 주도 PO · 마스터 체크리스트 결과	체크리스트로 PRD와 아키텍처의 정합성·누락 점검	미흡 항목 보완 지시, 기획 산출물 최종 확정
6 **문서 샤딩** AI 주도 PO · 샤딩된 문서 파일	대형 문서(PRD, 아키텍처 설계문서)를 에픽 단위 파일로 분할	분할 결과 확인 후 IDE 개발 단계로 전환

※ 기획 단계는 웹 UI(팀 번들)에서 진행 후 산출물을 IDE 프로젝트로 옮기는 방식도 지원

3. AI를 활용한 시스템 구축

라. AIDD 3대 방법론 비교_BMAD (3/5)

BMAD 절차 ② 개발 사이클 (스토리 단위 반복)

단계 · 산출물	AI 역할	사람 역할
7 **스토리 작성** AI 주도 SM · story-x.y.md	샤딩 문서를 바탕으로 구현 맥락이 담긴 상세 스토리 생성	스토리 검토 후 Draft → Approved 승인
8 **테스트 설계** 협업 QA (선택) · 테스트 전략	고위험 영역 식별, 테스트 시나리오 설계	위험 수용 여부와 테스트 범위 결정
9 **구현** AI 주도 Dev · 코드, 테스트	스토리 태스크를 순차 구현하고 테스트 작성·실행	진행 모니터링, 막히는 지점 의사결정
10 **품질 게이트** 협업 QA · 리뷰 결과, 게이트 판정	코드 리뷰·리팩터링, 요구사항 추적, PASS/CONCERNS/FAIL 판정	게이트 결과 검토, 수정 또는 수용 결정
11 **완료·반복** 사람 주도 커밋 · Done · 다음 스토리	변경 요약, 관련 문서 갱신	최종 승인과 커밋, 다음 스토리 착수 지시

3. AI를 활용한 시스템 구축

라. AIDD 3대 방법론 비교_BMAD (4/5)

BMAD 장점 · 단점 · 보완 방안

장점	단점	보완 방안
• 익숙한 애자일 역할 체계라 이해하기 쉬움 • PRD·아키텍처 등 기획 산출물이 풍부하게 생성 • 맥락 내장 스토리로 개발 에이전트의 일관성 확보 • QA 게이트로 품질 관리 체계 내장 • 오픈소스, 다양한 IDE·에이전트에서 사용 가능	에이전트·명령·워크플로가 많아 학습 곡선이 높음	PM·Architect·SM·Dev 핵심 에이전트로 시작해 단계적 확대
	소규모 작업에는 절차와 문서가 과도함	경량 트랙(빠른 흐름) 등 규모별 트랙 선택
	다수 문서 로드로 토큰·비용 증가	문서 샤딩(업무 분해), 필요한 문서만 로드, 기획은 저비용 모델 활용
	에이전트 간 인계 과정에서 품질 편차 발생	PO 체크리스트·QA 게이트 의무화, 사람 승인 지점 명시
	1인 개발자 중심 설계로 팀 협업 방식이 불명확	역할별 담당자 지정, 산출물을 Git PR로 리뷰

3. AI를 활용한 시스템 구축

라. AIDD 3대 방법론 비교_BMAD (5/5)

BMAD 주요 프레임워크 · 도구

BMAD-METHOD

오픈소스 (GitHub) · npx 설치

- 에이전트·태스크·템플릿·체크리스트·워크플로 묶음
- 프로젝트에 설치해 슬래시 명령/에이전트 호출로 사용
- 버전별로 구조가 다르므로 최신 문서 확인 필요

지원 IDE · 코딩 에이전트

도구 독립적

- Claude Code, Cursor, Windsurf, GitHub Copilot 등
- 에이전트 정의를 각 도구의 규칙·명령 형식으로 설치
- 개발 사이클(SM→Dev→QA)은 주로 IDE에서 수행

웹 팀 번들

기획 단계용

- 에이전트 묶음을 웹 챗(Gemini Gem, Custom GPT 등)에 업로드
- 대용량 컨텍스트로 브리프·PRD·아키텍처를 저비용 작성
- 산출물을 docs 폴더로 옮겨 IDE 단계와 연결

확장팩 · 모듈

도메인 확장

- 게임 개발, 크리에이티브, DevOps 등 도메인 특화 팩
- 커스텀 에이전트·워크플로 제작 도구 제공
- 소프트웨어 외 분야에도 동일한 방법론 적용

3. AI를 활용한 시스템 구축

라. AIDD 3대 방법론 비교_AI-DLC (1/5)

AI-DLC 프로세스

Inception
기획 단계

Mob Elaboration

현행 분석 및 비즈니스 의도
→ 요구사항 · 사용자 스토리
→ 워크플로 계획
→ Unit of Work 분해

→

Construction
구축 단계

Mob Construction

기능 설계/NFR 설계/인프라 설계
→ 코드 생성
→ 빌드 · 테스트

→

Operations
운영 단계

Deploy & Observe

IaC 기반 배포
→ 모니터링
→ 운영 이슈 대응 지원

모든 단계에서 반복되는 패턴

AI 계획 수립 → AI 질문·명확화 → 사람 검증·승인 → AI 실행

3. AI를 활용한 시스템 구축

라. AIDD 3대 방법론 비교_AI-DLC (2/5)

AI-DLC 절차 ① Inception (Mob Elaboration)

단계 · 산출물	AI 역할	사람 역할
1 **현황 분석** AI 주도 Detection · Reverse Eng.	신규/기존 프로젝트 판별, 기존 코드 구조·의존성 역공학 문서화	분석 결과의 정확성 검증 (기존 시스템인 경우)
2 **요구사항 분석** 협업 requirements.md	Intent를 요구사항으로 구체화하고 명확화 질문 생성	비즈니스 의도 제시, 질문 답변 후 요구사항 승인
3 **사용자 스토리** 협업 stories.md · personas.md	페르소나, 사용자 스토리, 수용 기준 작성	팀 전체(Mob)가 함께 검토·조정·합의
4 **워크플로 계획** 협업 execution-plan.md	프로젝트 성격에 맞춰 수행할 단계를 선정 (적응형)	단계 생략·추가 여부 결정, 실행 계획 승인
5 **Unit 분해** 협업 Application Design · Units	컴포넌트 설계, 독립 개발 가능한 Unit of Work로 분해	Unit 경계와 우선순위 확정, 팀별 배분

※ 단계 구성은 AWS 공개 워크플로(aidlc-workflows) 기준이며, 프로젝트 복잡도에 따라 일부 단계는 생략됨

3. AI를 활용한 시스템 구축

라. AIDD 3대 방법론 비교_AI-DLC (3/5)

AI-DLC 절차 ② Construction · Operations (Bolt 단위)

단계 · 산출물	AI 역할	사람 역할
6 **기능 설계** 협업 Functional / Domain Design	도메인 주도 설계(DDD)로 도메인 모델·비즈니스 로직 설계	비즈니스 규칙의 정확성 검증
7 **NFR 설계** 협업 NFR Requirements · Design	성능·보안·확장성 요구 도출, 적용 패턴 제안	품질 속성 간 트레이드오프 결정
8 **인프라 설계** 협업 Infrastructure Design (IaC)	클라우드 서비스 매핑, 배포 아키텍처·IaC 설계	비용·보안·운영 정책 결정
9 **코드 생성** AI 주도 코드 · 테스트	코드 생성 계획 제시 → 승인 후 코드·테스트 생성	생성 계획 승인, 코드 리뷰·기술 결정
10 **빌드·테스트** AI 주도 빌드·테스트 결과	빌드 및 단위·통합 테스트 지침 작성·실행	결과 확인, 다음 Bolt 또는 배포 승인
11 **운영** 사람 주도 Operations · 배포·모니터링	IaC 기반 배포, 모니터링·이상 징후 대응안 제시	운영 감독과 장애 대응 의사결정

3. AI를 활용한 시스템 구축

라. AIDD 3대 방법론 비교_AI-DLC (4/5)

AI-DLC 장점 · 단점 · 보완 방안

장점

- AI 속도를 활용하면서 사람의 통제권 유지
- Mob 세션으로 팀 합의와 의사결정이 빠름
- Bolt(짧은 주기)로 피드백 루프 단축
- 도메인·NFR·인프라까지 생명주기 전반을 포괄
- 모든 결정·산출물이 기록되어 추적성·감사 용이

단점	보완 방안
역할·회의 방식 등 조직 변화 폭이 큼	파일럿 팀으로 시작해 기존 애자일과 병행 운영
Mob 세션에 인원·시간이 집중됨	세션 타임박스, 핵심 의사결정자만 참여, 비동기 리뷰 병행
AWS 도구·생태계 중심의 사례가 많음	규칙 세트는 도구 중립적으로 활용, 인프라 단계는 자사 환경에 맞춤
비교적 신생이라 레퍼런스·성숙도가 부족	사내 플레이북 축적, 단계별 품질·속도 지표 측정
운영(Operations) 단계 구체화가 미흡	기존 DevOps·SRE 체계와 연계해 보완

3. AI를 활용한 시스템 구축

라. AIDD 3대 방법론 비교_AI-DLC (5/5)

AI-DLC 주요 프레임워크 · 도구



AI-DLC Workflows

AWS Labs · 오픈소스 규칙 세트

- Inception·Construction 단계별 규칙과 질문 템플릿
- 산출물을 aidlc-docs/ 폴더에 기록해 결정 이력 관리
- 프로젝트에 규칙 파일을 넣어 에이전트가 따르도록 구성



Amazon Q Developer

AWS · AI 코딩 어시스턴트

- 프로젝트 규칙으로 AI-DLC 워크플로 적용
- IDE·CLI에서 단계별 질문·승인 흐름 진행
- AWS 서비스와의 인프라 연계에 강점



Kiro

AWS · Spec 기반 AI IDE

- Steering 규칙으로 AI-DLC 워크플로 적용
- Spec 기능과 결합해 Unit별 구현 관리
- SDD와 AI-DLC를 함께 쓰는 접점



기타 코딩 에이전트

규칙 파일 기반 적용

- 규칙(마크다운) 기반이라 다른 에이전트에도 이식 가능
- Claude Code, Cursor 등 규칙 파일 지원 도구 활용
- 도구별 적용 방법은 저장소 문서 확인 필요

3. AI를 활용한 시스템 구축

라. AIDD 3대 방법론 비교_상황별 선택 가이드

SDD가 적합한 경우

- 기능 단위로 정확하게 구현하고 싶을 때
- 현재 팀 프로세스를 크게 바꾸지 않고 시작할 때
- AI 출력의 일관성 확보가 최우선일 때

BMAD가 적합한 경우

- 1인·소규모 팀이 아이디어부터 제품까지 만들 때
- PRD·아키텍처 등 기획 문서가 필요할 때
- 역할 기반으로 AI 작업을 분업하고 싶을 때

AI-DLC가 적합한 경우

- 팀·조직 전체의 개발 방식을 AI 중심으로 전환할 때
- 감사 추적·거버넌스가 중요한 엔터프라이즈
- 클라우드 인프라까지 포함한 전 주기 개발

결합 전략: 세 방법론은 배타적이지 않음

AI-DLC로 조직 프로세스의 큰 틀(Inception → Construction → Operations)을 잡고, **초기 기획은 BMAD 에이전트(Analyst·PM·Architect)**로 산출물을 만들며, **각 Unit의 구현은 SDD 도구(Spec Kit·Kiro)**로 명세 기반 진행하는 하이브리드 구성이 가능

공통 성공 원칙

어떤 방법론이든 성패는 "사람의 승인 지점"을 얼마나 명확히 설계하느냐에 달려 있음

승인 게이트 명시
단계마다 누가 무엇을 승인하는지 정의

산출물 Git 관리
명세·계획·결정을 코드처럼 리뷰

작은 작업 단위
AI 오류의 영향 범위 최소화

테스트 자동화
수용 기준을 테스트로 고정

4. 테스트 엔지니어의 역할 변화

Before — 기존

- 테스트 케이스를 수작업으로 설계·작성
- Selenium·Java 등으로 스크립트 직접 코딩
- UI 변경마다 깨진 스크립트 수동 유지보수
- 반복적 회귀 테스트 실행
- 기능 동작(Pass/Fail) 검증에 집중
- 결함 리포팅 중심

After — AI 시대

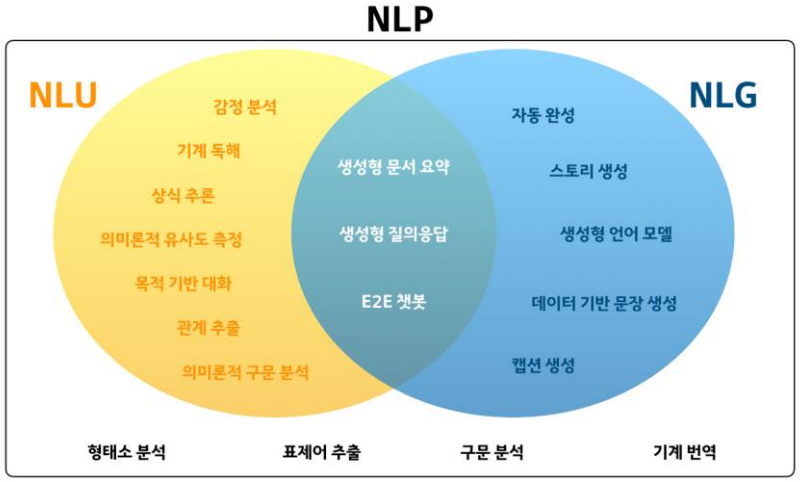
- 자연어로 의도를 기술하고 AI가 생성 → **사람은 검토·정제**
- **AI 출력 검증·큐레이션, 프롬프트(루프,그래프) 설계**
- 자가 치유 테스트 모니터링, **도구·파이프라인 오케스트레이션**
- 리스크 기반 테스트 전략 수립
- 품질 코치·게이트키퍼: 기준·문화 설계
- UX·비즈니스 리스크와 AI·데이터 거버넌스까지 책임

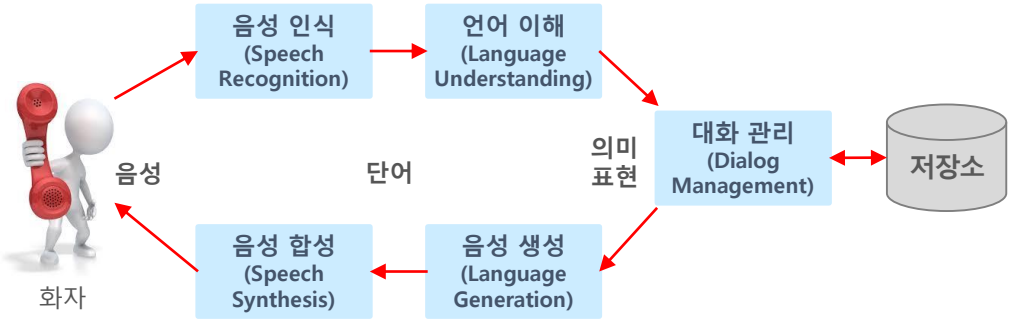
핵심 전환

- **'실행자(스크립트 작성)'에서 '전략가·검증자·오케스트레이터'로.**
- 핵심 역량은 **코딩에서 프롬프트>컨텍스트>하네스>루프>그래프 엔지니어링 관리자**로

별첨 #4. NLP, NLU, NLG

- **NLP(Natural Language Processing) - 자연어 처리**
  - 자연어(natural language)란 우리가 일상 생활에서 사용하는 언어를 말함
  - 자연어 처리(natural language processing)란 이러한 **자연어의 의미를 분석하여 컴퓨터가 처리할 수 있도록 하는 일**
- **NLP = NLU + NLG**





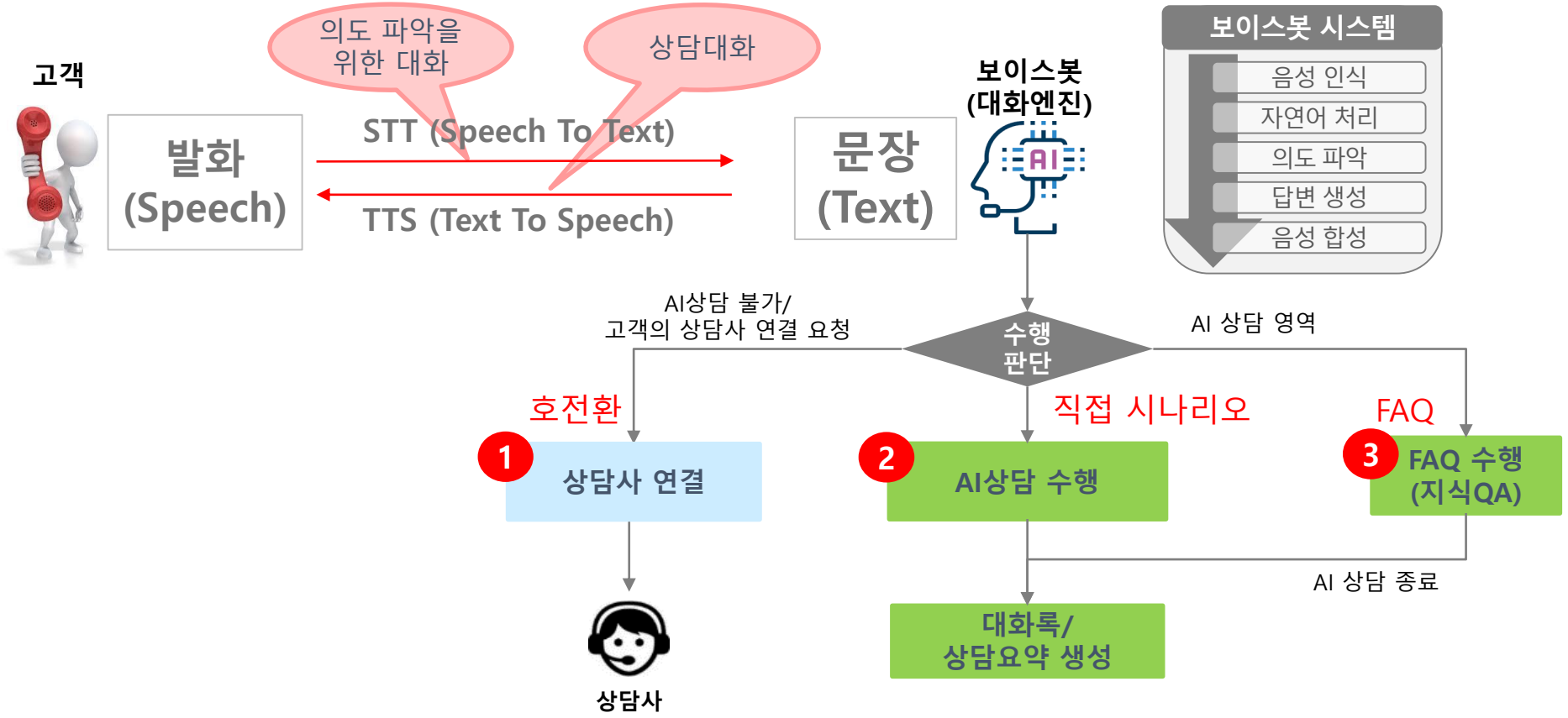
- **NLU(Natural Language Understanding) 자연어 이해** : 자연어 형태의 문장을 이해하는 기술
  사람-기계 상호작용이 필수인 경우 NLU는 핵심 기술
  ex) 구글에서 NLU 기술을 접목해 기존 키워드 매칭 방식과 비교해 더 나은 검색 서비스를 제공함. (BERT 모델)
- **NLG(Natural Language Generation) 자연어 생성** : 자연어 문장을 생성하는 기술
- NLU와 NLG는 사람-기계, 기계-기계 사이 사람이 자연어로 소통하는 데 필요한 기술

5. AICC 사례 > 가. AICC 보이스봇 개요

1). AICC 보이스봇 서비스 수행업무

AICC(AI Contact Center) 보이스봇 서비스 수행 업무

- AICC 보이스봇은 클라우드 환경에서 음성인식과 대화처리를 통한 AI 전화 상담 서비스
- 홈 상품 100번, 모바일 상품 114번



5. AICC 사례 > 가. AICC 보이스봇 개요

2). AICC 보이스봇 상담과정

고객 ←→ AI 보이스봇 상담 과정

1. 고객의 말 듣고 (청취)



상담 수행

고객

6. 음성으로 답변하고 (발화)

2. 고객의 말을 이해하고
- 상황 이해
- 질문/요청 이해
- 답변 이해

3. 해야할 일을 판단하고
- 처리 불가 업무인지 상담사 전환 처리
- 가상상담 가능 업무라면, 어떤 task를 수행하여야 하는지

AI 보이스봇

대화관리(판단/제어)

이해 → 처리 → 응대

5. 상담에 필요한 대화를 수행하고
- 인사말
- 고객 상황설명/불만에 대한 공감대화
- 고객 문의에 대한 답변
- 업무처리에 필요한 정보수집/재확인
- 업무처리 과정 안내/결과 안내
- 돌발질문에 대한 대응 (주제 이탈 시 대응)

4. Task를 수행하고
- 상담지식 조회하여 답변생성
- 시스템 조회하여 답변생성
- 업무 처리 요청 수행
- Offering 제안하고 수행
- 상담사 연결 요청에 대해 의도에 맞는 상담그룹 연결

- 상담지식조회
- 시스템 조회
- 시스템 통한 업무처리
- Offering 시나리오 수행
- 상담 그룹 연결

STT

TTS

대화

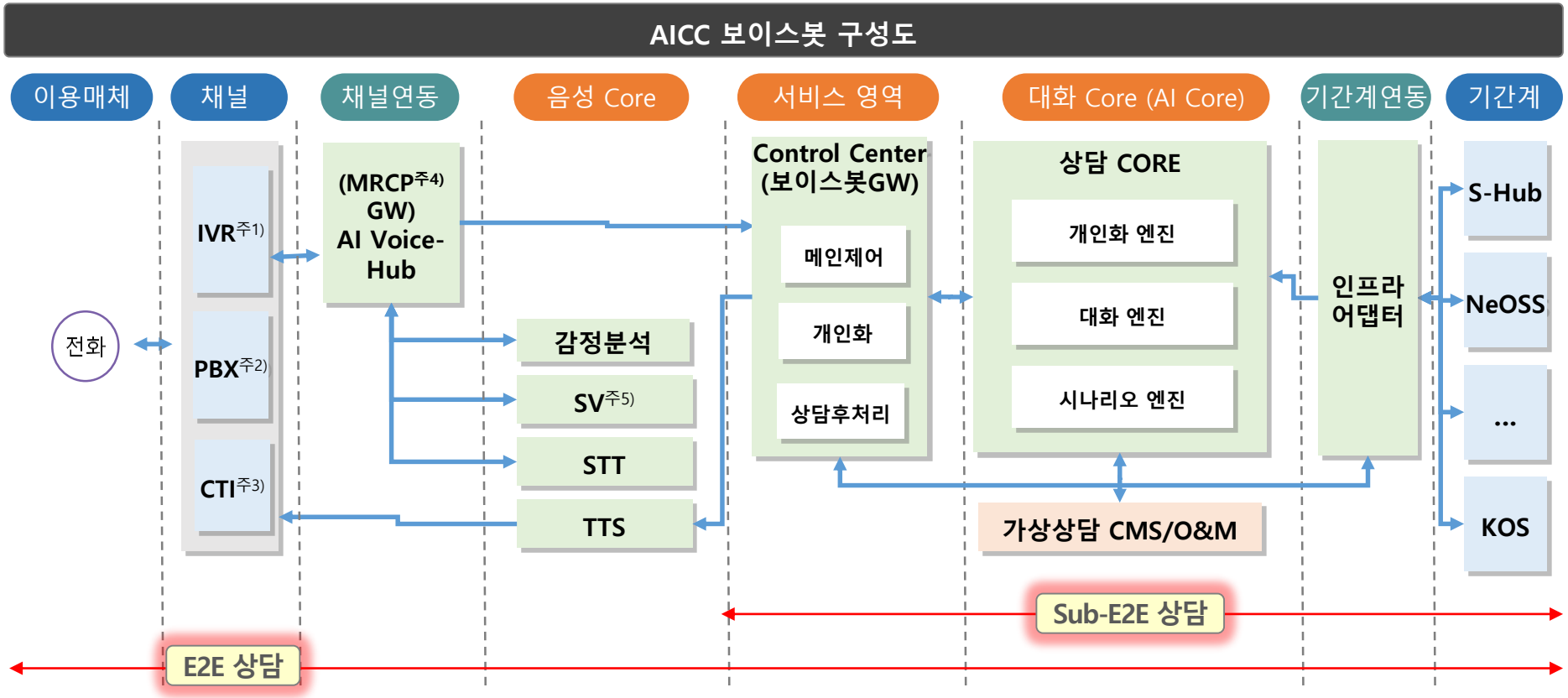
대화이해 | 대화관리(판단/제어) | 상담대화

Task수행

QnA | 시나리오 | 상담사 전환

5. AICC 사례 > 가. AICC 보이스봇 개요

3). AICC 보이스봇 구성도

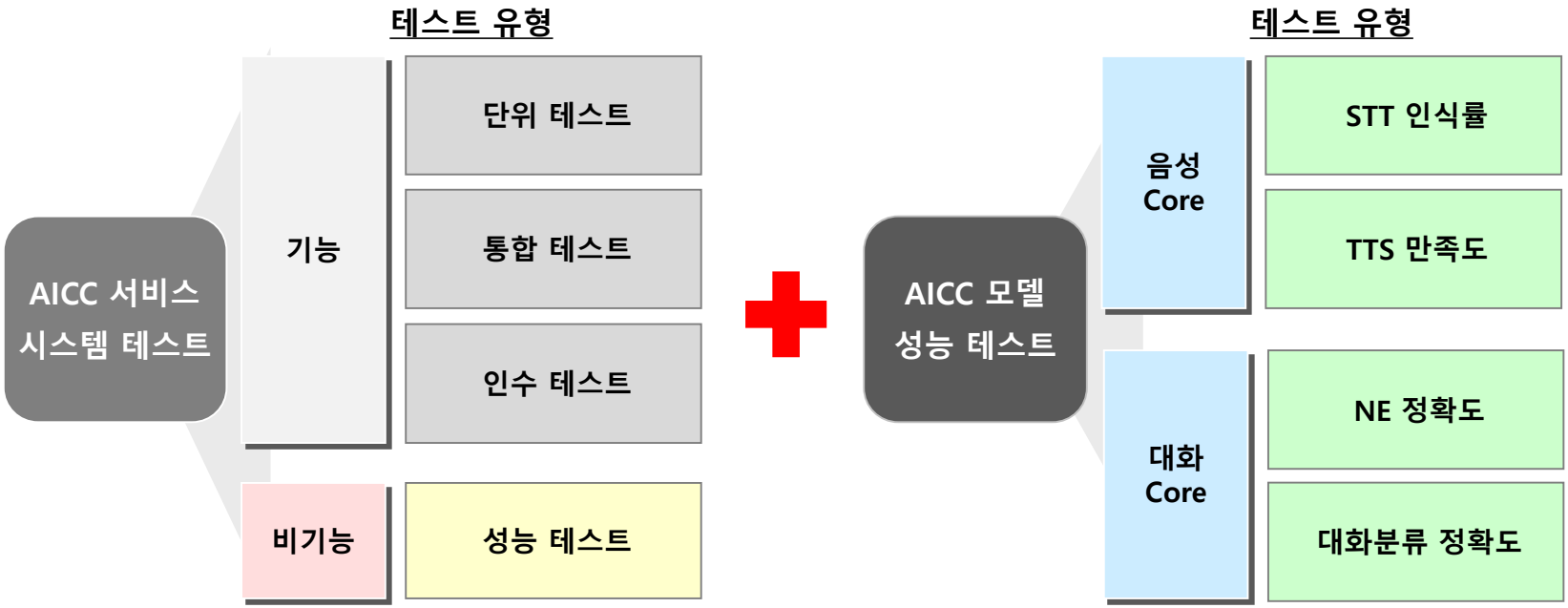


주1) IVR(Interactive Voice Response) : 고객의 기존 정보를 바탕으로 맞춤형 자동 음성안내(ARS)서비스를 제공
주2) IP-PBX(Private Branch Exchange) : 사내 자동 전화교환 시스템 및 통화기능을 제공
주3) CTI(Computer Telephony Integration) : 상담원들에게 고객의 전화를 연결해주고, 고객에 대한 정보를 팝업형태로 제공
주4) MRCP(Media Resource Control Protocol) : 음성서버에서 클라이언트에 다양한 음성 서비스를 제공하는데 사용하는 통신 프로토콜
주5) SV(Speaker Verification) : 목소리 인증, 목소리로 고객 인증

5. AICC 사례 > 나. 테스트 유형

1). AICC 보이스봇 시스템 테스트 유형

AICC (AI Contact Center) 보이스봇 시스템 테스트 유형



5. AICC 사례 > 나. 테스트 유형

2). AICC 서비스 시스템 테스트 유형

AICC (AI Contact Center) 보이스봇 시스템 - 서비스 테스트 유형

	테스트 유형		테스트 방법	테스트 설명
AICC 서비스 시스템 검증	기능	단위 테스트	수동 테스트 (단위 기능)	• 프로그램 유형별(온라인, 배치, 인터페이스) 단위 기능 테스트 • 단위 기능의 입력값 검증, 처리로직 검증, 출력값 검증 (온라인은 이벤트별 처리 기능 검증 및 UI 표준 검증 포함) • 정상/비정상 메시지 검증
		통합 테스트	수동 테스트 (Simulator)	• AI 기능 테스트 – 상담 흐름별 AI 단위 기능검증 • 상담 시나리오 테스트 - 의도분류, 호전환, 직접상당, FAQ 처리 등 다양한 상담 시나리오 기반으로 테스트 수행 • 시뮬레이터를 활용해 테스트 (Sub-E2E*주1))
		인수 테스트	수동 테스트 (실제 상담)	• AI 기능 테스트 – 상담 흐름별 AI 단위 기능검증 • 상담사가 다양한 상담 시나리오 기반 및 Ad-Hoc (상황에 따라 임의의) 시나리오로 테스트 수행 • 지역별 상담 전화번호로 상담사가 직접 통화하여 수행 (E2E)
	비기능	성능 테스트	자동 테스트 (J-Meter)	• AI First 고객센터 Voice Bot에 의해 직접 또는 호전환 처리되는 시나리오(subE2E)를 대상으로 시스템의 트래픽 처리 구간에 대해 성능테스트를 수행 • 주요 상담 시나리오(실시간 요금조회, 대리점 위치문의 등)

* 주1) Sub-E2E : 채널(IVR, ARS, CTI) 및 음성 CORE(SV, STT, TTS) 제외한 Control Center 이후의 시스템 검증

5. AICC 사례

다. AICC 모델 성능 테스트 유형

AICC (AI Contact Center) 보이스봇 시스템 테스트 유형

	테스트 유형	테스트 방법	테스트 설명
AICC 모델 성능 검증	음성 Core	STT 정확도	수동 샘플링 대사	• 보이스봇 고객 발화 녹음 내용을 전사(받아쓰기)하여 STT 결과와 비교 검증하여 평가
		TTS 만족도	수동 MOS 5단계 설문	• 고객센터에서 시나리오 테스트 후 MOS 5단계 설문 조사 및 이상한 TTS 내용을 개발팀에 전달
	대화 Core	NE 정확도	자동 (Bulk Test) 수동 (Simulator)	• 다양한 비표준 어휘를 표준어휘(표제어)로 정상 반환되는지 측정 - 선택지형 표제어에서 선택 요청하여 요청된 선택지 내에서 값이 반환되면 성공 - 직접처리 입력형 표제어 요청하여 대표 표제어로 반환되면 성공
		대화분류 정확도	자동 (Bulk Test)	• 대표발화 + 유사발화 혼합된 Test Set으로 직접처리 의도/호전환 의도 분류 정확도 측정 • Bulk 자동 테스트 - 발화문장을 Text로 일괄 업로드하여 대화 API 응답 로그의 Intent_name 포함 여부로 정답 판단

5. AICC 사례 > 다. AICC 모델 성능 테스트 유형

1). STT 정확도

STT 정확도 - CER(Character Error Rate)

1. **STT 정확도(순수한 받아적기 능력)를 측정하는 정량적 측정 방법으로 산업 표준임**
2. 정확도 측정을 위해서 오디오 + 휴먼 레이블(사람이 듣고 전사한 스크립트)이 기록된 정답 데이터 필요
3. 시스템 출력(인식) 결과와 레이블(전사)을 비교하여 인식률 측정
4. 한글은 띄어쓰기 모호성 때문에 word를 음절(Character)로 분리하여 계산
5. CER 기준 6% 이하

■ 평가 방법

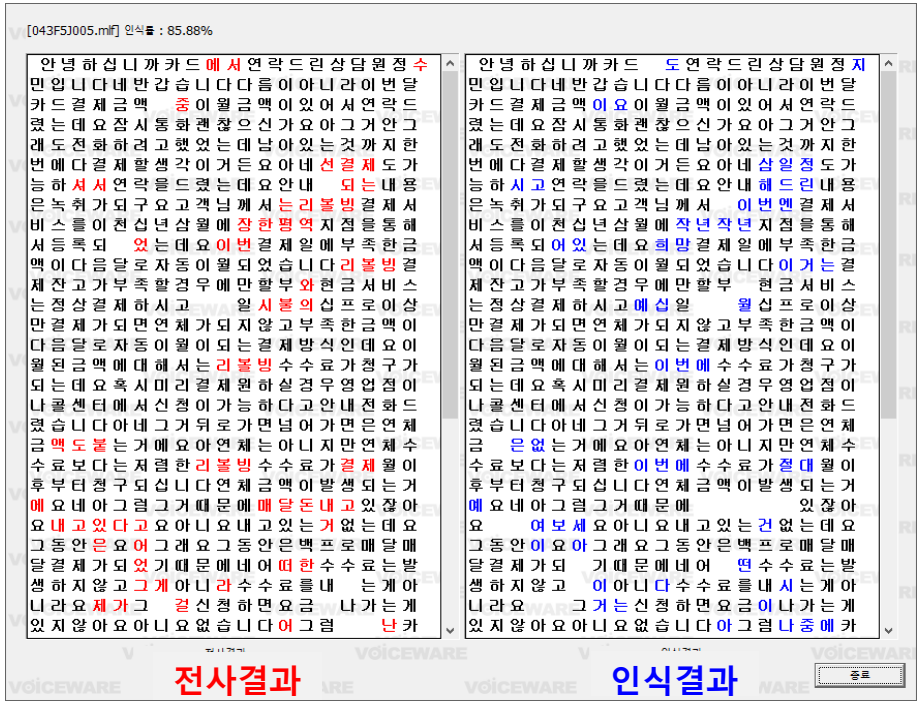
I D S

정답 : 상 담 원 홍 길 동 입 니 다
후보 : 상 담 직 원 길 돈 입 니 다

I : Character가 잘 못 추가된 삽입 에러
D : Character 가 사라진 삭제 에러
S : 다른 Character 로 바뀐 대체 에러
N : 전체 Character 수

$$CER\ (글자\ 오류율) = \frac{I + D + S}{N} * 100\%$$

$$인식률 = 100\% - CER$$



■ 평가 등급

CER(%)	품질 평가
<=5%	매우 우수
5~8%	**우수**
8~12%	보통
>=12%	개선 필요

5. AICC 사례 > 다. AICC 모델 성능 테스트 유형

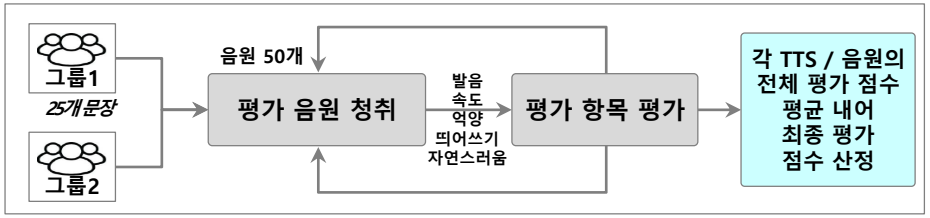
2). TTS 만족도 - MOS

TTS 만족도 - MOS(Mean Opinion Score)

■ 평가 개요

1. **TTS 품질을 측정하는 산업 표준이 되는 5단계 정성적 측정 방법**
2. 상담에 참여한 상담사(테스터) **여러 명이 AI가 만든 음원 청취후 주관적 점수 매긴 것을 종합하여 음성품질 성능 평가**
3. 1~5점까지 점수로 평가하며, 각 TTS 평가 점수를 평균내어 최종 MOS 점수로 산정
4. 성우 음성 점수를 만점으로 TTS 평가 4.0 이상

■ MOS 평가 프로세스



* ACR 테스트 방식
  - 평가자가 여러 실험 자극(음원)을 순서대로 한번만 듣고 평가하는 방식
  - 평가 순서는 무작위로 제시

■ MOS 평가 등급

MOS		시스템 출력		
Level	음질	이해도(속도, 띄어 읽기, 의미전달)	자연스러움(상담 도메인 적합성, 감정, 운율)	총평
5	최상	낮은 빈도의 고유명사도 잘 알아듣게 발음	기계느낌보다 사람이 말하는 것처럼 편안함	**편안한 상태에서 잘 들림**
4	양호	숫자도 또박또박 명료함	밝고 경쾌한 느낌	**들으려는 약간의 노력 필요**
3	보통	의미를 모르고 하는 소리를 듣는 기분	감정이 메마른 듯한 딱딱한 어조	**들으려는 일반적인 노력 필요**
2	약간 나쁨	쉬어야 하는 구간에서 쉬지 않고 읽음	비음에서 소리가 왜곡됨	**들으려는 상당한 노력 필요**
1	나쁨	문장간 끊어 읽는 구간이 비정상	억양이 제멋대로여서 듣기 불편함	**노력을 기울여도 알아듣기 곤란**
AVE				평균

- 상담에 참여한 상담사(테스터)에 TTS 만족도 설문을 하고, 그 평균값으로 평가함

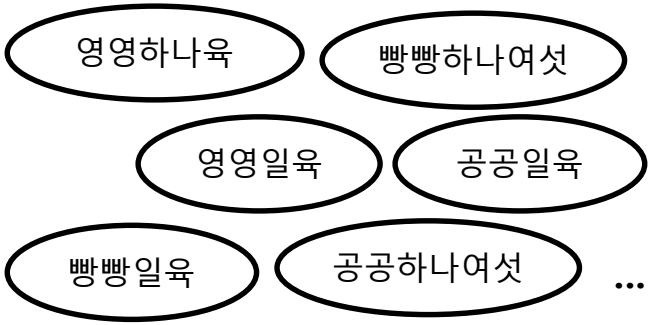
5. AICC 사례 > 다. AICC 모델 성능 테스트 유형
3). NE(Named Entity) 정확도

NE-표제어*주1) 처리 정확도

*NE(Named Entity-개체명) 고유한 의미를 가지는 명사

□ 표제어 복원 - Inverse Text Normalization(ITN)

- ○ 비표준어휘(Non-Standard word)는 전화번호, 시간, 금액 같은 숫자 또는 고유명사로 이루어진 정보
- ○ 표제어 복원은 **비 표준어휘를 구어체(spoken form)가 아닌 문어체(written form) 형태의 결과로 변환하는 과정**
- ○ 참고로 문어체 => 구어체 변환은 Text Normalization과정으로 TTS전처리, STT독음(발음열)에 사용



숫자 '0016'

$$ACC = \frac{1}{N}\sum_{i=1}^{N} I(Z_i = Y_i)$$

$$I(true) = 1; I(false) = 0;$$

□ 평가 방식

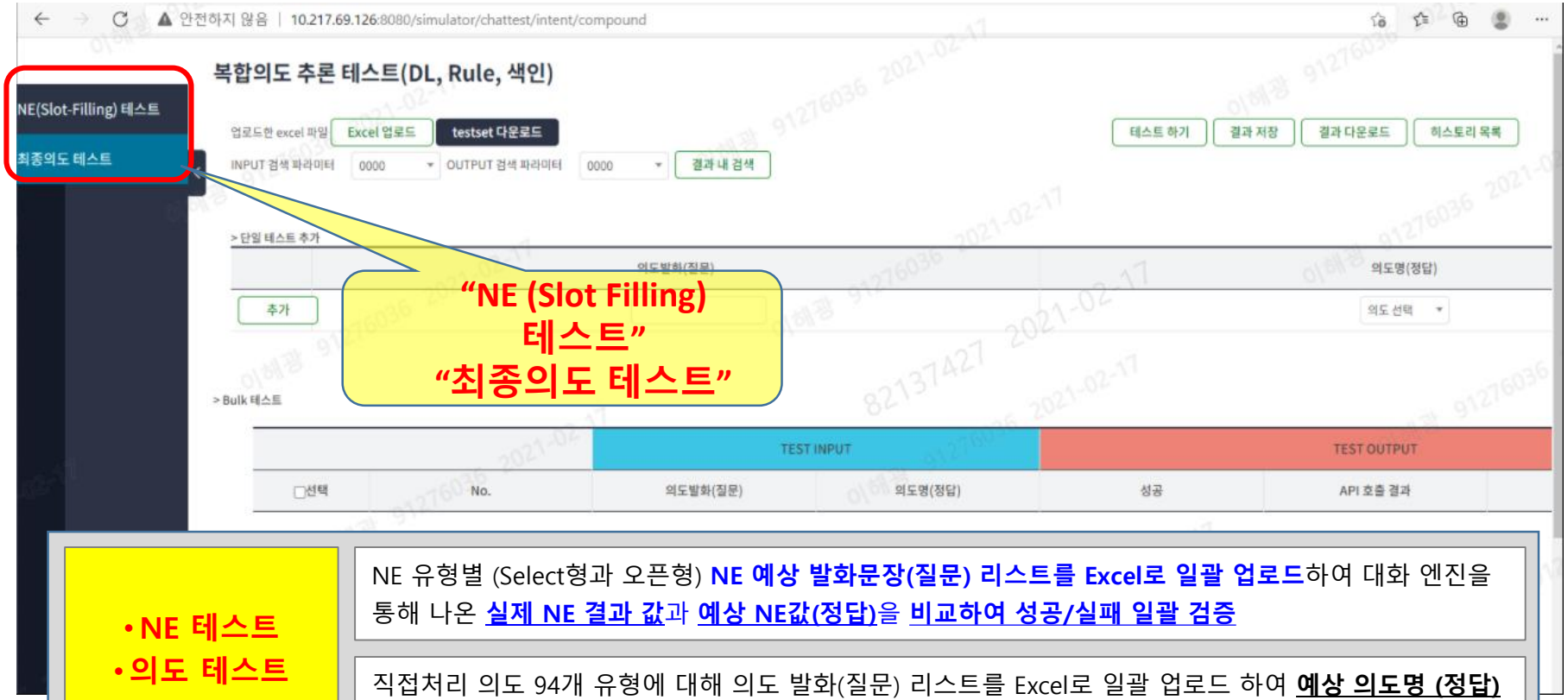
- ○ **비 표준어휘 대상의 표제어 복원 정확도를 구하여 가상상담에 중요한 전화번호, 시간 등 다양한 고객 발화 패턴을 잘 처리하는지 측정**
- ○ 정확도 측정을 위해서 범주(전화번호, 시간 등) + ITN 정답 데이터 준비
- ○ **시스템 출력 결과와 정답의 각 범주 별 내용을 비교하여 정확도(F1) 측정**
- ○ 정답 레이블링(범주화)의 어려움을 감안하여 각 차수별로 일부만 취하여 시행
- ○ 고객 발화의 순수한 표제어복원능력만 체크
- ○ 영어의 경우 전화번호(95%), 시간(75%), 날짜(99%), 금액(97%), 숫자(85%) 가 SOTA*주2)(State of the art)

※ **주1) 표제어 : '기본 사전형 단어'라는 의미**로 표제어 추출은 단어들이 다른 형태를 가지더라도, 그 뿌리 단어는 같음 (예, am, are, is의 표제어는 be
주2) SOTA : '현재 최고 수준의 결과'를 가진 모델로, 현재 수준에서 가장 정확도가 높은 모델을 의미

5. AICC 사례 > 다. AICC 모델 성능 테스트 유형

3). NE(Named Entity) 정확도

✓ **Test Bed : http://10.100.101.126:8080/simulator/chattest/NE/slotFilling**



- **NE 테스트**
- **의도 테스트**

NE 유형별 (Select형과 오픈형) **NE 예상 발화문장(질문) 리스트를 Excel로 일괄 업로드**하여 대화 엔진을 통해 나온 **실제 NE 결과 값**과 **예상 NE값(정답)**을 **비교하여 성공/실패 일괄 검증**

직접처리 의도 94개 유형에 대해 의도 발화(질문) 리스트를 Excel로 일괄 업로드 하여 **예상 의도명 (정답)**과 대화 엔진을 통해 나온 **실제 결과 값을 비교하여 성공/실패 일괄 검증**

5. AICC 사례 > 다. AICC 모델 성능 테스트 유형

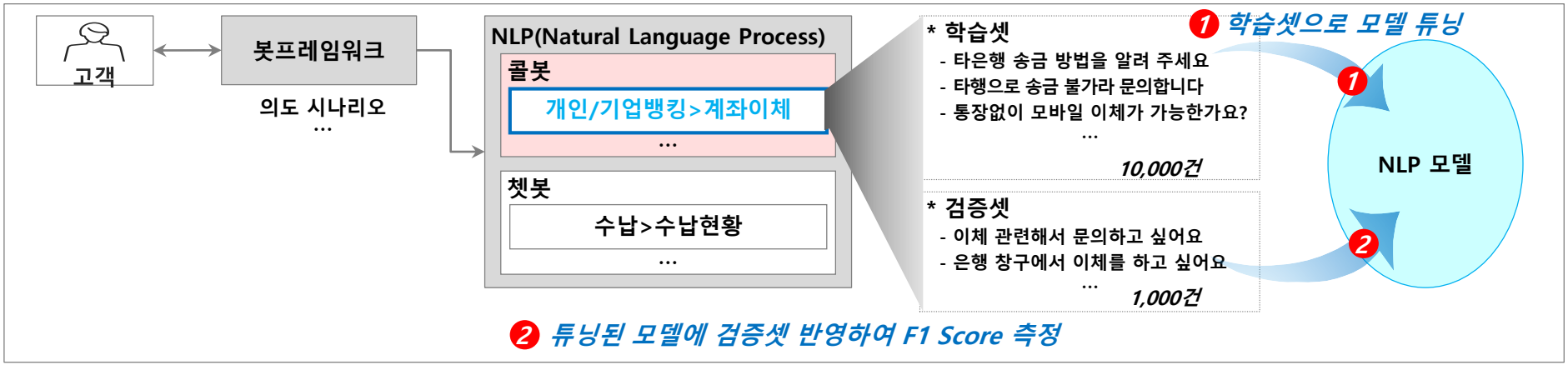
4). 대화분류 정확도_상담의도분류 정확도

NLP 상담의도분류 정확도 : 100% (목표 100%)

■ 평가 개요

- 구현단계에서 수집한 지식그래프와 콜봇 시나리오에 따른 의도, 개체명 데이터를 활용하여 콜봇, 챗봇 학습 진행함
- 학습셋을 통해 모델을 튜닝하고 검증셋을 통해 평가 진행, 실제 서비스에 즉시 투입 가능한 수준인 90% 이상 달성이 되도록 튜닝 진행함

■ 평가 방법



■ 평가 등급

점수범위	품질해석	설명
0.0~0.6	낮음	의도 파악 보다는 무작위 추측에 가까우며 비즈니스 모델로 사용하기 어려운 수준
0.6~0.69	미흡	잦은 오분류로 인해 사용자가 불편을 느끼는 수준이며 학습데이터 보강이 필요한 수준
0.7~0.79	보통	문맥이 모호하거나 단어가 중복되는 의도에서 혼동을 보이는 수준, 오답 시 '재질의' 로직 필요
0.8~0.89	우수	전반적으로 안정적인 성능을 보이며 상용화 모델의 일반적인 목표치
0.9이상	**매우우수**	**인간 수준에 근접한 성능이며 실제 서비스에 (챗봇, 콜봇 등) 즉시 투입 가능한 수준**

■ 의도 유형

의도 유형	의도명	상담 스킬셋
직접처리의도	이용정지_일정_문의	billingIBCenter
호전환의도	유선_요금_업무	broadbandGeneral
호전환의도	유선_인터넷전화요금제_업무	CRGDirect
호전환의도	유선_부가서비스_업무	dataTechnical
호전환의도	유선_제휴할인_업무	KTdotcom
직접처리의도	가설_중_연락처_변경	broadbandGeneral
…	…	…

5. AICC 사례 > 다. AICC 모델 성능 테스트 유형

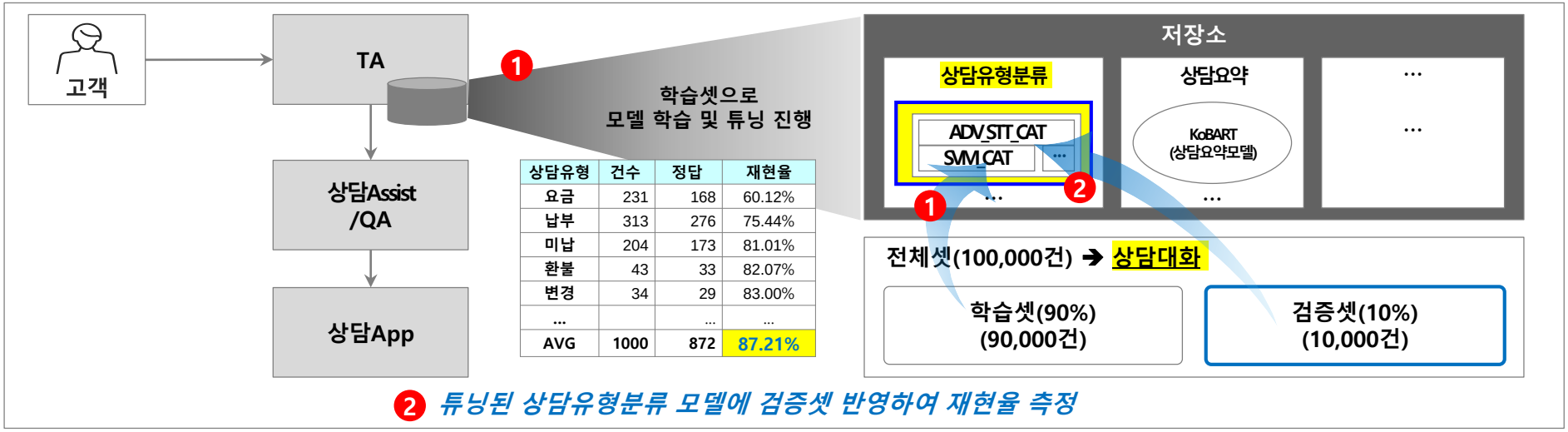
4). 대화분류 정확도_상담유형분류 재현율

TA(Text Analysis) - 상담유형분류 재현율

■ 평가 개요

- TA 상담유형분류 재현율 및 TA 상담요약 유사도는 고객이 제공한 상담대화 데이터의 학습셋으로 튜닝된 모델에 검증셋을 반영하여 점검 진행함

■ 평가 방법



상담유형	건수	정답	재현율
요금	231	168	60.12%
납부	313	276	75.44%
미납	204	173	81.01%
환불	43	33	82.07%
변경	34	29	83.00%
...		...	...
AVG	1000	872	87.21%

■ 평가 등급

점수범위	품질해석	설명
0.0~0.6	미흡	분류 능력이 낮아 반드시 추가 학습 필요
0.6~0.69	보통	분류를 정답으로 예측하지 않는 경우가 있어 모델 튜닝(학습) 필요
0.7~0.79	양호	일부 정답으로 예측하지 못하는 경우가 있을 수 있음
0.8~0.89	**우수**	**대부분의 정답을 예측함. 실무 적용에 충분한 수준**
0.9이상	매우우수	실제 정답을 거의 다 예측함

■ 평가 결과

구분	검증 건수 및 검증 결과
대상 상담유형(건)	1,521
학습셋(건)	100,000
검증셋(건)	10,000
검증 재현율(상담유형분류)	87.21%

SAMPLE

5. AICC 사례 > 다. AICC 모델 성능 테스트 유형

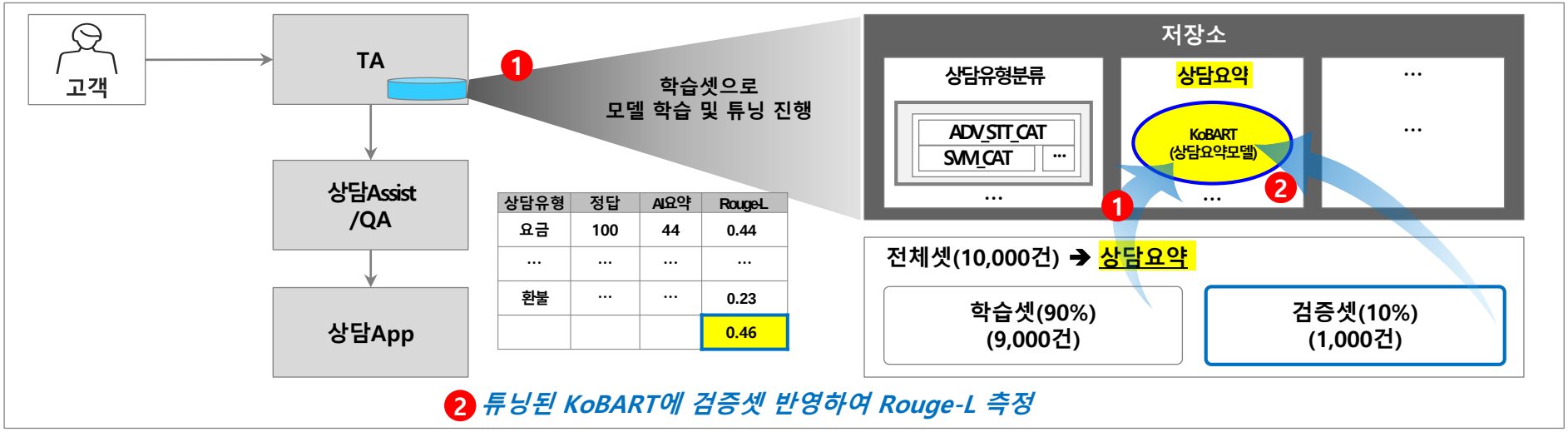
4). 대화분류 정확도_상담요약 유사도

TA(Text Analysis) - 상담요약 유사도

■ 평가 개요

- 상담유형분류는 **최종 재현율이 80% 이상 되도록 튜닝**하고, **상담요약은 중요한 키워드와 문맥 대부분을 포함**하여 사람이 이해 가능한 점수인 **0.35 이상 되도록 튜닝함**

■ 평가 방법



상담유형	정답	AI요약	Rouge-L
요금	100	44	0.44
…	…	…	…
환불	…	…	0.23
			0.46

■ 평가 등급

점수범위	품질해석	설명
0.0~0.2	매우낮음	핵심 내용의 누락이 많고 관련 없는 문장이 많음
0.2~0.35	낮음	일부 겹치는 정보가 있으나, 요약이 부정확하거나 누락이 있음
0.35~0.5	**보통이상**	**중요한 키워드와 문맥 대부분을 포함, 사람이 이해 가능한 수준**
0.5~0.7	좋음	핵심 내용을 잘 요약했으며 표현도 유사함
0.7이상	매우좋음	원문과 거의 동일(복사) 수준

■ 평가 결과

구분	검증 건수 및 검증 결과
전체셋(건) (B)+(C)	10,000
학습셋(건) (B)	9,000
검증셋(건) (C)	1,000
검증 평균 Rouge(상담요약)	0.48

SAMPLE

Contents

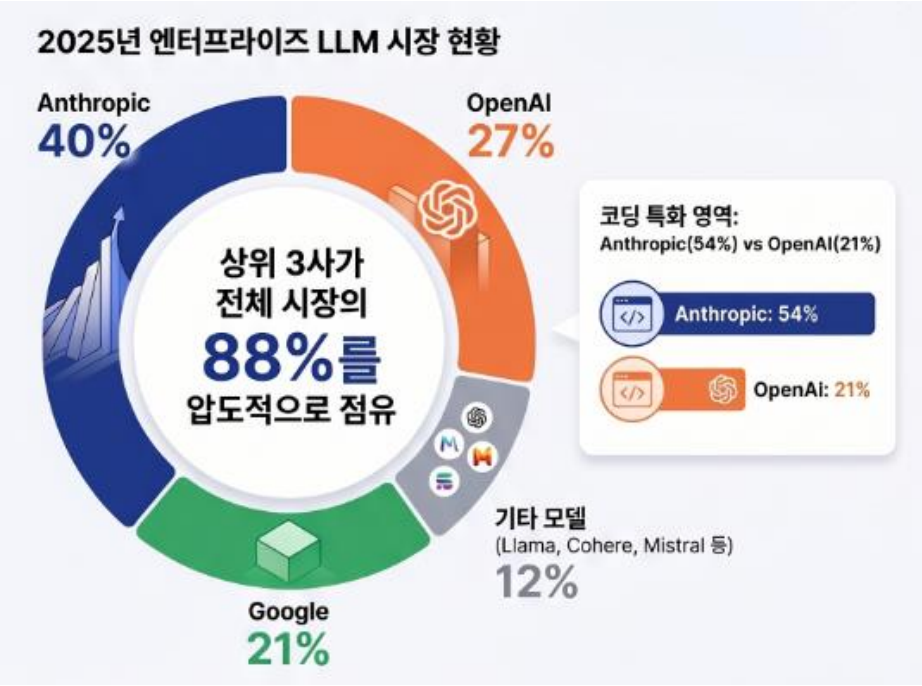
1	인공지능 이해
2	품질과 테스트 이해
3	AI 발전과 프로젝트 활용
부록 #1	AI 발전 트랜드

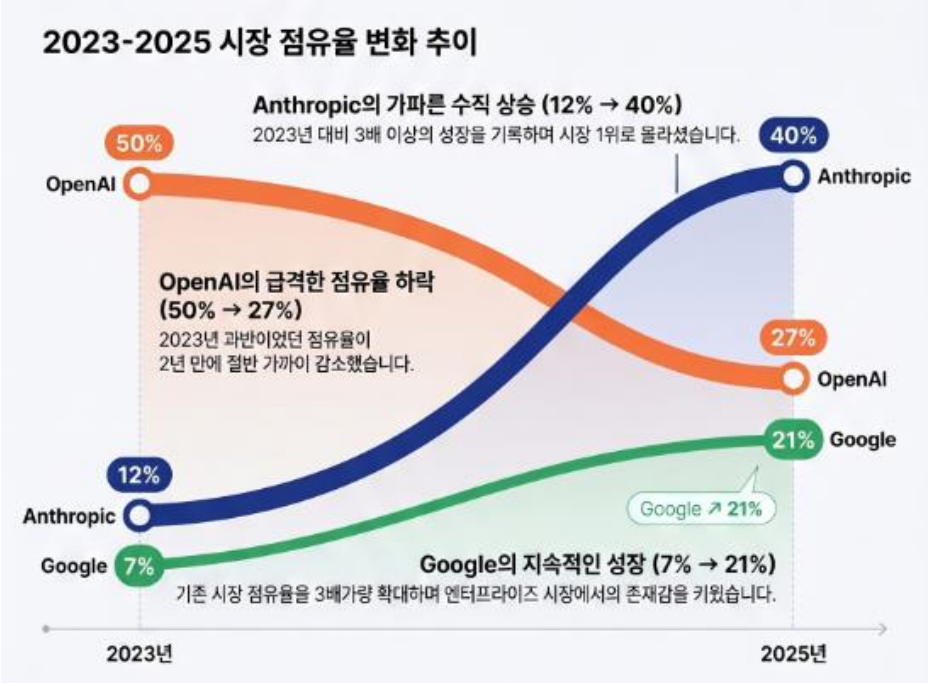
부록 #1. AI 발전 트랜드

가. LLM 점유율 추이

엔터프라이즈 LLM 시장의 지각변동

Anthropic의 급부상





※ 데이터 출처 : Menlo Ventures

부록 #1. AI 발전 트랜드

나. AI Agent Framework 현황

Quick-Reference Comparison: AI Agent Frameworks in 2026

Evaluated across 7 enterprise deployments in healthcare, logistics, fintech, and e-commerce (2025–2026)

FRAMEWORK	BEST FOR	PRODUCTION	LEARNING CURVE	HUMAN-IN-LOOP	COST CONTROL	BACKING
LangGraph	Complex stateful workflows, compliance-heavy verticals	★★★★★	Steep	Native	High	LangChain (126K stars)
CrewAI	Fast prototyping, role-based collaboration, content pipelines	★★★★	Low	Limited	Medium	CrewAI Inc.
AutoGen 2.0	Research pipelines, Azure environments, async workflows	★★★★	Moderate	Proxy pattern	Low	Microsoft Research
OpenAI Agents SDK	OpenAI-native apps, rapid agent prototyping	★★★	Low	Basic	Medium	OpenAI
Anthropic Agent SDK	Code agents, high-accuracy tasks, safety-critical workflows	★★★★	Moderate	Built-in	High	Anthropic
Google ADK	Vertex AI environments, multimodal agents	★★★	Moderate	Configurable	Medium	Google DeepMind
LlamaIndex Workflows	RAG-heavy applications, knowledge retrieval agents	★★★★	Moderate	Event-driven	High	LlamaIndex
Intuz Agentic Framework	Enterprise multi-agent orchestration, self-learning systems	★★★★★	Managed	Full	High	Intuz (16+ yrs, 1,700+ projects)

※ 데이터 출처 : Medium.com

다. AI Agent 오픈소스 생태계 (1/2)



※ 데이터 출처 : 넥스트플랫폼: AI & Tech Logs
https://nextplatform.net/next-agents-ai-agent-framework-top5/

다. AI Agent 오픈소스 생태계 (2/2)

분야별 TOP5 에이전트

카테고리	주요 프로젝트 및 특징
에이전트 구현 프레임워크	**OpenClaw**(멀티채널 메시징), **Hermes Agent**, **LangChain**, **MetaGPT**, **Autogen**
에이전트 기반 소프트웨어 개발	**OpenCode**, **OpenHands**(구 OpenDevin), **MetaGPT**, **Cline**, **Aider**
에이전트 기반 테스트 및 평가	**Arize-Phoenix**, **Manifest**(비용 추적), **EvoAgentX**, **Open-RAG-Eval**, **Voice Lab**
에이전트 기반 리서치	**Storm**(지식 큐레이션), **GPT Researcher**, **AI Scientist**, **DeepAnalyze**, **AIDE**
에이전트 기반 지식 관리	**Private GPT**, **Local GPT**, **Screenpipe**(화면/오디오 기록), **Hindsight**, **LLocalSearch**
에이전트 기반 자동화	**Phidata**, **AgentGPT**, **XAgent**, **Steel Browser**(AI 전용 브라우저), **OpenAgents**
에이전트 기반 게임 및 시뮬레이션	**Voyager**, **Camel-AutoGPT**, **MiroShark**(군집 지능), **SkyAGI**, **Enclave**

AI 에이전트 프레임워크 TOP5

- OpenClaw : LLM을 지속적이고 선제적인 개인 AI 에이전트로 전환하는 오픈소스 AI 에이전트 프레임워크. 멀티채널 메시징(Signal, Telegram, Discord, WhatsApp), cron 스케줄링, 메모리 시스템, MCP 통합, 스킬 플러그인, 서브 에이전트 생성, 브라우저 자동화 지원.
- Hermes Agent : 사용자와 함께 성장하는 에이전트
- LangChain : 복잡한 상태 기반 워크플로우의 기본 선택. 분기·재시도·HITL(휴먼인더루프) 제어가 명시적.
- MetaGPT : 멀티 에이전트 메타 프로그래밍 프레임워크: 한 줄의 요구사항을 입력하면 PRD, 설계, 작업, 저장소, CI를 반환
- Autogen : 차세대 대규모 언어 모델 애플리케이션 지원

AI 테스트 에이전트 TOP5

- Arize-Phoenix : 에이전트 테스트, 평가, 관측 가능성을 위한 오픈소스 라이브러리
- Manifest : AI 에이전트를 위한 오픈소스 실시간 비용 관측 플랫폼. 로컬 우선 대시보드로 토큰, 비용, 메시지, 모델 사용량 추적. 28개 이상 LLM 모델, OTLP 수집, 셀프 호스팅 지원
- EvoAgentX : AI 에이전트의 자기 진화 생태계를 구축하며, 에이전틱 워크플로 평가 및 진화를 위한 자동화 프레임워크 제공
- Open-RAG-Eval : 정답(golden answer)이 필요 없는 오픈소스 RAG 평가 프레임워크. AI 에이전트에 연결된 RAG 도구(Agentic RAG) 성능 평가에 사용 가능
- Voice Lab : 언어 모델, 프롬프트, 에이전트 페르소나 전반에 걸친 음성 에이전트 테스트 및 평가를 위한 종합 프레임워크

※ 데이터 출처 : 넥스트플랫폼: AI & Tech Logs
https://nextplatform.net/next-agents-ai-agent-framework-top5/

부록 #1. AI 발전 트랜드

라. 주요 통계 - 인간 기준선 대비 주요 AI 성능

벤치마크를 추월하는 역량. 프런티어 모델이 다수 항목에서 인간 기준선을 넘어섰다. 인간 전문 가에게 유리하게 설계된 Humanity's Last Exam에서도 1년 만에 30%p 상승했다.

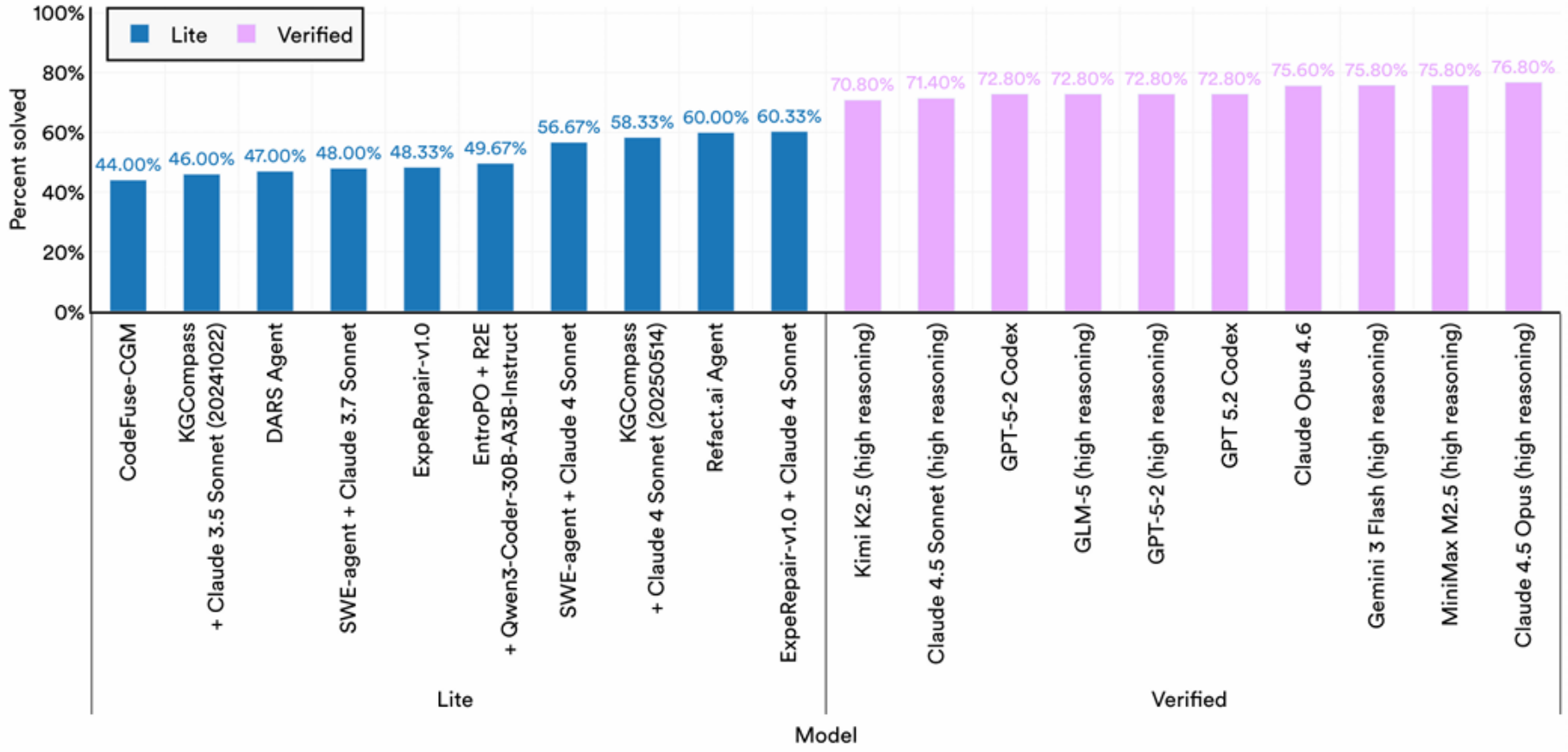


※ 데이터 출처 : 2026 AI Index report

부록 #1. AI 발전 트랜드

라. 주요 통계 - 소프트웨어 수정 능력 급상승

소프트웨어 수정 능력 급상승. SWE-bench에서 실제 코드 수정 과제 해결률이 1년 만에 인간 기준의 60%→거의 100% 수준으로 올랐다.

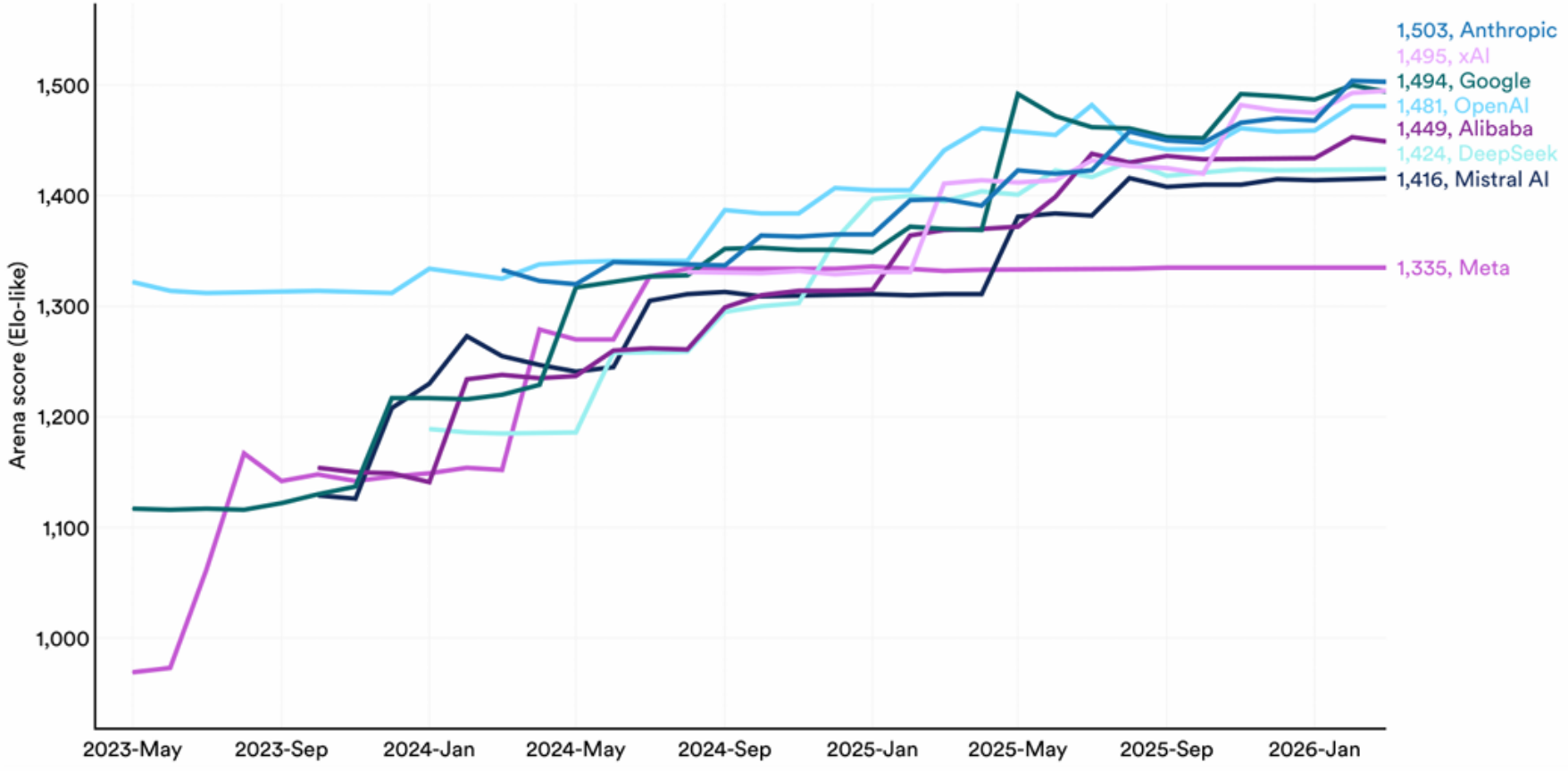


※ 데이터 출처 : SWE-bench Leaderboard, 2026

부록 #1. AI 발전 트랜드

라. 주요 통계 - 주요 모델별 성능 수렴

상위 모델 성능 수렴. 상위 기업의 평가 점수 차이가 적어졌다. 모델 성능이 비슷해지면서 경쟁의 초점이 비용·신뢰성·도메인 특화로 이동하고 있다.

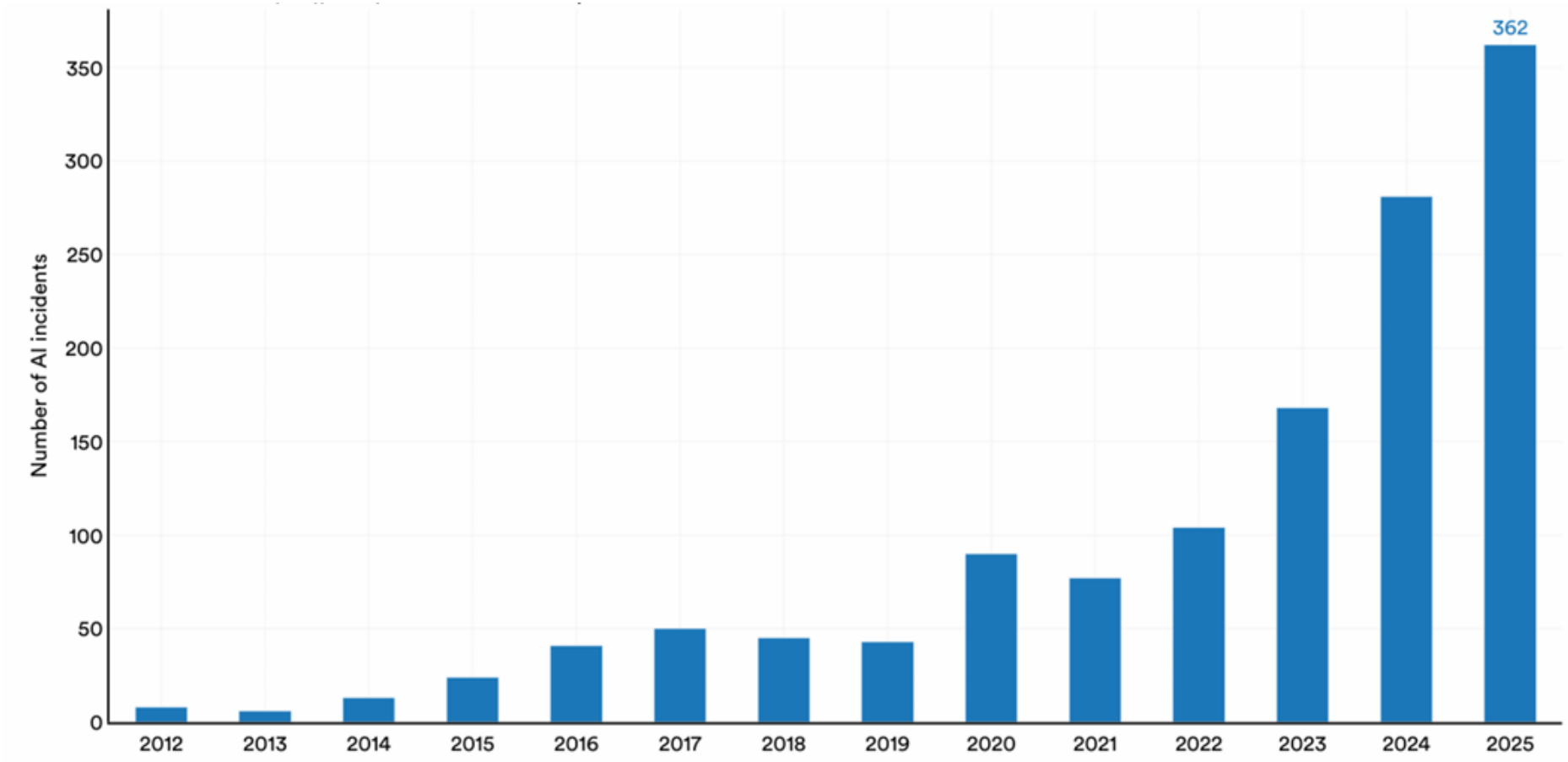


※ 데이터 출처 : Arena. 2026

부록 #1. AI 발전 트랜드

라. 주요 통계 - AI 사고 급증

문서화된 AI 사고가 2024년 233건에서 2025년 362건으로 늘었다. 역량 벤치마크 보고는 보편화됐지만 책임있는 AI 벤치마크 보고는 여전히 드물다.

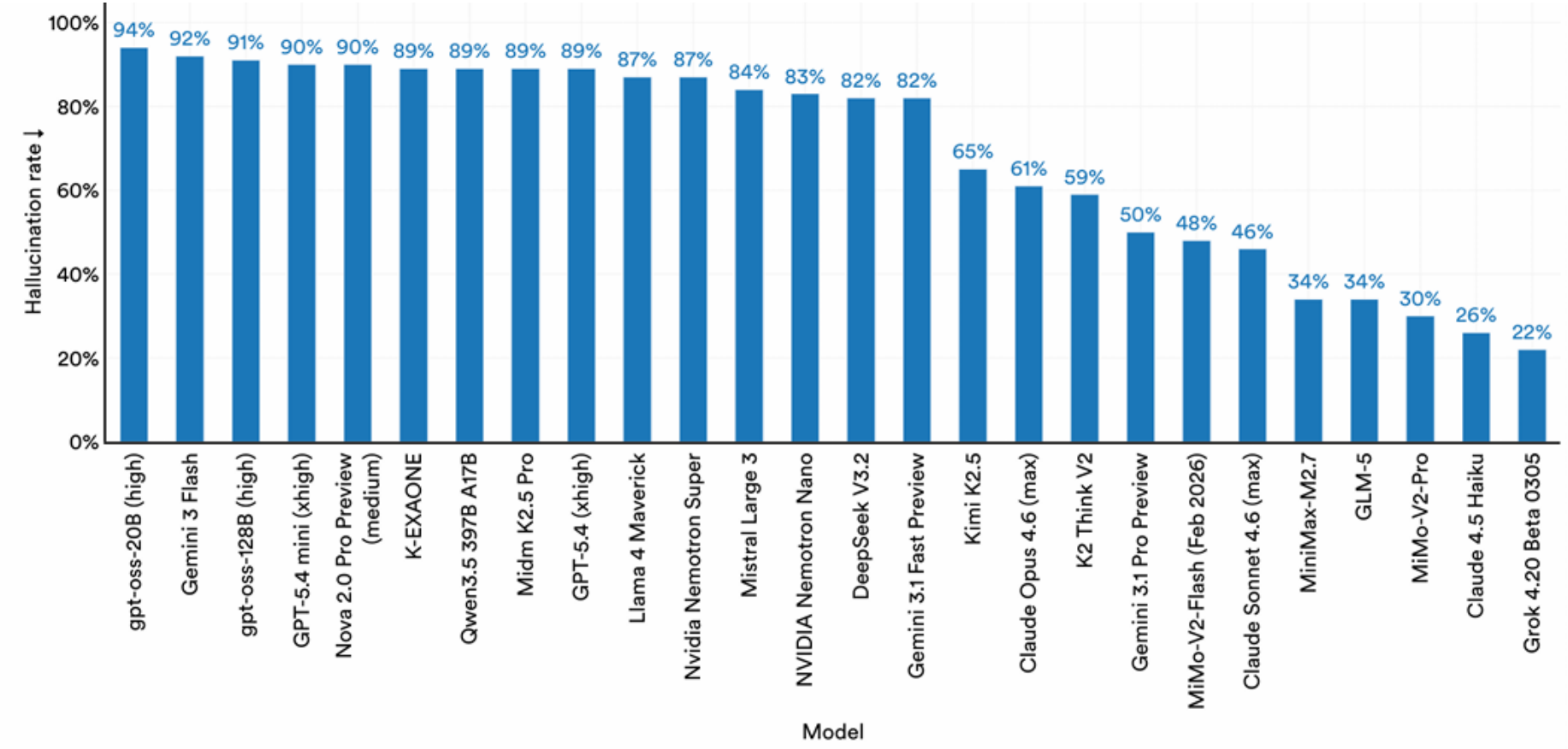


※ 데이터 출처 : AI Incident Database (AIDD), 2025

부록 #1. AI 발전 트랜드

라. 주요 통계 - 환각 문제

지식과 믿음을 구분 못 함. 26개 상위 모델의 환각률은 22~94%로 편차가 크다. 거짓 진술을 '사용자의 믿음'으로 제시하면 정확도가 붕괴한다.

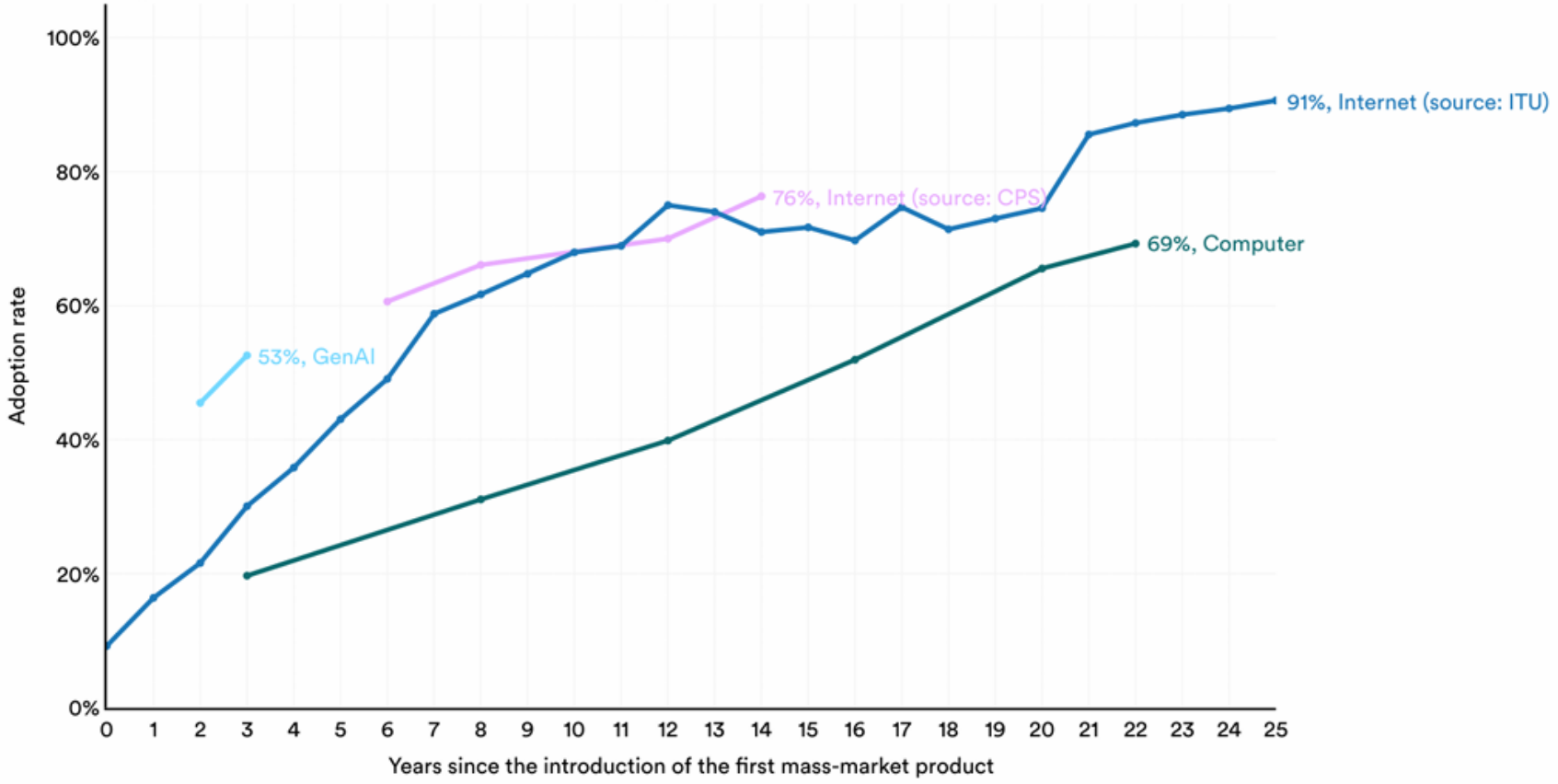


※ 데이터 출처 : Artificial Analysis, 2026

부록 #1. AI 발전 트랜드

라. 주요 통계 - 생성형 AI 도입 속도

PC·인터넷보다 빠른 도입. 생성형 AI는 3년 만에 53% 보급에 도달했다.
(50% 도달하는데, 인터넷은 6년, PC는 15년 소요됨)
다만 국가별 편차가 크고(미국 28.3%로 24위) GDP와 강한 상관을 보인다



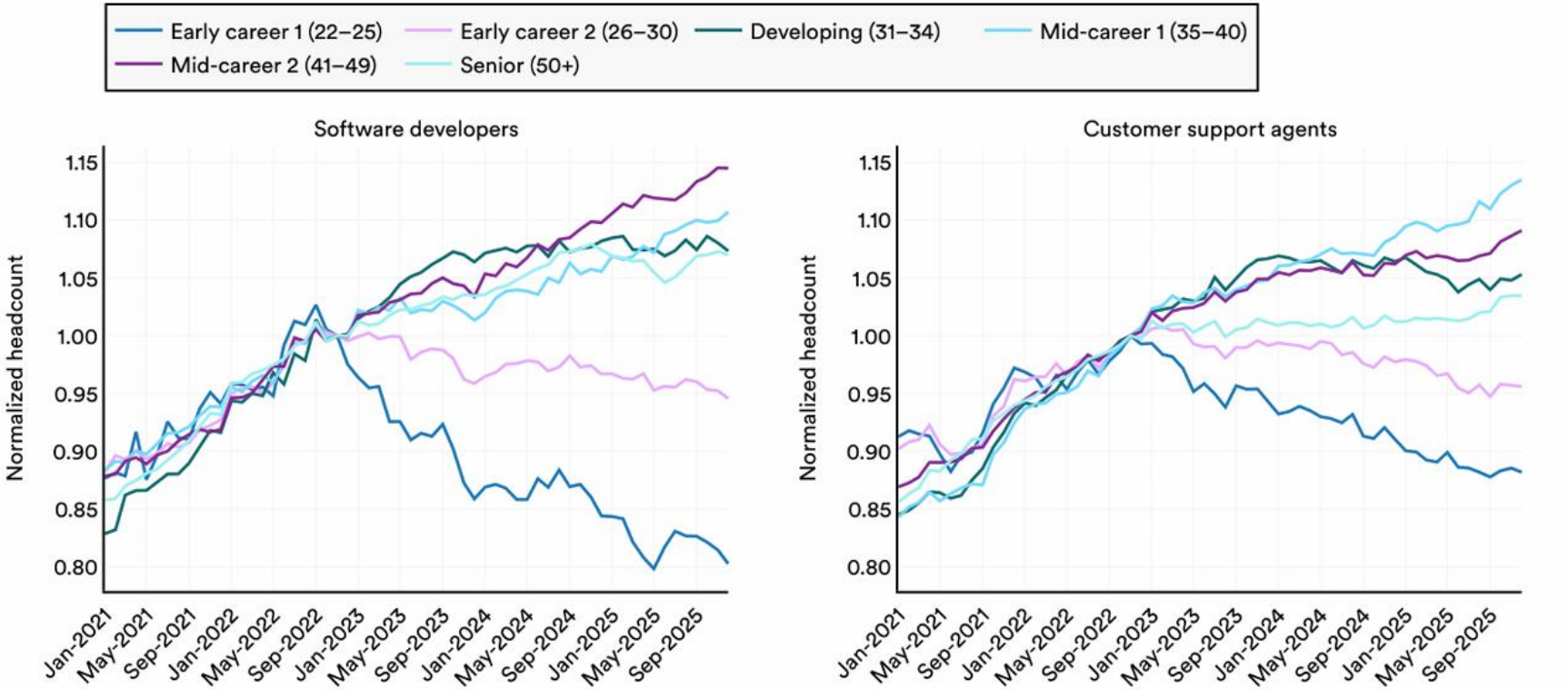
※ 데이터 출처 : The Project on Workforce at Harvard, 2025

부록 #1. AI 발전 트랜드

라. 주요 통계 - 젊은 개발자 고용 감소

AI 생산성 향상이 뚜렷한 소프트웨어 개발에서, 22~25세 미국 개발자 고용이 2024년 대비 약 20% 줄었다(고령 개발자는 계속 증가).

인력/조직 핵심 : 조직 도입률 88%(AI 에이전트는 한 자릿수) · 매출 급증하나 연산·인프라 비용도 기록적(Google capex 1,500억 달러+) · 생산성 향상은 구조화 업무서 최대(고객지원 14~15%, 개발 26%, 마 케팅 50%) · 조직 1/3이 1년 내 인력 감축 예상 · 중국이 산업로봇 54% 설치



※ 데이터 출처 : Brynjolfsson et ai., 2025

마. AI 활용 프로젝트 생산성_AI 도입 현황

한눈에 보는 AI 도입 현황

개발 현장의 AI 사용은 보편화 단계 — 도입은 넓지만, 신뢰와 성과 전환은 별개의 과제



90%

AI 도입률

응답 개발자의 90%가 업무에 AI 사용

DORA 2025



46%

AI 생성 코드 비중

평균 46%, Java 프로젝트는 최대 61%

GitHub



80%+

생산성 향상 체감

80% 이상이 생산성이 올랐다고 응답

DORA 2025



2시간

일평균 AI 사용

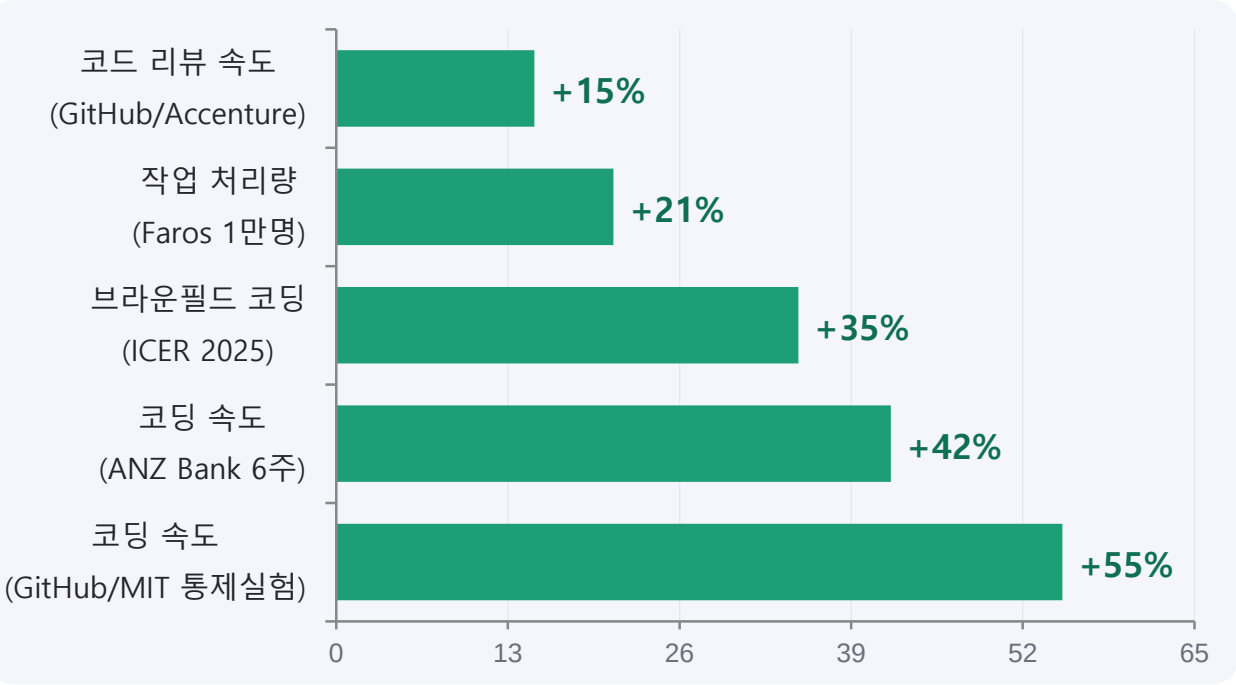
하루 약 2시간(근무의 1/4)을 AI와 작업

DORA 2025

마. AI 활용 프로젝트 생산성_영역별 생산성 향상

영역별 생산성 향상

정형·신규 작업일수록 효과가 크다 — 단위: % (작업 속도 향상 또는 처리량 증가)



PR 사이클 타임 단축

9.6일 → 2.4일

Pull Request 처리 기간
약 75% 감소 (GitHub 연구)

읽는 법

수치는 연구마다 측정 방식이 다릅니다.
통제실험의 '속도'와 텔레메트리의 '처리량'은 직접 비교가 아니라 **영역별 효과 크기의 참고치**로 보십시오.

마. AI 활용 프로젝트 생산성_생산성의 역설

① 숙련자는 오히려 느려졌다 ⚠

METR 무작위 통제실험(RCT) · 평균 5년 다룬 자체 코드베이스에서 246개 실제 과제 수행 · arXiv :2507.09089

사전 예상

+24%

AI가 더 빠르게 해줄 것

사후 체감

+20%

끝낸 뒤에도 빨라졌다고 느낌

실제 측정

−19%

오히려 시간이 더 걸림

인식과 실제의 격차 39%p. AI는 그린필드·반복 작업엔 강하지만, 복잡한 레거시·높은 품질 기준 작업에서는 검토·검증 비용이 실행 절감분을 넘어설 수 있습니다.

마. AI 활용 프로젝트 생산성_생산성의 역설

② 동반되는 품질 비용 ↑

Faros AI 텔레메트리 (2025–2026, 개발자 22,000명) · 개인 산출은 늘지만 하류에 비용이 쌓인다

+54%

개발자당 버그

이전 데이터셋 +9% 대비 급격한 가속

+441%

PR 리뷰 시간(중앙값)

PR 크기 +51%로 인지 부하·리뷰 지연

+243%

PR당 인시던트

변경 1건당 운영 장애 확률 3배 이상

처리량 (Throughput) ↑

더 많은 코드·기능을 더 빠르게 출시 (2024년 대비 긍정 반전)

안정성 (Stability) ↓

변경량 증가가 장애로 — 통제 시스템 없으면 가속이 불안정으로

→ **자동화 테스트 · 버전관리 · 빠른 피드백 루프가 가속을 성과로 전환하는 전제 조건**

마. AI 활용 프로젝트 생산성_시사점

시사점 — SI 프로젝트 관점

"AI 생산성은 영역과 맥락에 따라 정반대로 나타난다" — 과대평가를 피하는 세 가지 원칙

1

정형·신규 작업에 강하고, 복잡한 레거시엔 약하다

보일러플레이트·신규 코드 작성은 통제실험에서 42–55% 향상. 그러나 성숙한 코드베이스의 숙련 작업은 오히려 느려질 수 있어, 적용 영역을 선별해야 합니다.

2

개인 생산성 ≠ 조직 성과

개인 산출(작업·PR)은 크게 늘지만, 통제 체계가 없으면 버그·리뷰 시간·인시던트 증가로 조직 전달 지표에서 상쇄됩니다. 효과는 시스템 성숙도에 비례합니다.

3

생산성 증가에 따른 안정성과 품질 확보 필요

코드와 기능을 더 빠르게 작성하지만, **안정성과 품질 검증에 대한 프로세스와 메트릭 관리가 더욱 중요**해 집니다.

출처: GitHub·MIT/Accenture 통제실험 · METR RCT(arXiv:2507.09089) · DORA 2025(n≈5,000) · Menlo Ventures · Faros AI · Stack Overflow Developer Survey 2025

Thank You!